

## MODULE 1

# JVM Architecture and Runtime Model

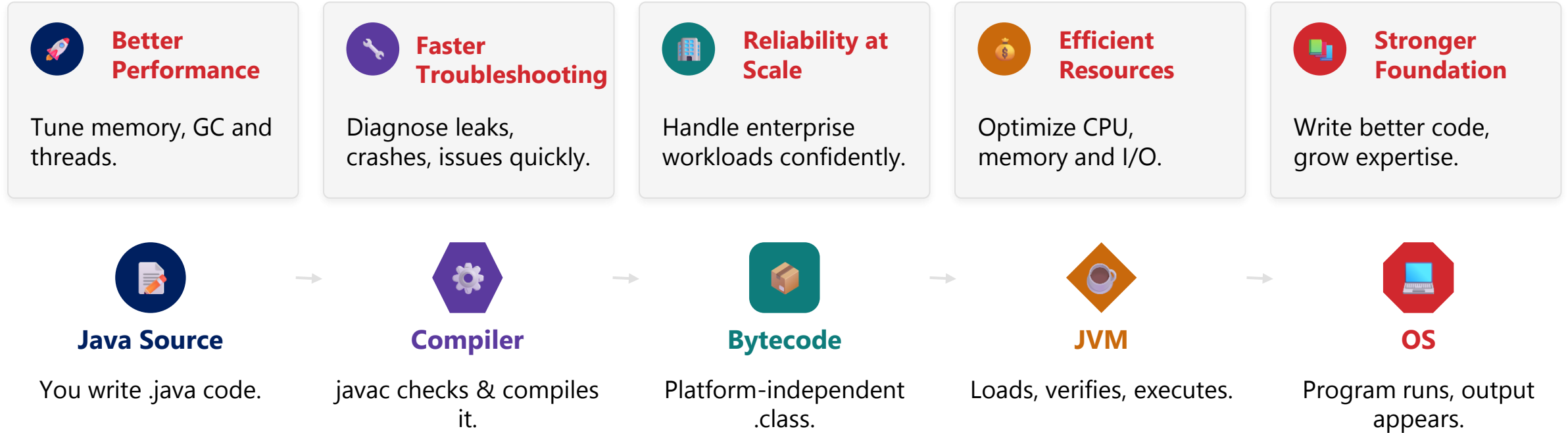
How your Java code comes to life — from source file, through bytecode, to a running program managed by the Java Virtual Machine.





# WHY UNDERSTANDING THE JVM MATTERS

*The JVM is the foundation of every Java application — understanding it builds applications that are reliable, efficient, and production-ready.*



- Real-world impact: handle millions of requests, avoid outages and downtime, deliver a consistent user experience, and scale with confidence.

**KEY TAKEAWAY** The JVM is the bridge between your code and the operating system — master it to build software that performs and scales.

# MODULE 1 LEARNING OBJECTIVES

*By the end of this module, you will understand the core components, memory model, and execution flow of the JVM.*

- Explain the role of the JVM in the Java platform and its importance in application execution.
- Identify and describe the major components of the JVM and how they work together.
- Understand the compilation process and the structure of Java bytecode.
- Describe the class loading mechanism and the lifecycle of a class in the JVM.
- Explain the JVM memory model, including Heap, Stack, Metaspace and Method Area.
- Understand garbage collection concepts and object lifecycle management.
- Analyze JVM performance factors and common memory-related issues.
- Use JVM tools and monitoring concepts to diagnose and troubleshoot runtime problems.

**KEY TAKEAWAY** This module builds the foundation every later module — memory, GC, performance — depends on.

# ENTERPRISE SCENARIO: DIAGNOSING A PRODUCTION MEMORY ISSUE

*A customer management application is experiencing high memory usage and frequent OutOfMemoryError in production, impacting users.*



Users



Load Balancer



App Servers



PostgreSQL Database

## SYMPTOMS OBSERVED



**Heap Usage 95%+** Frequent Full GC taking several seconds.



**Slow Threads** OutOfMemoryError appears in application logs.



**Timeouts** Service interruptions; users see errors and slow responses.

*Business impact: Lost transactions, unhappy customers, SLA breaches.*

**KEY TAKEAWAY** Learning outcome: understand how JVM knowledge helps identify the root cause and resolve memory issues in production.

# HOW JVM KNOWLEDGE HELPS

*Turning a production outage into a diagnosed, fixed, and prevented issue.*

- Understand the JVM memory model: Heap, Stack, Metaspace, etc.
- Identify the memory leak using heap dumps and memory analysis tools.
- Analyze object allocation and garbage collection behavior.
- Tune JVM parameters for optimal memory usage.
- Implement code fixes and prevent future memory issues.
- Restore application stability and improve performance.

**Business Impact: faster issue resolution, improved application stability, better user experience, and reduced operational costs.**

**KEY TAKEAWAY** Deep understanding of JVM architecture, memory management and runtime behavior is essential to resolve real-world production issues.

# WHAT IS JAVA?

*A high-level, class-based, object-oriented language designed by Sun Microsystems (now Oracle), released in 1995.*



## Simple

- Easy to learn, clean syntax, reduces complexity.



## Object-Oriented

- Encapsulation, Inheritance, Polymorphism, Abstraction.



## Platform Independent

- Write Once, Run Anywhere (WORA) via the JVM.



## Secure

- Built-in security, automatic memory management, no pointer arithmetic.



## Robust

- Strong memory management, exception handling, multi-threading.



## High Performance

- JIT compilation and efficient runtime optimizations.

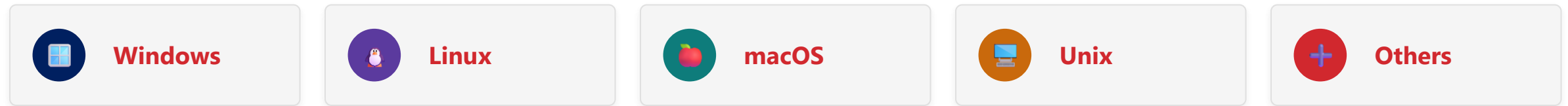
**KEY TAKEAWAY** These six characteristics are why Java has remained a dominant enterprise language for three decades.

# HOW JAVA WORKS

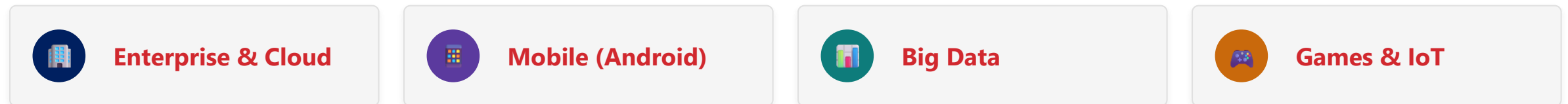
*The same journey from source code to execution repeats throughout this module.*



## WRITE ONCE, RUN ANYWHERE

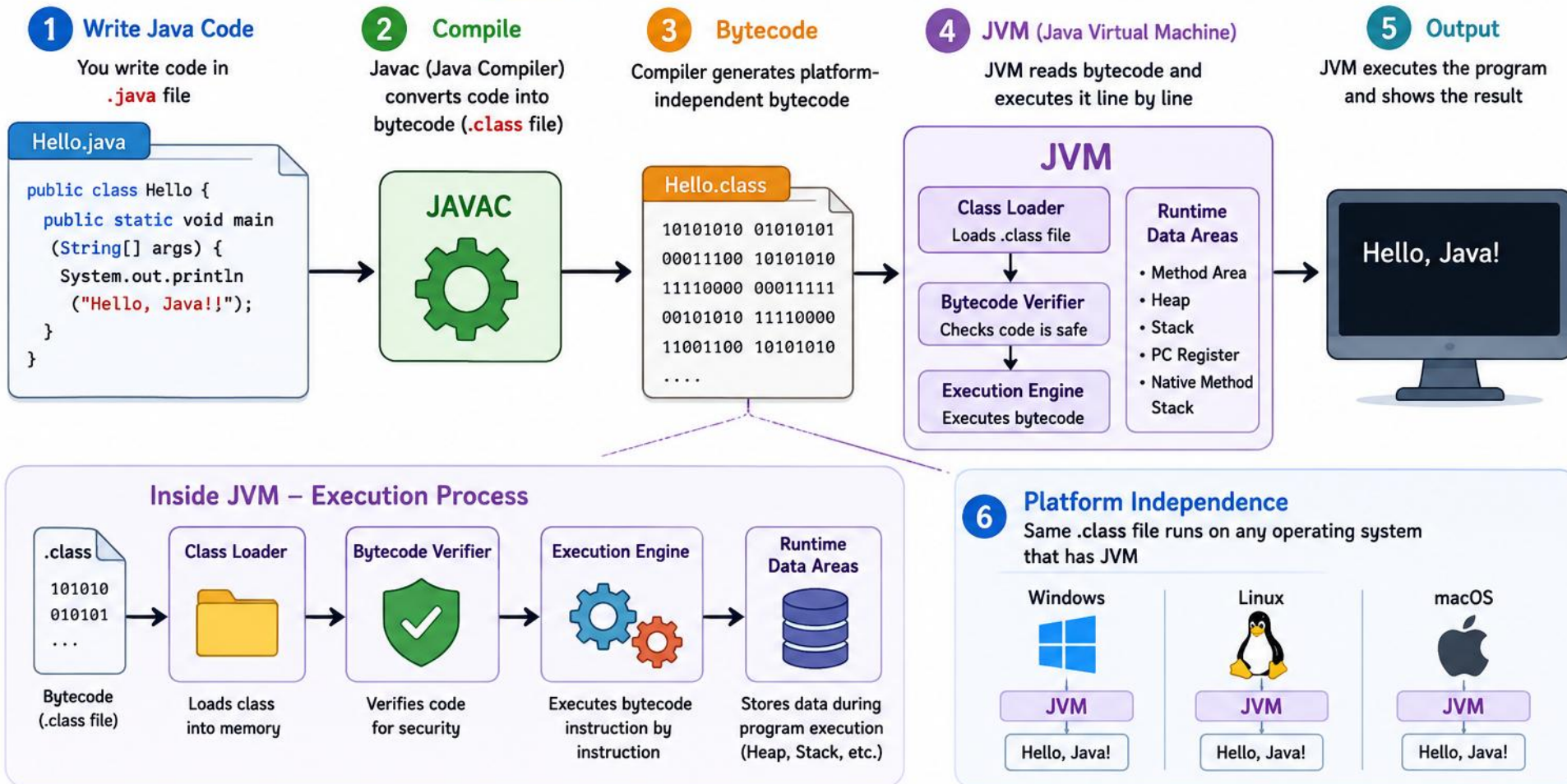


## WHERE IS JAVA USED?



**KEY TAKEAWAY** Bytecode is the platform-independent middle step that makes every one of these use cases possible.

# How Java Works



## Summary

Write Code (.java) → Compile (javac) → Bytecode (.class) → Run on JVM → Output

Java source code is compiled into bytecode, and JVM executes it.



# JAVA PLATFORM OVERVIEW

*A rich, secure, and reliable environment for building, deploying, and running enterprise applications.*



## Platform Independent

Write once, run anywhere.



## Secure

Built-in security & best practices.



## Robust

Memory mgmt, exceptions, typing.



## High Performance

JIT compiler, optimizations.



## Large Ecosystem

Rich libraries, frameworks, tools.

## JAVA SE — WHAT IT'S MADE OF



### Dev Tools & APIs

- javac, javadoc, JConsole, JMC, and more.



### Core Libraries (API)

- Collections, IO/NIO, Concurrency, JDBC, Networking.



### Java Virtual Machine

- Class Loader, Runtime Areas, Execution Engine, GC, Security Mgr.



### Operating System

- Windows, Linux, macOS, Unix, and others.

**KEY TAKEAWAY** The JVM is only one layer of the Java Platform — tools, libraries, and the OS all sit around it.

# JAVA PLATFORM EDITIONS

*Three editions of the platform, sized for three very different classes of application.*



## Java SE (Standard Edition)

- Foundation for desktop apps, small servers, core enterprise services.
- Core Java APIs, development tools.
- Desktop and server applications.



## Java EE (Enterprise Edition)

- Built for enterprise apps with distributed-system APIs.
- Web, EJB, JMS, JPA, Security.
- Scalability & high availability — now evolved as Jakarta EE.



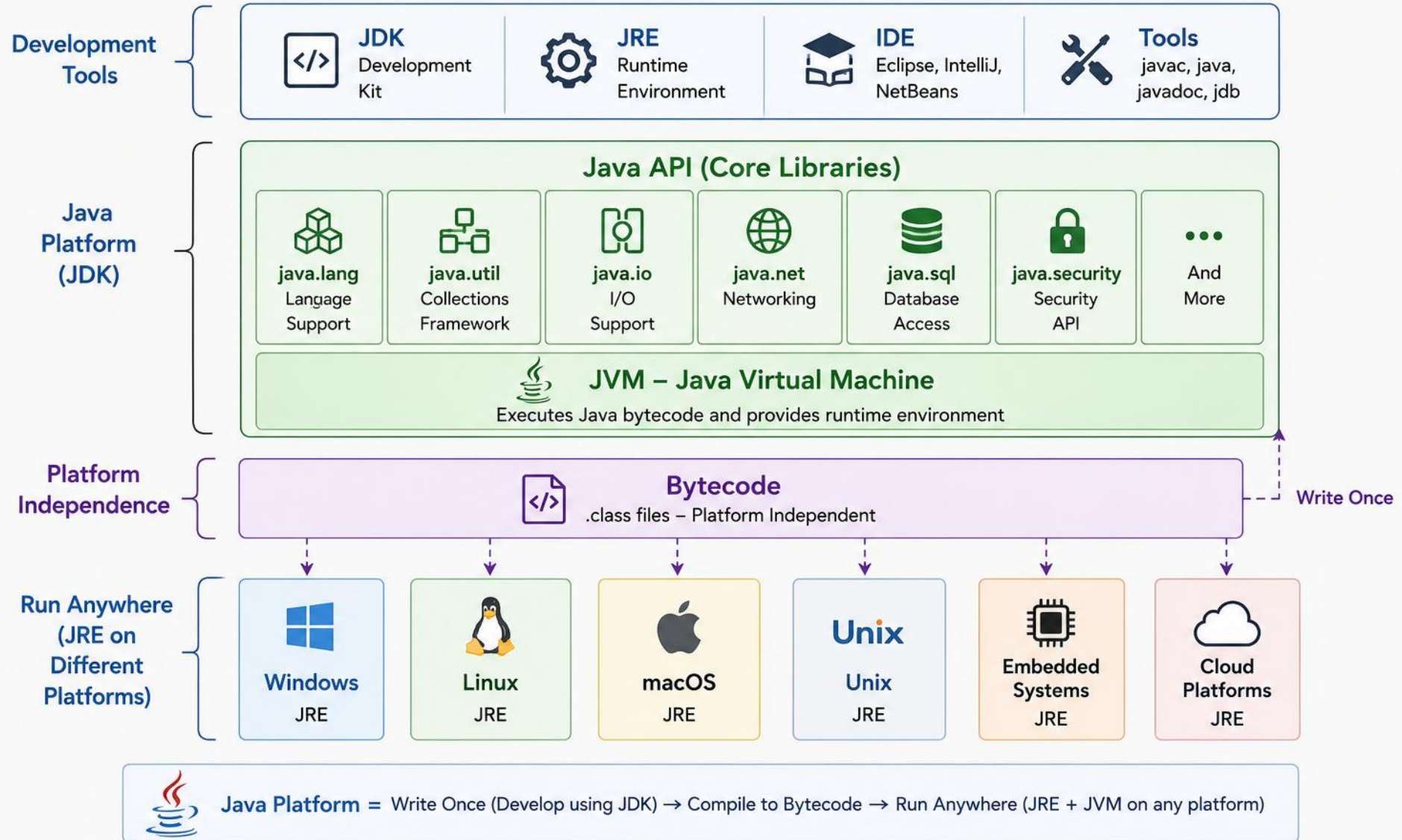
## Java ME (Micro Edition)

- For resource-constrained and embedded devices.
- Compact APIs optimized for small devices.
- IoT and embedded use cases.

**KEY TAKEAWAY** This bootcamp focuses on Java SE — the foundation every other edition builds on.

# JAVA PLATFORM

Write Once, Run Anywhere



# JDK VS JRE VS JVM

*Three essential components of the Java Platform, each with a specific purpose.*



## JDK

- Complete dev environment to build, test, package apps.
- Includes JRE + Dev Tools (javac, java, javadoc, jar, jdb, jlink).
- Used by: Developers.



## JRE

- Libraries, JVM, and components required to run apps.
- Includes JVM + Core Libraries + supporting files.
- Used by: End users, to run Java applications.



## JVM

- The heart of Java — executes bytecode, provides platform independence.
- Loads, verifies, executes bytecode; manages memory.
- Used by: JRE and Java applications.



JDK



JRE



JVM

**KEY TAKEAWAY** JDK is a superset containing JRE + dev tools; JRE provides everything to run an app; JVM is the engine that executes bytecode.

# JDK VS JRE VS JVM — IN PRACTICE

*A developer compiles with the JDK; the JVM inside a JRE runs the result on any machine.*

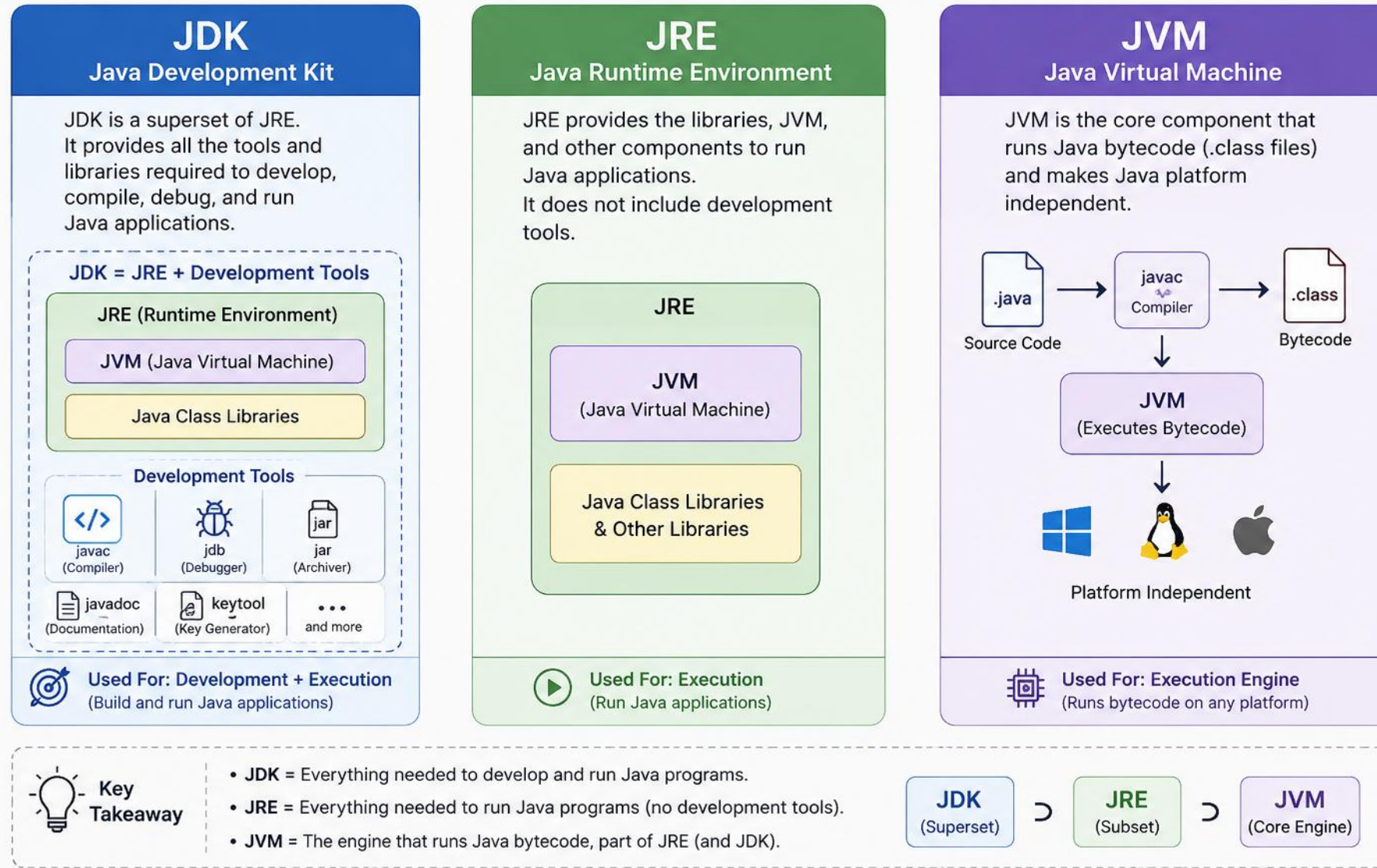
```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello Java!");  
    }  
}
```

- The JDK compiles Hello.java into Hello.class using javac — only developers need the JDK.
- The JRE (JVM + core libraries) is enough to run the compiled Hello.class on an end user's machine.
- The JVM inside that JRE loads, verifies, and executes the bytecode to print "Hello Java!".

**KEY TAKEAWAY** JDK is for developers, JRE is for running applications, and JVM is the engine that makes Java platform independent.

# JDK vs JRE vs JVM

## Understanding the Relationship and Key Differences



# WRITE ONCE, RUN ANYWHERE (WORA)

*Java's platform independence lets the same compiled bytecode run on any platform with a compatible JVM.*



- Platform Independent: Java bytecode runs on any system with a JVM.
- Consistent Behavior: the same program behaves the same way across different platforms.
- Global Reach: deploy Java applications anywhere without changing the code.
- Developer Productivity: focus on building features, not managing platform differences.

**The JVM acts as an intermediary between the bytecode and the underlying OS, ensuring the same program runs consistently across platforms.**

**KEY TAKEAWAY** Write your code once, compile it into bytecode, and run it anywhere with a JVM — the foundation of Java's portability.

# JAVA DEVELOPMENT AND EXECUTION WORKFLOW

*From writing Java code to running an application — the journey from source code to execution.*

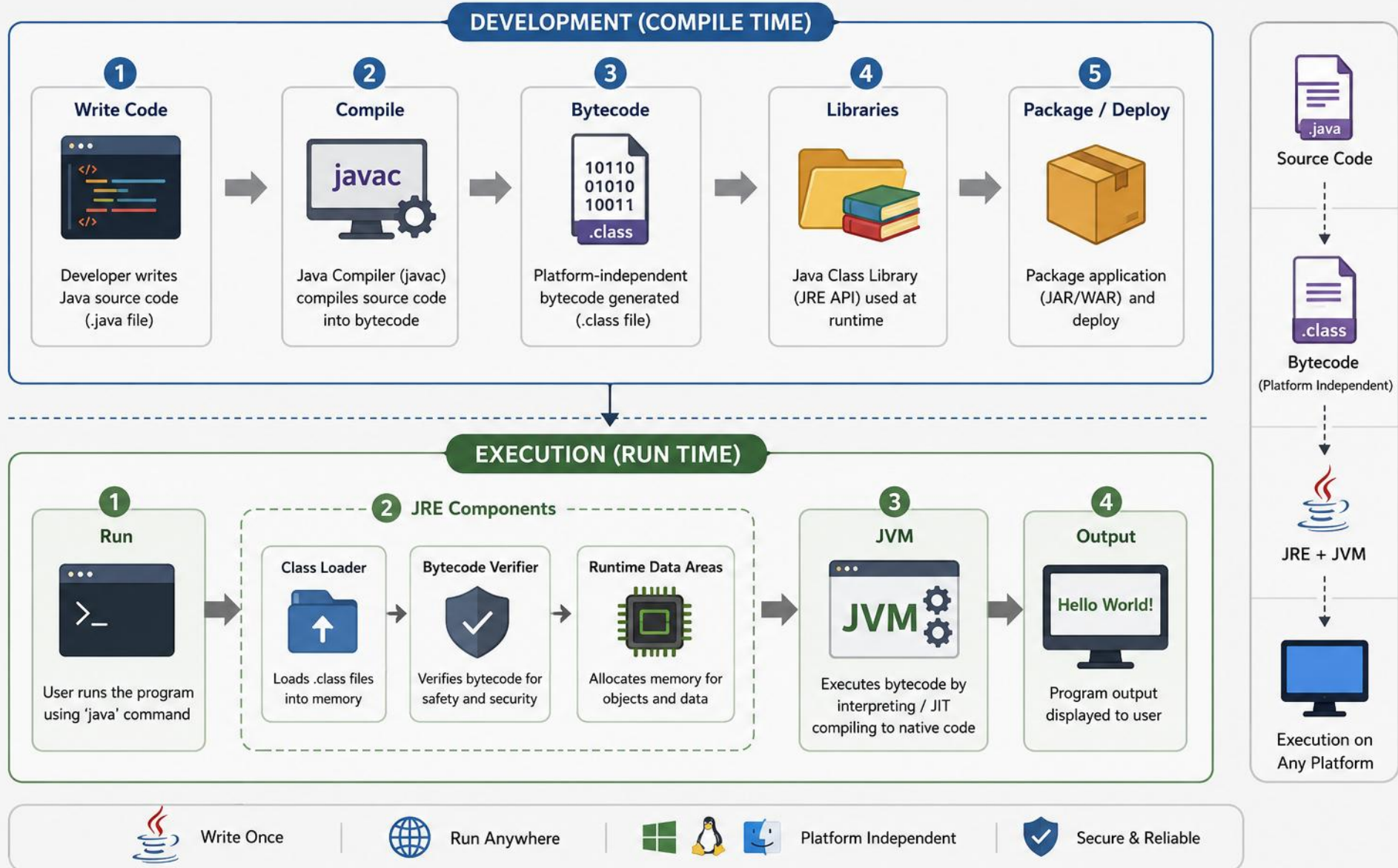


## WHAT HAPPENS INSIDE THE JVM?

- Class Loader: loads the .class file.
- Bytecode Verifier: verifies bytecode for safety.
- Execution Engine: interprets / JIT compiles and executes bytecode.
- Runtime Data Areas: manage memory for the application.
- Garbage Collector: reclaims memory automatically.

**KEY TAKEAWAY** Java source code is compiled into platform-independent bytecode, which the JVM executes on any platform. Write Once, Run Anywhere!

# Java Development and Execution Workflow



# TRY IT YOURSELF — EXERCISE 1: HELLO WORLD

*Reinforces: the edit, compile, run workflow you just saw.*

| STEP        | WHAT TO DO                                                                      |
|-------------|---------------------------------------------------------------------------------|
| Goal        | Write, compile, and run a program that prints "Hello, JVM!"                     |
| File        | Hello.java (in examples/module-01-exercises/)                                   |
| Entry point | public static void main(String[] args) — the method the java launcher looks for |
| Compile     | javac Hello.java produces Hello.class (bytecode)                                |
| Run         | java Hello starts a JVM and executes that bytecode                              |
| Reference   | exercises/exercise-01-hello-world.md                                            |

```
$ cd examples/module-01-exercises
$ javac Hello.java
$ java Hello // prints: Hello, JVM!
$ javap -c Hello // preview the bytecode – Exercise 8 covers this in depth
```

**TRY IT NOW** Complete Exercise 1 before continuing — see exercises/exercise-01-hello-world.md.

# TRY IT YOURSELF — EXERCISE 2: PLATFORM INDEPENDENCE (WORA)

*Reinforces: Write Once, Run Anywhere — the idea you just covered.*

| STEP        | WHAT TO DO                                                                        |
|-------------|-----------------------------------------------------------------------------------|
| Goal        | Run an existing .class file and explain why no recompile is needed for another OS |
| File        | Hello.class (already compiled in Exercise 1 — no source edit needed)              |
| Command     | java Hello — starts a JVM, loads Hello.class, runs main                           |
| Key idea    | The same .class bytecode runs unchanged on any OS with a matching JVM             |
| Deliverable | 2–3 sentences: source vs. bytecode vs. JVM, saved as notes/wora-notes.md          |
| Reference   | exercises/exercise-02-wora.md                                                     |

```
$ cd examples/module-01-exercises
$ java Hello
$ java Hello // re-run with no javac – proves you're executing bytecode, not the .java file
$ java -version // the same .class would run under this JVM on Windows, macOS, or Linux
```

**TRY IT NOW** Complete Exercise 2 before continuing — see exercises/exercise-02-wora.md.

# JVM COMPONENTS OVERVIEW

*The JVM is the runtime engine that executes Java bytecode and provides the runtime environment for Java applications.*

## KEY RESPONSIBILITIES



### Execute Bytecode

Runs compiled Java bytecode.



### Platform Independence

Write Once, Run Anywhere.



### Manage Memory

Allocates & reclaims automatically.



### Ensure Security

Secure execution environment.



### Optimize Performance

JIT compilation & tuning.

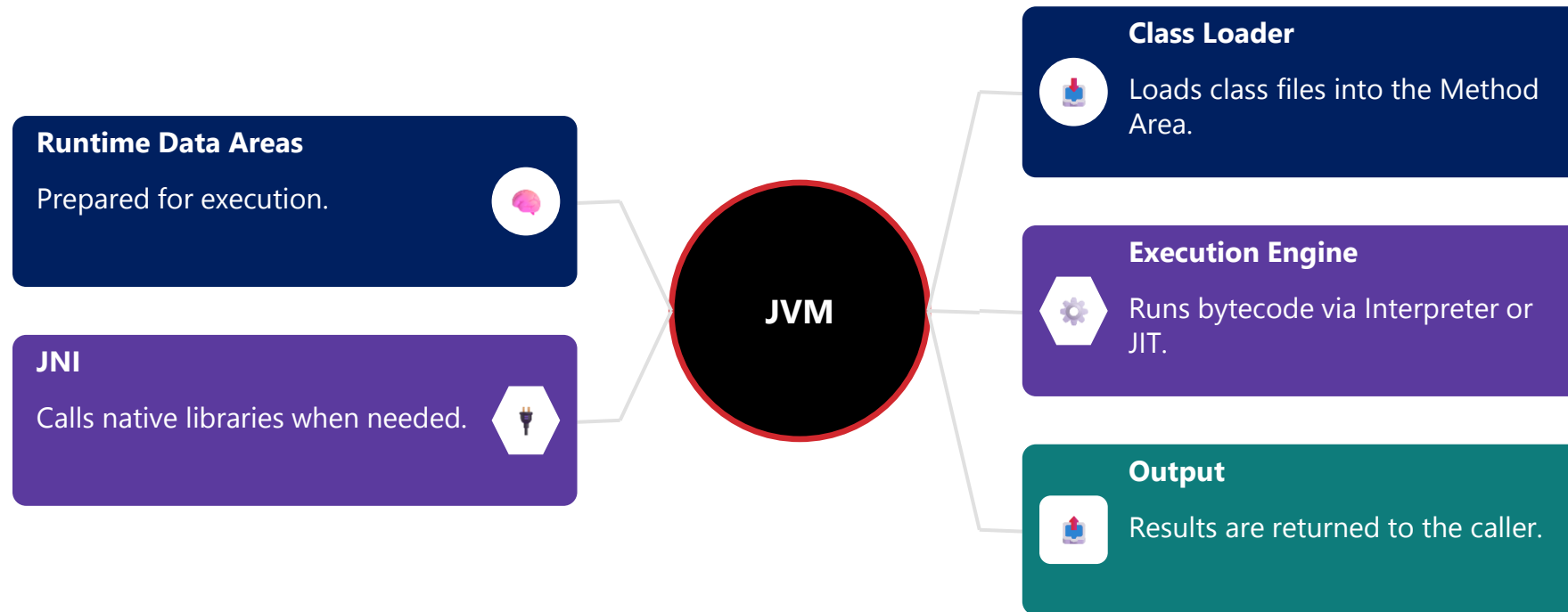
## JVM ARCHITECTURE (HIGH LEVEL)

- Class Loader Subsystem: loads .class files into the JVM.
- Runtime Data Areas: Method Area, Heap, Java Stacks, PC Register, Native Method Stack.
- Execution Engine: interprets bytecode; JIT Compiler optimizes and executes code.
- Native Method Interface (JNI): lets Java code call native libraries and methods.
- Native Method Libraries: native libraries (C/C++, OS) used by the JVM and applications.

**KEY TAKEAWAY** JVM is the heart of the Java Platform — it provides platform independence, security, and performance through optimization.

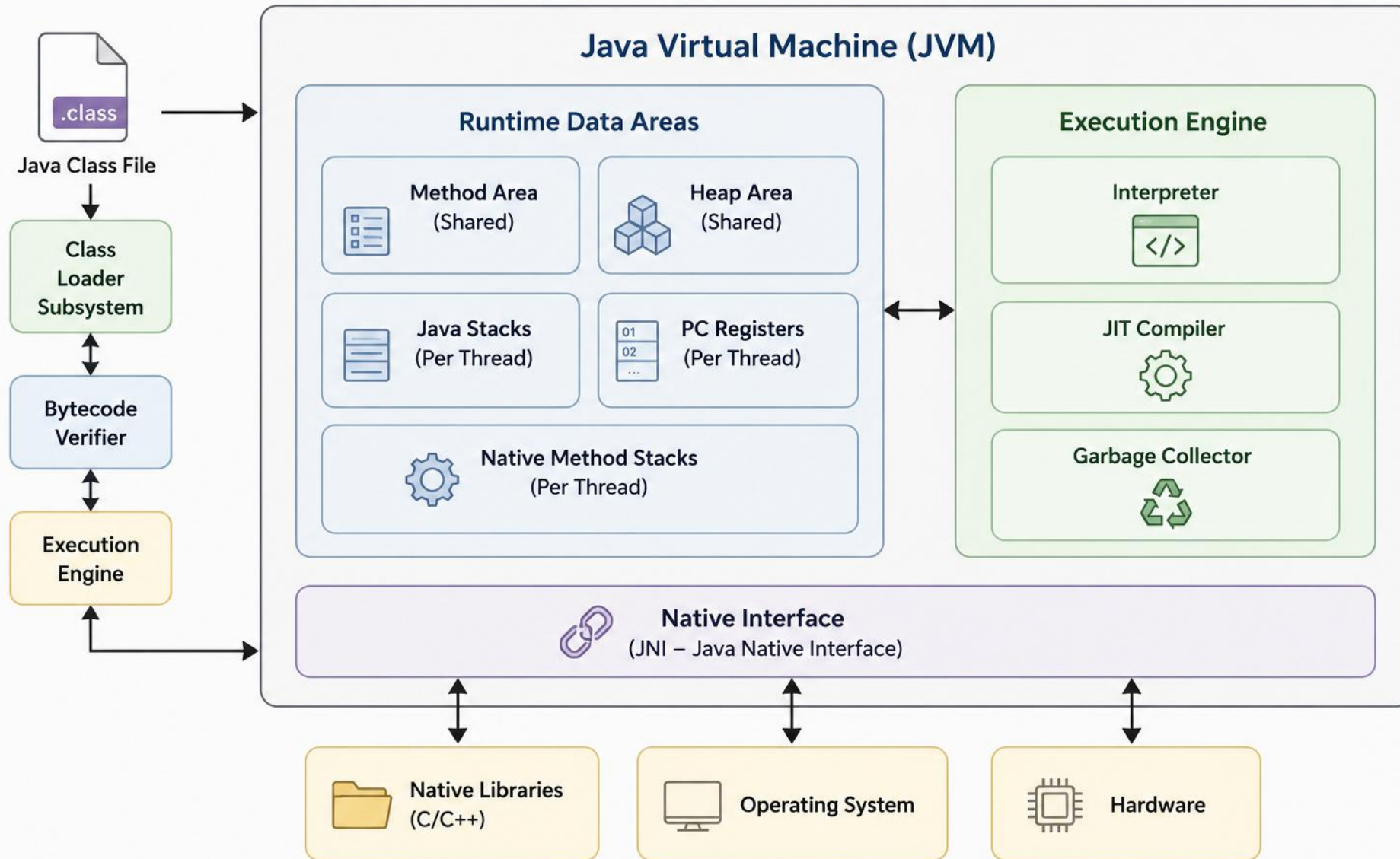
# HOW JVM COMPONENTS WORK TOGETHER

*Five components collaborate to load classes, manage memory, execute bytecode, and interact with the OS.*

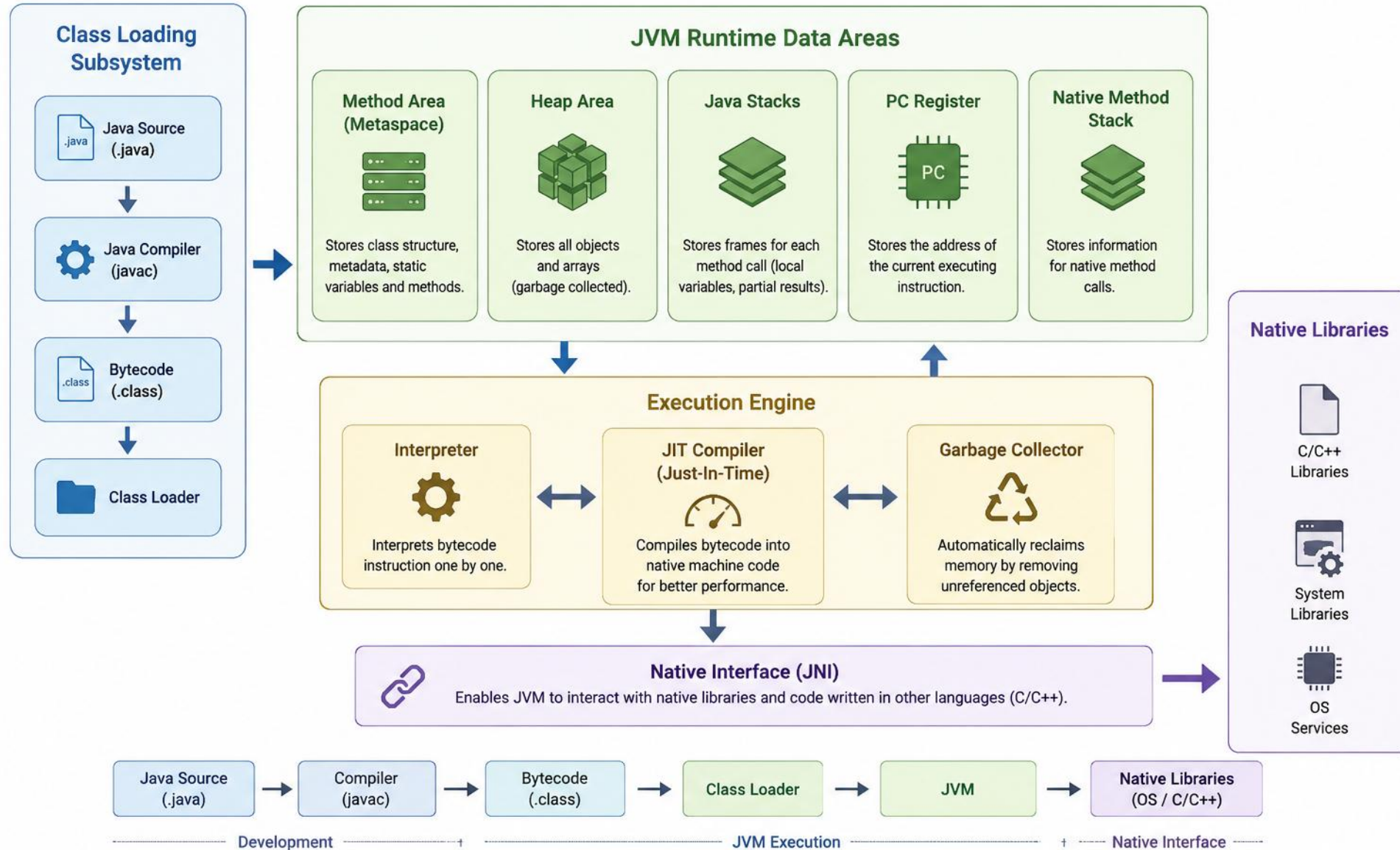


**KEY TAKEAWAY** These five components collaborate on every single method call your program makes.

# JVM Components



# Java Virtual Machine (JVM) Architecture



# CLASS LOADER SUBSYSTEM

*Responsible for loading .class files into the JVM, linking them, and preparing them for execution.*

## WHAT IT DOES

- Loads class files from various sources, links and verifies them, ensures they are safe and valid to execute, and prepares them for runtime usage.
- Class file sources: file system, JAR/WAR files, network locations, and databases.

## CLASS LOADER ARCHITECTURE — DELEGATION MODEL

### 1 Bootstrap

- Loads core classes from rt.jar / Java 9+ modules.
- e.g. java.lang.\*, java.util.\*, java.io.\*

### 2 Platform (Extension)

- Loads from jre/lib/ext or modules.
- e.g. javax.\*, org.w3c.dom.\*, org.xml.sax.\*

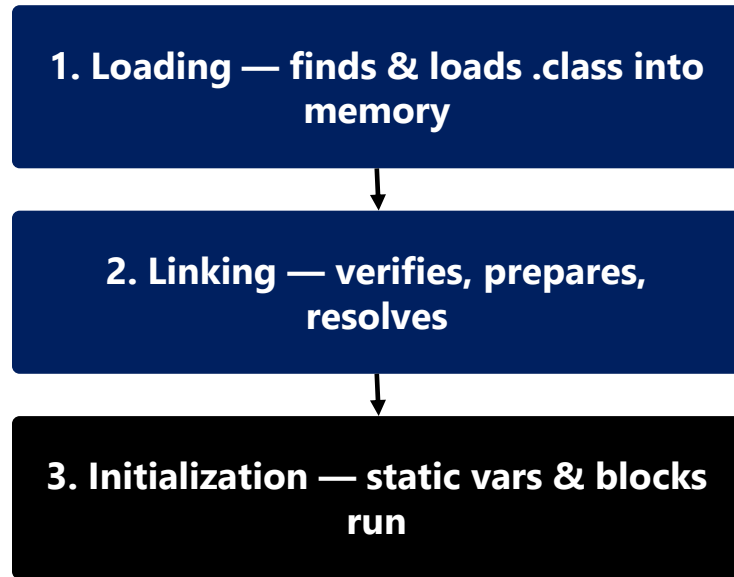
### 3 Application (System)

- Loads from the application classpath (-cp).
- e.g. com.myapp.\*, com.example.\*

**KEY TAKEAWAY** Requests delegate upward through the hierarchy — this is the parent delegation model, and it exists for security.

# CLASS LOADER — LIFECYCLE & COMPLETE FLOW

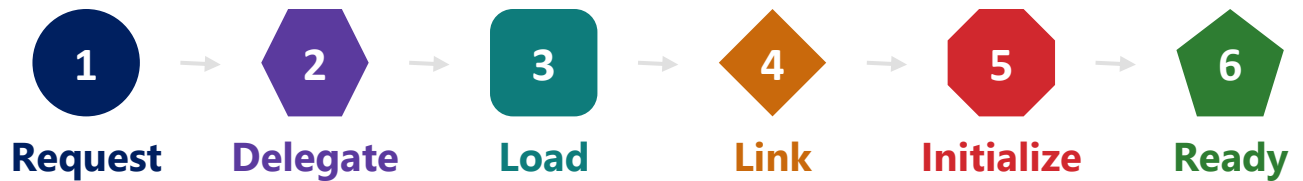
*Every class passes through the same three-phase lifecycle before it is ready to use.*



## KEY FEATURES

- Follows the Parent Delegation Model (security first).
- Supports custom class loaders.
- Enables dynamic class loading.
- Foundation for JVM security and isolation.

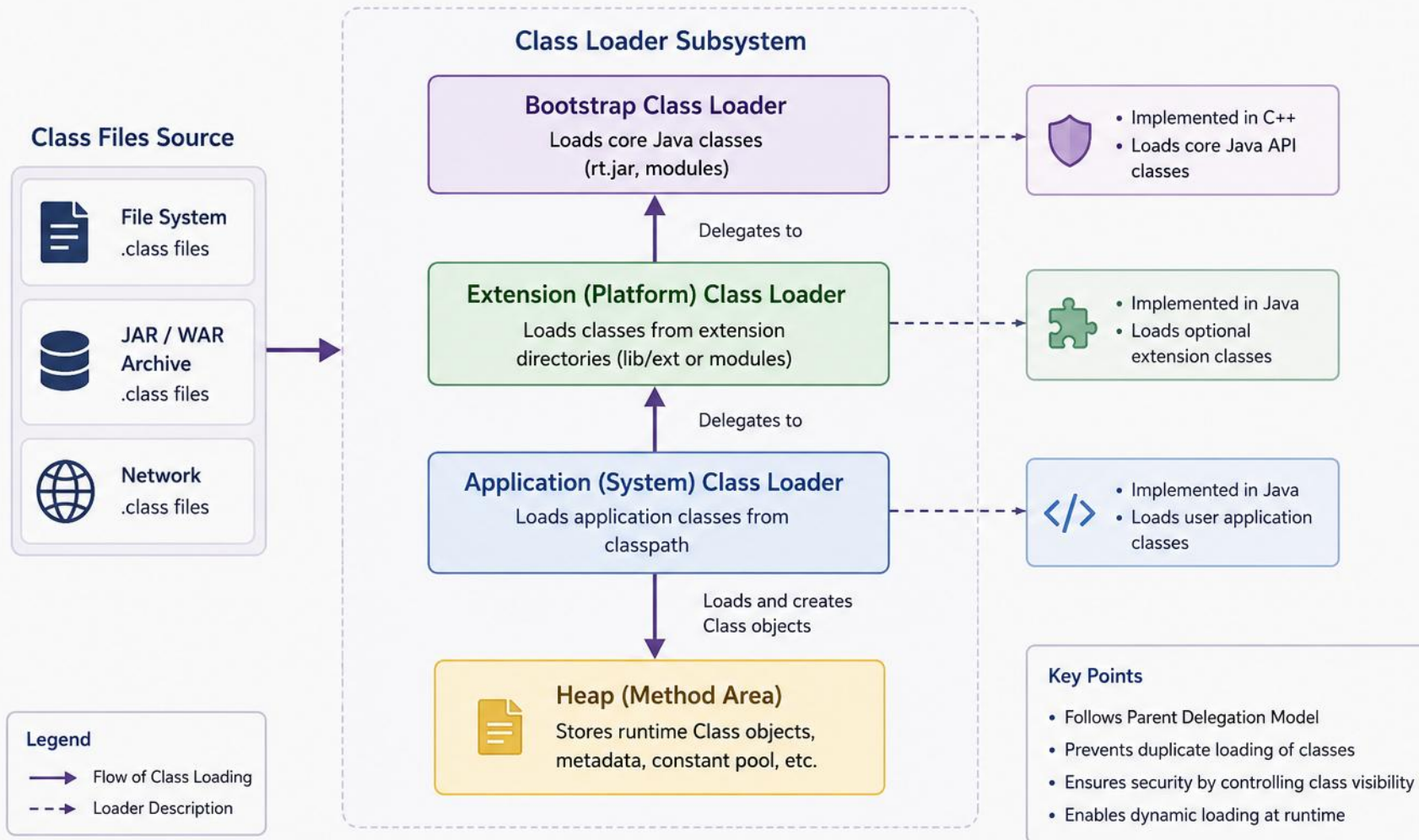
## COMPLETE FLOW



**KEY TAKEAWAY** The Class Loader Subsystem ensures that only valid and secure classes are loaded and used by the JVM.

# Class Loader Subsystem

Dynamically loads .class files into the JVM and creates Class objects in the Heap.



# RUNTIME DATA AREAS

*The JVM manages memory using runtime data areas, each with a specific purpose and lifecycle.*

| AREA                  | SCOPE               | PURPOSE                                                                                |
|-----------------------|---------------------|----------------------------------------------------------------------------------------|
| Method Area           | Shared              | Class-level data: constant pool, field/method data, static variables.                  |
| Heap Area             | Shared              | All objects and arrays; managed by the Garbage Collector; the largest area.            |
| Java Stacks           | Per-thread          | Frames per method invocation: local variables, operand stack, constant-pool reference. |
| PC Register           | Per-thread          | Address of the next instruction to execute; undefined for native methods.              |
| Native Method Stack   | Per-thread          | Supports native (C/C++) method calls, created and destroyed with the thread.           |
| Runtime Constant Pool | Part of Method Area | Constants used in compiled code: literals and symbolic references.                     |

**KEY TAKEAWAY** Memory is allocated and reclaimed automatically — but `OutOfMemoryError` can still occur if memory runs out.

# RUNTIME DATA AREAS — WHY IT MATTERS

*Six areas, one summary each — the vocabulary you'll use for the rest of this module.*



## Method Area

- Class structure and metadata.



## Heap Area

- Objects live here.



## Java Stack

- Manages method execution per thread.



## PC Register

- Tracks the next instruction.



## Native Method Stack

- Handles native code execution.



## Runtime Constant Pool

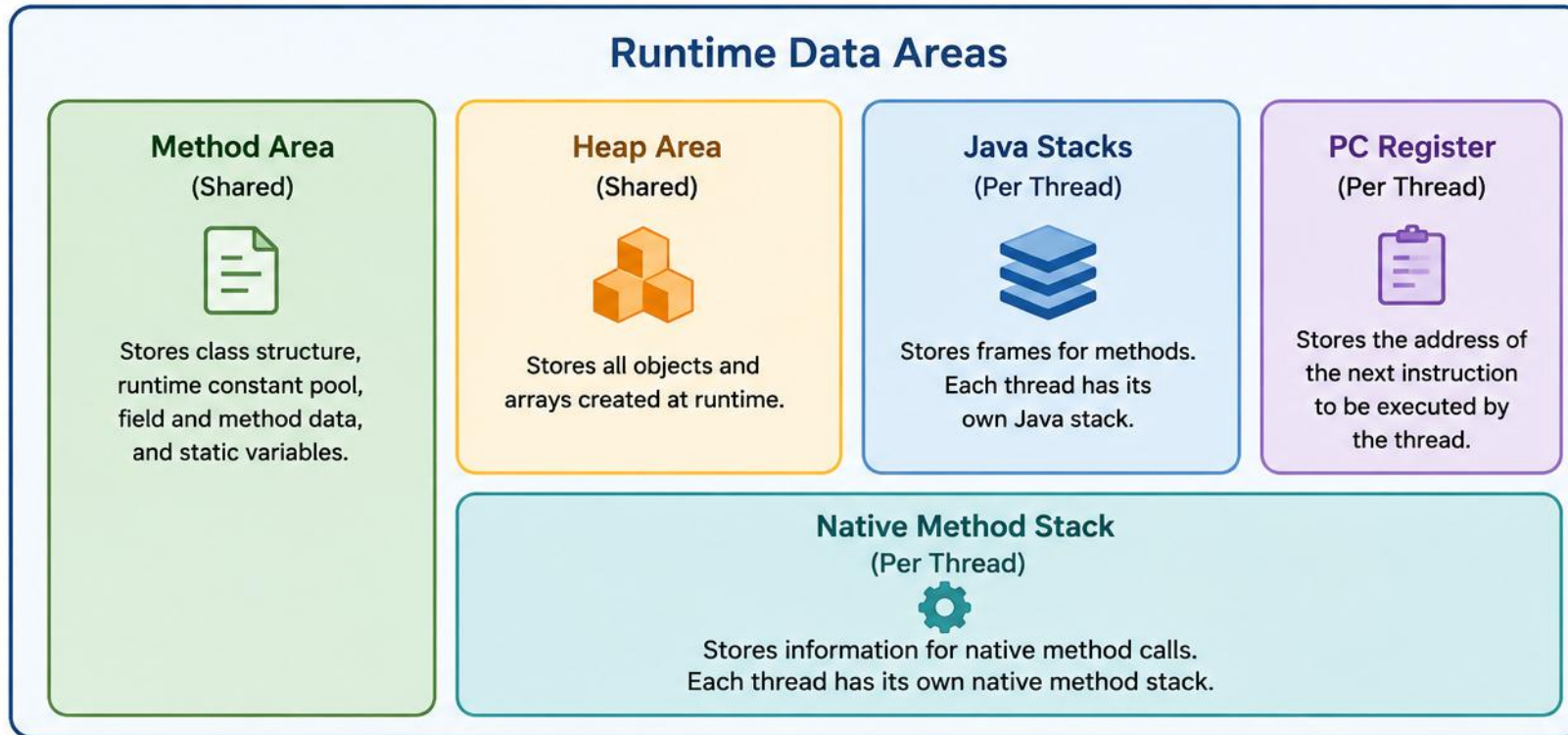
- Holds constants & symbolic references.

**Understanding runtime areas helps in memory tuning, performance optimization, diagnosing `OutOfMemoryError`, JVM monitoring, and building robust, scalable applications.**

**KEY TAKEAWAY** All areas are created by the JVM; the Heap and Method Area are shared across threads, the rest are thread-specific.

# JVM Runtime Data Areas

Memory areas used by the JVM during program execution



# EXECUTION ENGINE

*The heart of the JVM — it executes bytecode, optimizes performance, and works with Runtime Data Areas and native libraries.*

## KEY RESPONSIBILITIES

- Execute bytecode: interprets and executes instructions.
- Optimize performance using JIT compilation and other techniques.
- Manage runtime operations: object creation, synchronization, exception handling, method invocation.
- Interface with native libraries via JNI when needed.

## COMPONENTS OF THE EXECUTION ENGINE



### Interpreter

- Reads bytecode instruction by instruction; fast startup.



### JIT Compiler

- Compiles hot spots to native code for throughput.



### GC Interface

- Works with the Garbage Collector to reclaim objects.



### Runtime Support

- Threading, sync/locking, exceptions, deoptimization.

**KEY TAKEAWAY** The Execution Engine works with memory areas, the class loader, and the garbage collector for correct, efficient execution.

# EXECUTION FLOW & JIT COMPILATION

*How bytecode moves from the class loader to a finished result.*



## JIT COMPILATION — HOW IT HELPS



### Detects Hot Spots

- Finds methods/loops executed frequently.



### Compiles to Native Code

- Converts bytecode to optimized machine code.



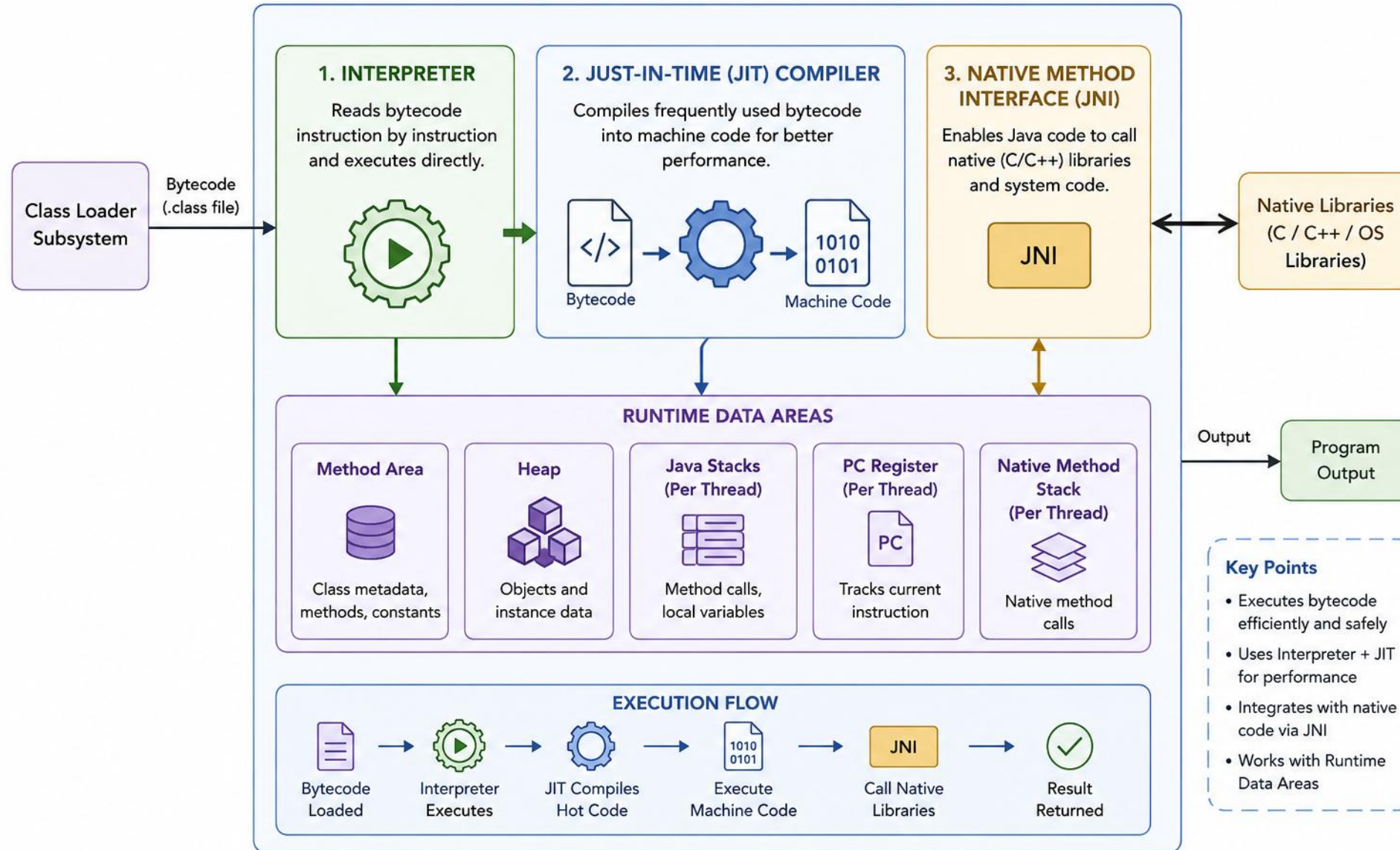
### Better Performance

- Reduces interpretation overhead, improves throughput.

**KEY TAKEAWAY** The Execution Engine turns bytecode into real work — JIT makes Java fast after warm-up, enabling both portability and performance.

# EXECUTION ENGINE

Executes bytecode and manages runtime operations



# JAVA NATIVE INTERFACE (JNI)

*A standard mechanism that lets Java code in the JVM call, and be called by, native applications written in C or C++.*

## WHY JNI?



### Reuse Native Code

Leverage existing libraries and legacy code.



### Performance

Performance-critical operations in native code.



### Platform Features

Access platform-specific capabilities.



### Integration

Connect with non-Java systems and frameworks.

## HOW JNI WORKS

1

**Calls Native Method**



2

**JVM Loads Library**



3

**JNI Bridges**



4

**Native Code Runs**



5

**Result to Java**

**KEY TAKEAWAY** JNI does not break "Write Once, Run Anywhere" — it loads the appropriate native library for each platform.

# JNI — DATA TYPES & EXAMPLE

*Every Java type maps to a specific C/C++ JNI type.*

| JAVA TYPE | JNI TYPE | JAVA TYPE | JNI TYPE               |
|-----------|----------|-----------|------------------------|
| boolean   | jboolean | char      | jchar (unsigned short) |
| byte      | jbyte    | short     | jshort                 |
| int       | jint     | long      | jlong                  |
| float     | jfloat   | double    | jdouble                |
| Object    | jobject  | String    | jstring                |
| void      | void     | Array     | j<type>Array           |

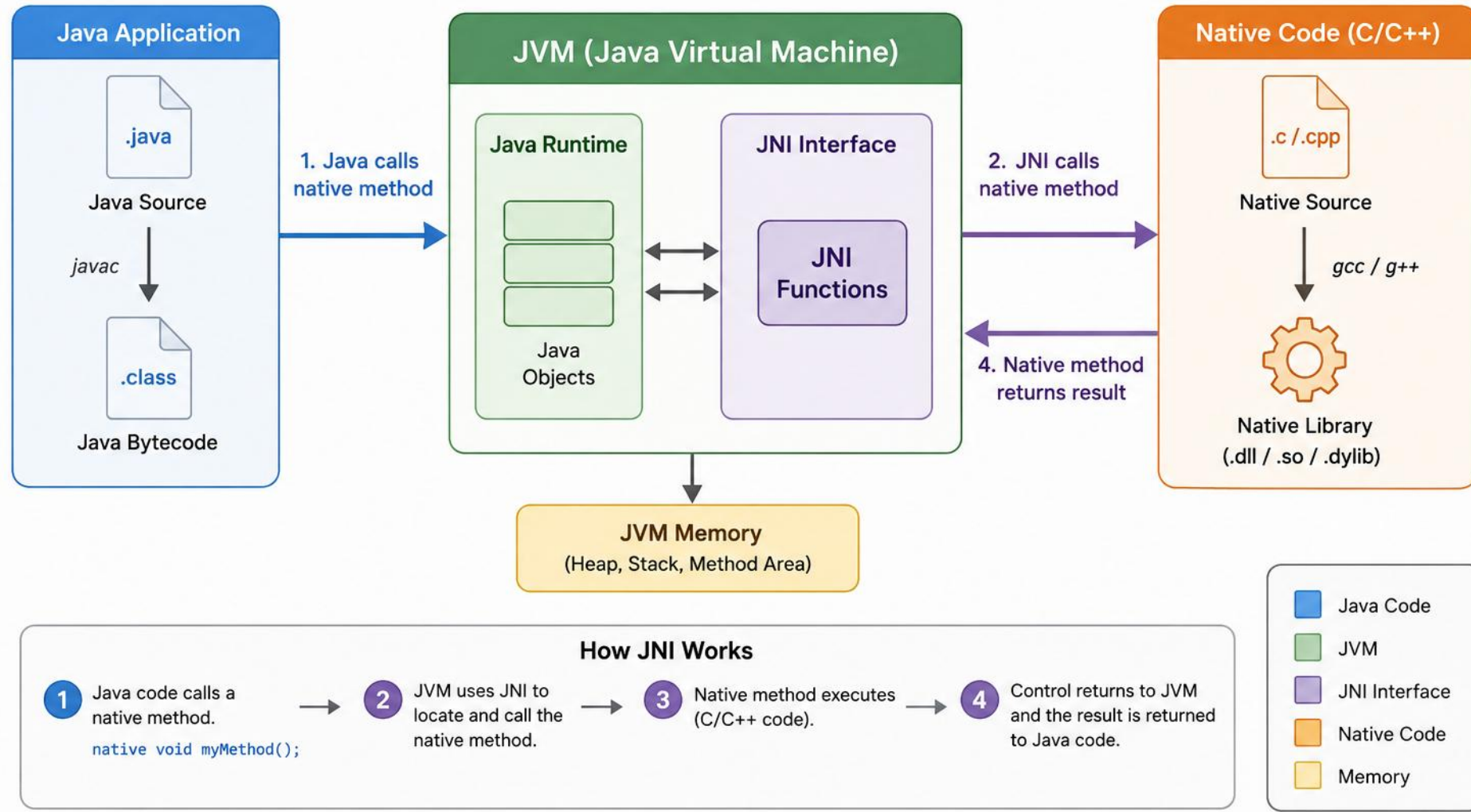
```
public class MyClass {
    native void nativeMethod();
    static { System.loadLibrary("native"); }
}
```

```
#include <jni.h>
JNIEXPORT void JNICALL
Java_MyClass_nativeMethod(JNIEnv *e, jobject o) {
    // native C implementation
}
```

**KEY TAKEAWAY** Always release native resources, follow naming conventions for native methods, handle exceptions across the boundary, and test on every target platform.

# Java Native Interface (JNI)

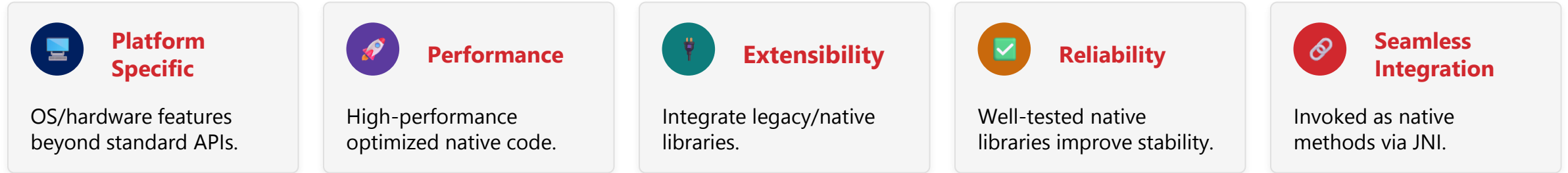
JNI enables Java code running in the JVM to call native code written in languages like C or C++ and vice versa.



# NATIVE METHOD LIBRARIES

*Collections of native (non-Java) code that Java programs call through JNI, extending Java with platform-specific capability.*

## KEY FEATURES



## HOW NATIVE LIBRARIES WORK



**KEY TAKEAWAY** Native libraries are platform-dependent: DLL on Windows, .so on Linux, .dylib on macOS — matched to 32-bit/64-bit architecture.

# NATIVE LIBRARIES — CHARACTERISTICS & EXAMPLES

*Common native libraries used across graphics, networking, databases, and audio.*



## Written in C/C++

- Compiled into shared libraries per platform.



## Loaded at Runtime

- Loaded into the JVM as needed.



## Low-Level Access

- Access to low-level system resources.

| LIBRARY                      | PURPOSE                           | PLATFORM(S)       | EXAMPLE USE CASE                  |
|------------------------------|-----------------------------------|-------------------|-----------------------------------|
| OpenSSL                      | Cryptography and SSL/TLS          | All               | Secure communication              |
| Graphics Libraries           | High-performance graphics         | Platform specific | Games, 3D rendering               |
| Multimedia Libraries         | Audio/video processing            | All               | Media players, streaming          |
| JNI Onload Libraries         | Native impls of Java methods      | All               | Custom native functionality       |
| System Libraries (zlib, ICU) | Compression, internationalization | All               | Data compression, text processing |

**KEY TAKEAWAY** Native libraries deliver enhanced performance, native capability reuse, and improved reliability — at the cost of platform-specific builds.

# COMMON NATIVE METHOD LIBRARIES IN PRACTICE

*Real-world Java APIs that are backed by native libraries under the hood.*



## OpenGL

2D/3D graphics rendering.



## Oracle/MySQL JDBC Drivers

Native database connectivity.



## Java Sound (PortMixer/ALSA/CoreAudio)

Audio I/O.



## Java Advanced Imaging (JAI)

Image processing.



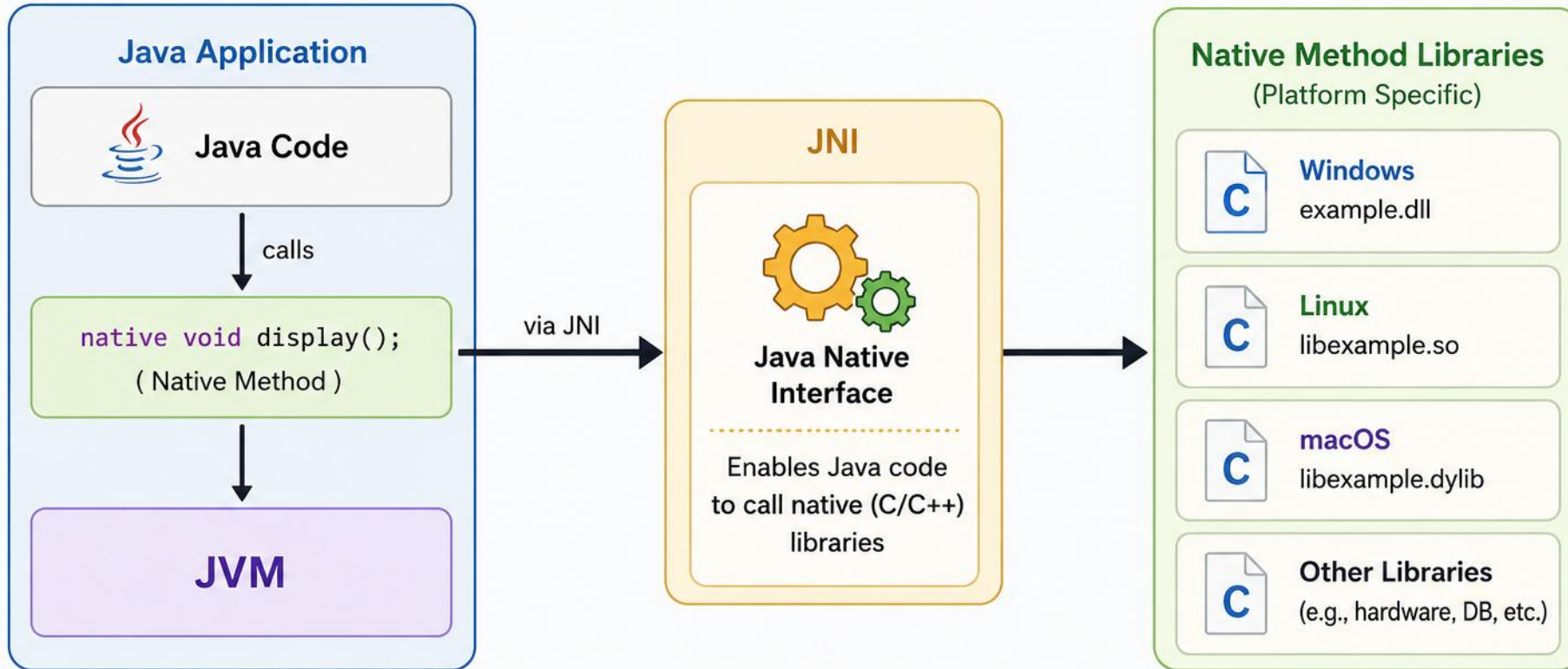
## Netty/OpenSSL

Networking and SSL.

**KEY TAKEAWAY** Many everyday Java APIs — graphics, databases, audio, networking — are Java-friendly wrappers around native libraries.

# Native Method Libraries

Native method libraries contain platform-specific code that can be invoked by Java applications through the JNI.



- Native method libraries are written in languages like C/C++.
- They provide access to platform-specific features and optimized performance.
- Examples: File I/O, Network operations, OS APIs, Hardware integration.

# END-TO-END JVM EXECUTION FLOW

*From Java source code to program execution and result — eight steps, start to finish.*

- 
- 1 Write Java Source Code (MyClass.java).
  - 2 Compile with javac → MyClass.class.
  - 3 Class Loader loads .class into the JVM.
  - 4 Runtime Data Areas are allocated.
  - 5 Bytecode Execution via Interpreter or JIT.
  - 6 Native Method called via JNI, if needed.
  - 7 Execution & Result — program logic runs.
  - 8 Program Terminates — JVM cleans up.

**KEY TAKEAWAY** Source (.java) → Compiler (javac) → Bytecode (.class) → Class Loader → Runtime Data Areas → Execution Engine → Result → Termination.

# INSIDE THE JVM — PUTTING IT TOGETHER

*The four subsystems that cooperate on every run.*



## Class Loader Subsystem

- Loading → Linking → Initialization; delegation flows Bootstrap → Platform → Application.



## Runtime Data Areas

- Method Area, Heap, Java Stacks, PC Register, Native Method Stack.



## Execution Engine

- Interpreter executes bytecode; JIT compiles hot code; GC reclaims memory.



## Native Integration via JNI

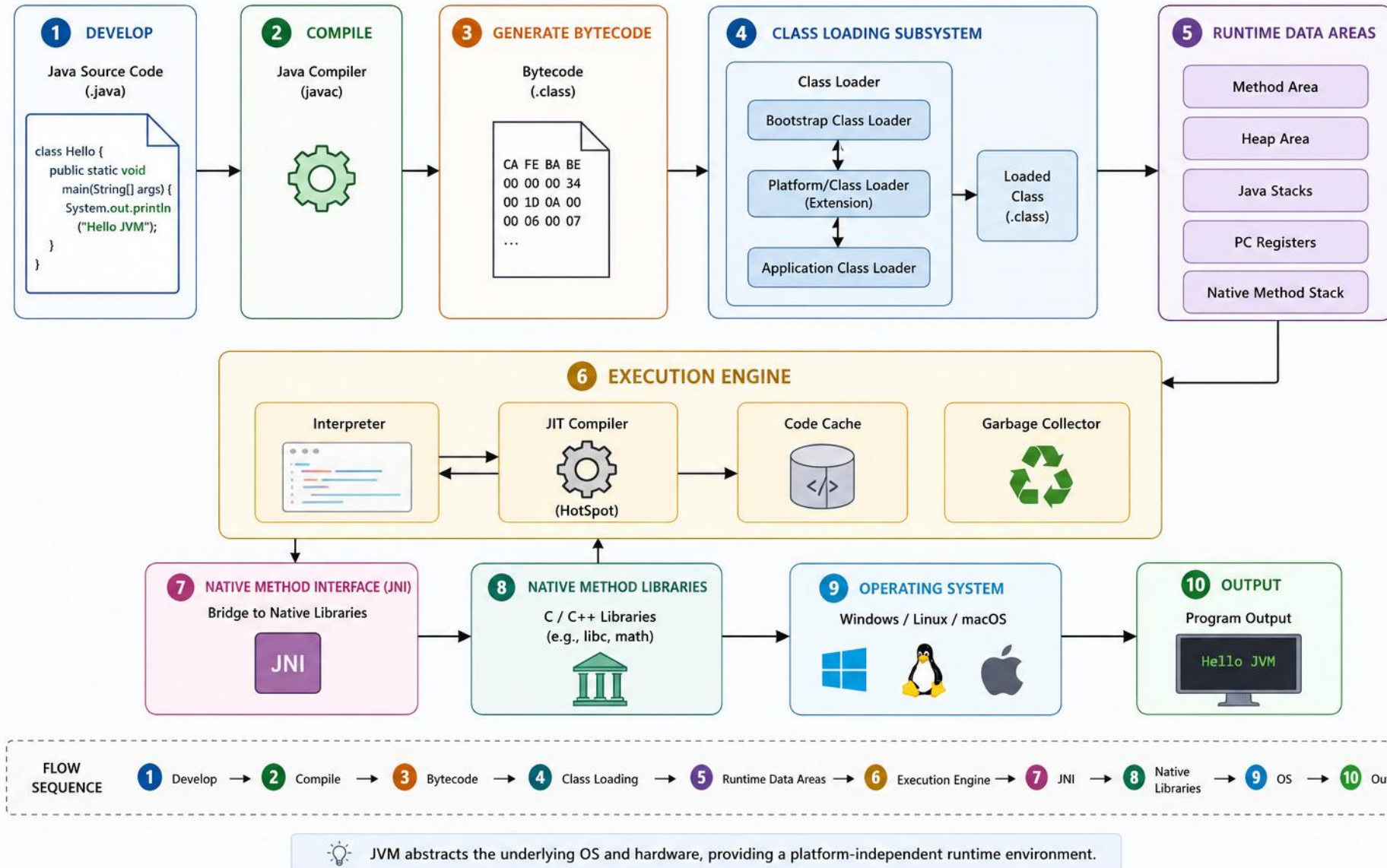
- Java Code ↔ JNI ↔ Native Libraries (.dll / .so / .dylib).

- Benefits of this architecture: platform independence, security, performance, scalability and extensibility, and reliability.

**KEY TAKEAWAY** JIT compilation improves performance over time, and the Garbage Collector automatically manages memory throughout.

# END-TO-END JVM EXECUTION FLOW

From Java Source Code to Program Execution



# FROM SOURCE CODE TO BYTECODE

*How Java code becomes platform-independent bytecode.*

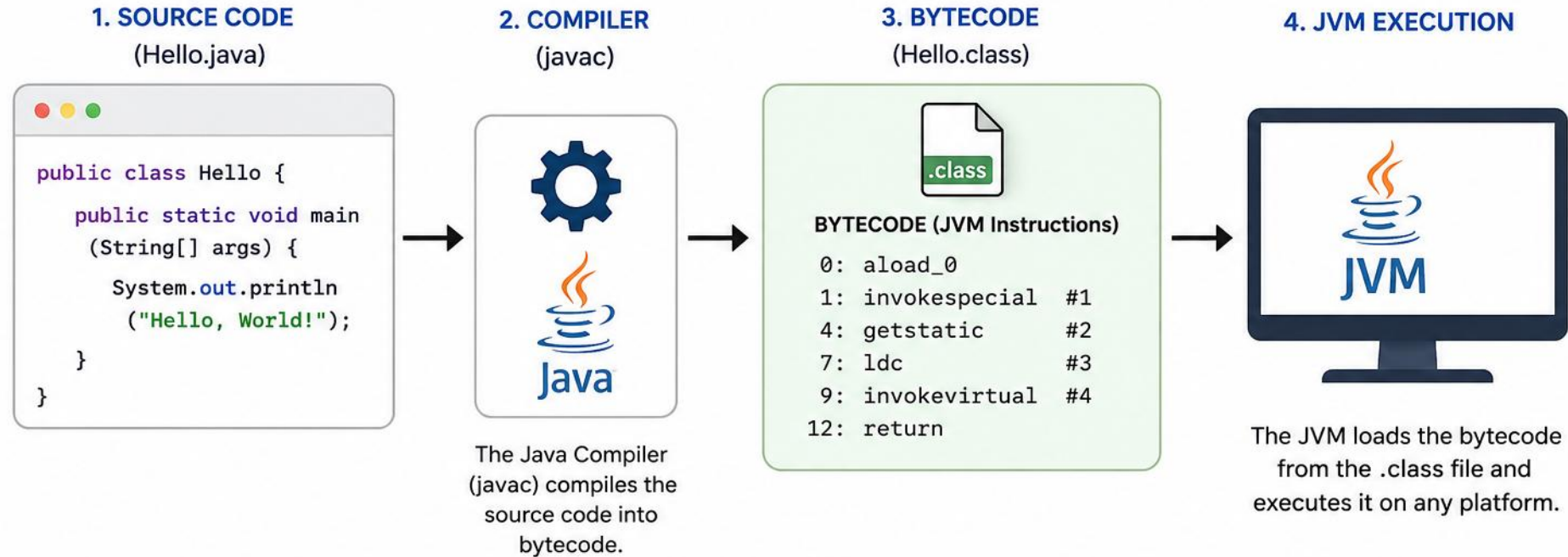
- 1 Write Source Code (.java file, text editor).
- 2 Invoke Compiler: javac from the command line.
- 3 Syntactic & Semantic Analysis: types, declarations, syntax.
- 4 Generate Bytecode: platform-independent .class file.
- 5 Ready for JVM: executable by any JVM, any platform.

```
$ javac HelloWorld.java  
Compilation Successful. No errors found.  
Bytecode generated.
```

| FILE             | GENERATED BY   |
|------------------|----------------|
| HelloWorld.java  | Developer      |
| HelloWorld.class | javac Compiler |

**KEY TAKEAWAY** The compiler does not create machine code — it creates bytecode that is executed by the JVM.

# FROM SOURCE CODE TO BYTECODE



## 1. Source Code

Developer writes code in a .java file using any text editor or IDE.



## 2. Compiler (javac)

The Java compiler checks for syntax and semantic errors and converts the source code into bytecode.



## 3. Bytecode (.class)

Bytecode is platform-independent and stored in a .class file. It contains JVM instructions.



## 4. JVM Execution

The JVM loads the bytecode and executes it after just-in-time (JIT) compilation to native machine code.



## Key Point:

Bytecode acts as an intermediary between source code and machine code, enabling Java's platform independence (Write Once, Run Anywhere).

# UNDERSTANDING JAVA BYTECODE

*The platform-independent language of the JVM — an intermediate instruction set generated by javac.*



## Stack-Based

- Operates on an operand stack.



## Strongly Typed

- All variables/operations strictly typed.



## Verified

- JVM verifies safety before running.



## Explicit Local Var Mgmt

- Uses slots in the local variable array.



## Rich Instruction Set

- Control flow, objects, exceptions, method calls.



## Includes Metadata

- Constant pool, class info, access flags, attributes.

## .CLASS FILE STRUCTURE

- Magic Number (0xCAFEBAFE), Version, Constant Pool, Access Flags.
- This Class / Super Class / Interfaces / Fields / Methods / Attributes (Code, LineNumberTable, etc).

**KEY TAKEAWAY** Not machine code, not platform-specific — designed for the JVM, safe, verifiable, and secure.

# BYTECODE INSTRUCTIONS IN ACTION

*Categories of instructions, and a stack trace for `int c = a + b`.*

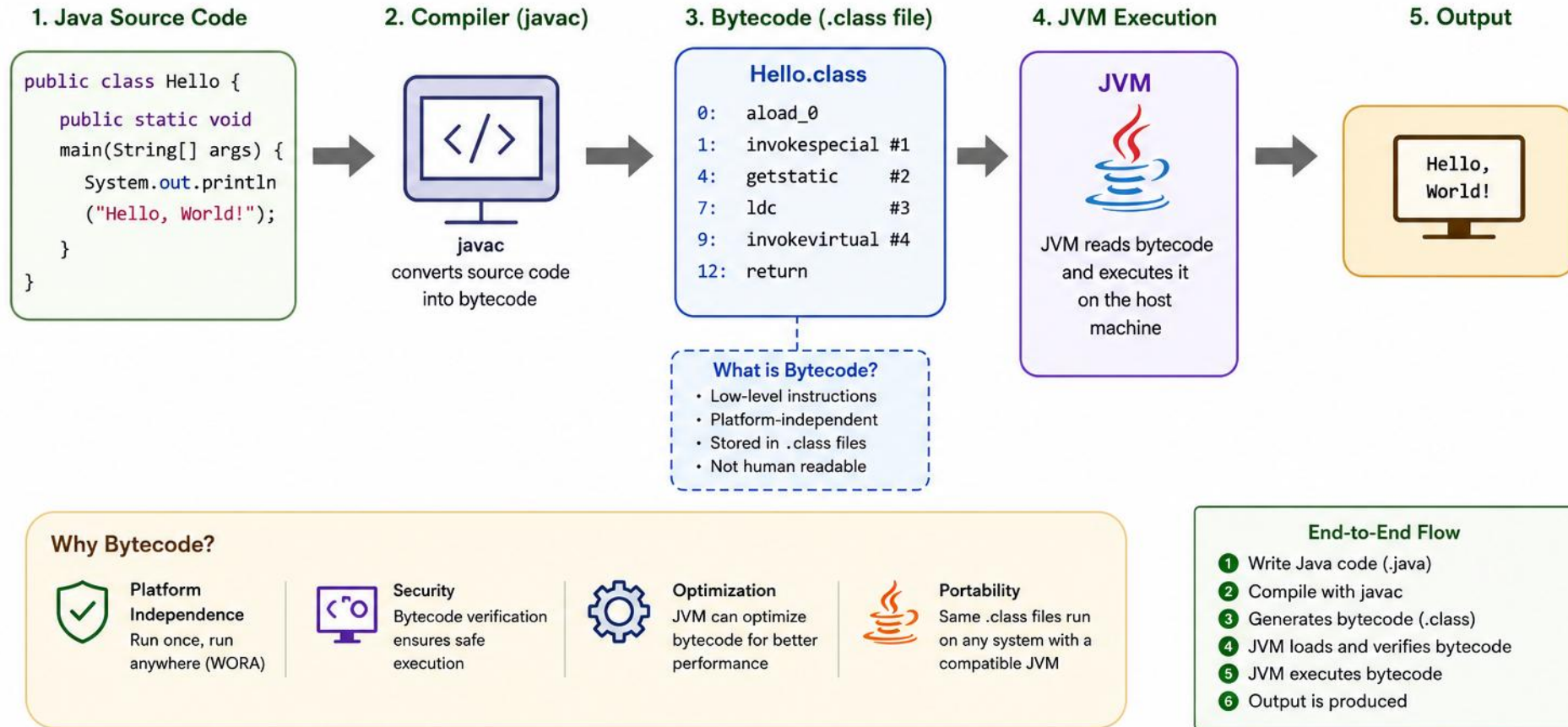
| CATEGORY           | PURPOSE                         | EXAMPLE INSTRUCTIONS         |
|--------------------|---------------------------------|------------------------------|
| Load & Store       | Move data between locals/stack  | iload, aload, istore, astore |
| Arithmetic         | Perform arithmetic operations   | iadd, isub, imul, idiv       |
| Object Operations  | Create objects, access fields   | new, getfield, putfield      |
| Method Invocation  | Call methods                    | invokestatic, invokevirtual  |
| Control Flow       | Branching and looping           | ifeq, ifne, goto             |
| Type Conversion    | Convert between primitive types | i2l, i2d, i2b, i2c           |
| Return             | Return from a method            | ireturn, areturn, return     |
| Exception Handling | Throw and catch exceptions      | athrow, try-catch blocks     |

```
int c = a + b;    // iload_1 (a) → iload_2 (b) → iadd → istore_3 (c)
```

**KEY TAKEAWAY** Bytecode is the heart of Java's portability and security — compact, efficient, and safe. Explore it with javap, ASM, or CFR.

# Understanding Java Bytecode

Java bytecode is an intermediate representation of Java source code. It is platform-independent, generated by the Java compiler (.class files), and executed by the JVM on any platform.



**Key Takeaway:** Java bytecode is the bridge between your Java program and the platform it runs on, enabling portability, security, and performance.

# TRY IT YOURSELF — EXERCISE 3: CONTROL FLOW

*Reinforces: the branch and jump bytecode instructions you just saw.*

| STEP        | WHAT TO DO                                                                                        |
|-------------|---------------------------------------------------------------------------------------------------|
| Goal        | Create ControlFlow.java using if, for, while, and switch                                          |
| File        | ControlFlow.java (in examples/module-01-exercises/)                                               |
| Compile     | javac ControlFlow.java                                                                            |
| Run         | java ControlFlow — runs all four demos                                                            |
| Key concept | Each if/switch branch compiles to a conditional jump (e.g. ifeq, goto) like the ones you just saw |
| Reference   | exercises/exercise-03-control-flow.md                                                             |

```
$ cd examples/module-01-exercises
$ javac ControlFlow.java
$ java ControlFlow // prints even/odd, a countdown, and a weekday from switch
$ javap -c ControlFlow // look for ifeq / goto – the branch instructions behind if and while
```

**TRY IT NOW** Complete Exercise 3 before continuing — see exercises/exercise-03-control-flow.md.

# JAVA COMPILATION PROCESS

*From .java source code to .class bytecode — the stages javac performs.*



| COMMAND                                 | DESCRIPTION                                  |
|-----------------------------------------|----------------------------------------------|
| <code>javac Hello.java</code>           | Compiles Hello.java to Hello.class           |
| <code>javac *.java</code>               | Compiles all Java files in current directory |
| <code>javac -d out Hello.java</code>    | Places the .class file in the out directory  |
| <code>javac -cp lib/* Hello.java</code> | Uses libraries in lib for compilation        |
| <code>javac -verbose Hello.java</code>  | Shows detailed compilation steps             |
| <code>javap -c Hello.class</code>       | Disassembles bytecode in human-readable form |

**KEY TAKEAWAY** Compilation happens once; execution can happen many times — and every top-level class produces its own .class file.

# INSIDE THE COMPILER

*What javac actually does between reading your source file and emitting bytecode.*

- 1 Lexical Analysis (Tokenizing) — breaks source into tokens.
- 2 Syntax Analysis (Parsing) — builds the parse tree / AST.
- 3 Semantic Analysis — type checking, inheritance, access control, flow analysis.
- 4 Intermediate Representation — internal form before bytecode.
- 5 Bytecode Generation — emits the final .class bytecode.

## COMPILATION ERRORS

- Syntax errors
- Type mismatch
- Undeclared variables/methods
- Access violations

Fix errors in source code and recompile until compilation succeeds.

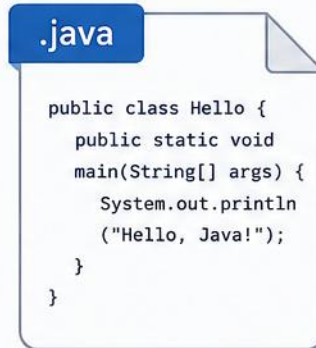
**SYMBOL TABLE (EXAMPLE)**

| NAME      | KIND      | TYPE     | SCOPE  | MODIFIERS     |
|-----------|-----------|----------|--------|---------------|
| Hello     | Class     | —        | Global | public        |
| main      | Method    | void     | Class  | public static |
| args      | Parameter | String[] | Method | —             |
| a, b, sum | Variable  | int      | Method | local         |

# Java Compilation Process

From Source Code to Bytecode

## 1. Write Source Code



Hello.java



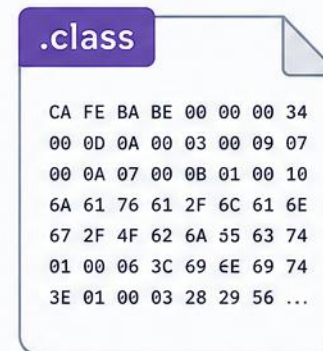
Developer writes Java source code in a .java file.

## 2. Compiler (javac)



The Java compiler (javac) checks the source code for syntax and semantic errors and converts it into bytecode.

## 3. Bytecode Output



Hello.class



The output is platform-independent bytecode stored in a .class file.

## 4. Ready for Execution



The .class file is loaded by the JVM and executed. JVM converts bytecode into machine code at runtime.



### Key Points

- javac compiles .java files into .class files.
- Bytecode is platform-independent and runs on any system with a JVM.
- Compilation happens once; execution can happen many times.
- Errors in source code are caught at compile time.

# INTRODUCTION TO CLASS LOADING

*How the JVM dynamically locates, loads, and prepares classes for execution.*

- When a class is needed, the JVM locates the .class file, loads the class data, links it (verify/prepare/resolve), and initializes it (runs static initializers). After this, the class is ready to use.

| CLASS LOADER         | PARENT      | RESPONSIBILITY              | LOADS FROM                |
|----------------------|-------------|-----------------------------|---------------------------|
| Bootstrap            | None (root) | Loads core Java classes     | rt.jar / Java 9+ modules  |
| Platform (Extension) | Bootstrap   | Loads extension libraries   | jre/lib/ext or extensions |
| Application (System) | Platform    | Loads application classes   | CLASSPATH, -cp, jars      |
| Custom               | Application | Loads classes per app needs | Network, DB, user-defined |

**KEY TAKEAWAY** Parent Delegation Model: a class loader first delegates to its parent; if the parent can't find the class, it tries to load it itself.

# WHEN IS A CLASS LOADED?

*Not every reference to a class causes it to be loaded — only active use does.*

## ROLE OF THE CLASS LOADER SUBSYSTEM



## A CLASS IS LOADED WHEN:

- It is first actively used: new, a static method call, or accessing a static variable.
- `java.lang.Class.forName("...")` is called.
- A subclass is used, or an interface with default methods is used.

## WHY CLASS LOADING MATTERS



### Security

Prevents unauthorized classes.



### Efficiency

Loads classes on demand.



### Flexibility

Supports custom loaders.



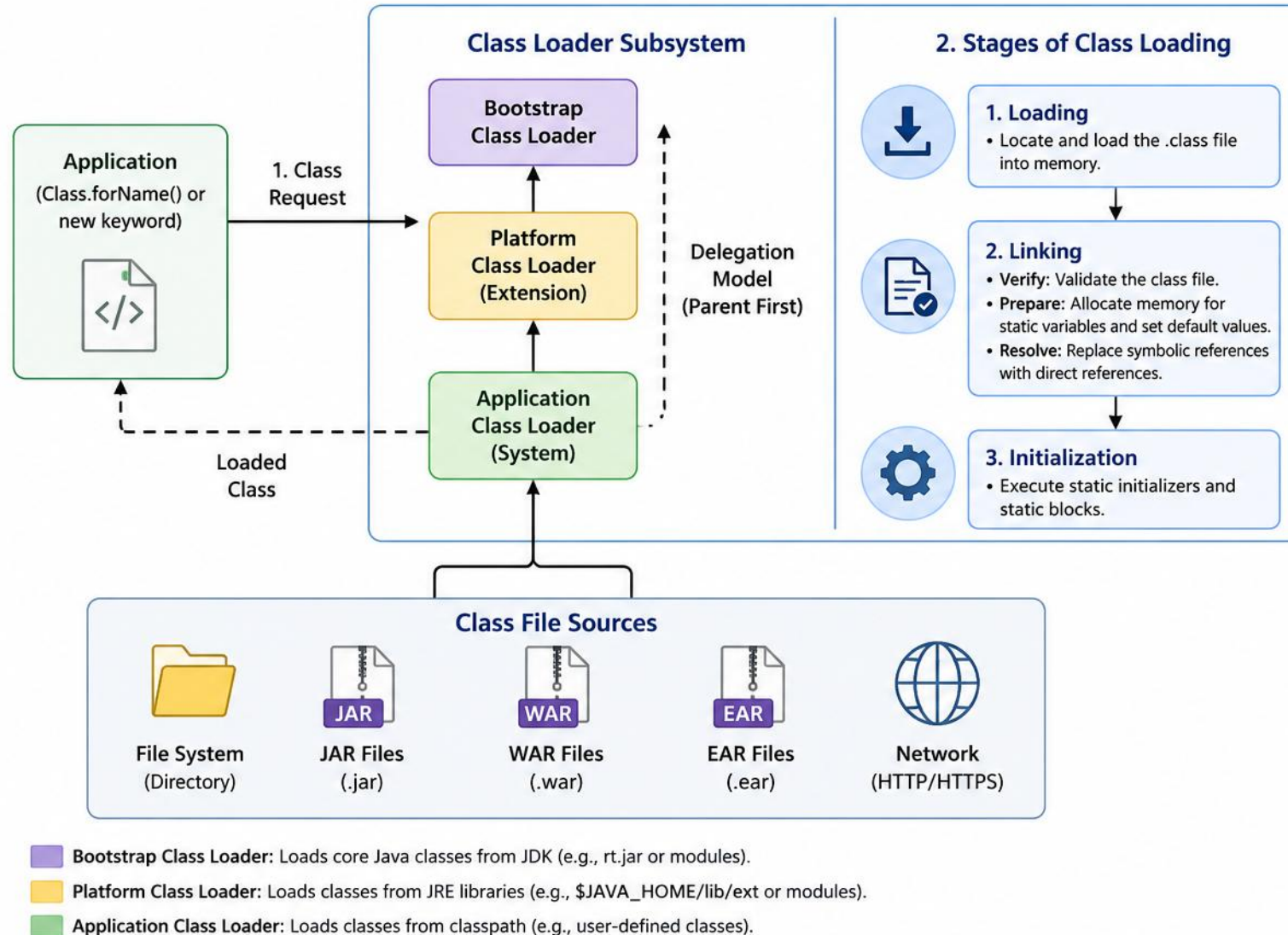
### Platform Independence

Enables portability.

**KEY TAKEAWAY** Understanding class loading helps in debugging `ClassNotFoundException` and `NoClassDefFoundError`, and in building frameworks, containers, and custom class loaders.

# Java Class Loading

The Class Loader Subsystem is responsible for loading .class files into the JVM.



## Key Points

- Uses Delegation Model (Parent First) for security and stability.
- Bootstrap loads core Java classes (rt.jar / modules).
- Application Class Loader loads application-specific classes.



## Benefits

- **Security:** Prevents overriding core classes.
- **Modularity:** Different class loaders can load different versions of classes.
- **Flexibility:** Supports dynamic loading at runtime.

# BOOTSTRAP CLASS LOADER

*The root of Java's class-loading hierarchy — the built-in native class loader in the JVM.*



## Native & Core

- Implemented in native code (C/C++); part of the JVM.



## Always Trusted, No Parent

- Root of the hierarchy; classes cannot be overridden.



## Loads Once

- Classes loaded once and cached in the JVM.

| PACKAGE PREFIX  | EXAMPLES OF CLASSES                                                     |
|-----------------|-------------------------------------------------------------------------|
| java.*          | java.lang.String, java.util.List, java.lang.Object, java.io.InputStream |
| javax.*         | javax.crypto.Cipher, javax.xml.parsers.DocumentBuilder                  |
| jdk.* (Java 9+) | jdk.internal.misc.Unsafe, jdk.nio.*                                     |
| sun.*           | sun.misc.Unsafe, sun.reflect.Reflection                                 |

**KEY TAKEAWAY** You cannot directly extend or customize the Bootstrap Class Loader, and it never loads application or user-defined classes.

# HOW THE BOOTSTRAP CLASS LOADER WORKS

*Every class-loading request starts here — the first stop in the delegation chain.*

- 1 JVM asks the Bootstrap Class Loader first.
- 2 It searches the bootstrap classpath.
- 3 If found, it loads and returns the class.
- 4 If not found, it delegates to the Extension (Platform) Class Loader.
- 5 The hierarchy continues down, or a `ClassNotFoundException` is thrown.

## CHECK IF A CLASS WAS LOADED BY BOOTSTRAP

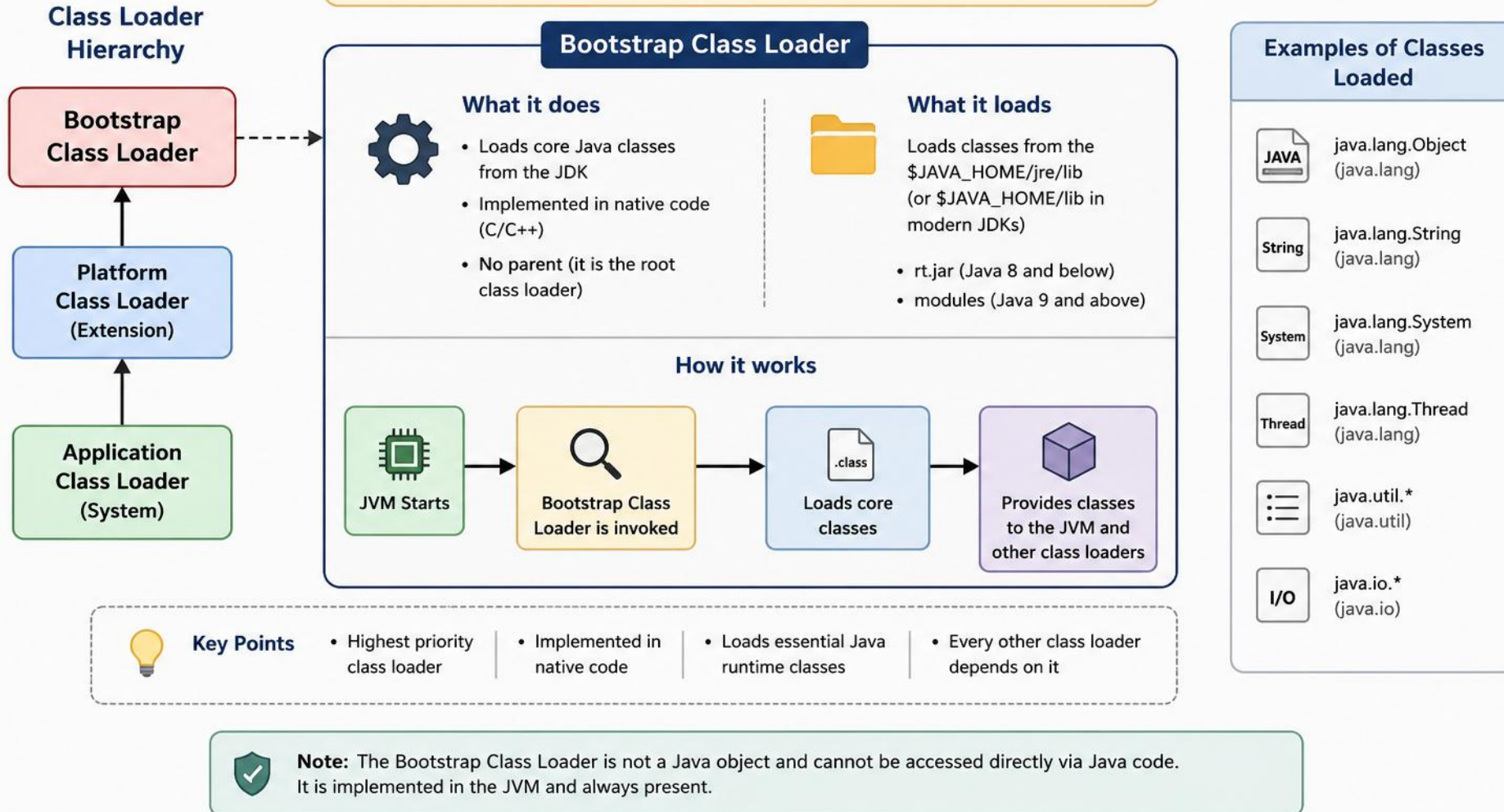
```
System.out.println(String.class.getClassLoader());  
// prints: null
```

**A null return means the class was loaded by the Bootstrap Class Loader — it has no Java representation.**

# Bootstrap Class Loader

## The root of Java Class Loading Hierarchy

Bootstrap Class Loader is the first and highest priority class loader. It loads core Java classes from the JDK and provides the foundational classes required by the JVM.



# PLATFORM CLASS LOADER (EXTENSION CLASS LOADER)

*The bridge between core Java and applications — child of Bootstrap, parent of Application.*



- Implemented in Java, unlike Bootstrap which is native.
- Java 8 and earlier: loads from \$JAVA\_HOME/jre/lib/ext/\*.jar (Extension Class Loader). Java 9+: loads from JDK modules instead.

## WHAT DOES IT LOAD?

| CATEGORY                       | EXAMPLES                                     |
|--------------------------------|----------------------------------------------|
| Platform Modules (Java 9+)     | java.sql, java.management, jdk.crypto.*      |
| Java Standard Extensions       | javax.*, org.w3c.dom, org.xml.sax            |
| User/Custom Platform Libraries | libraries added via ext dir or --module-path |

**KEY TAKEAWAY** The Platform Class Loader ensures both the JDK and applications have access to essential extension APIs.

# HOW THE PLATFORM CLASS LOADER WORKS

*Second stop in the delegation chain — after Bootstrap, before Application.*

- 1 Application requests a class.
- 2 Application Class Loader delegates up to Platform first.
- 3 Platform Class Loader checks its own sources (modules/ext).
- 4 If found, it loads and returns the class.
- 5 If not found, delegation continues to the Application Class Loader.

## KEY POINT

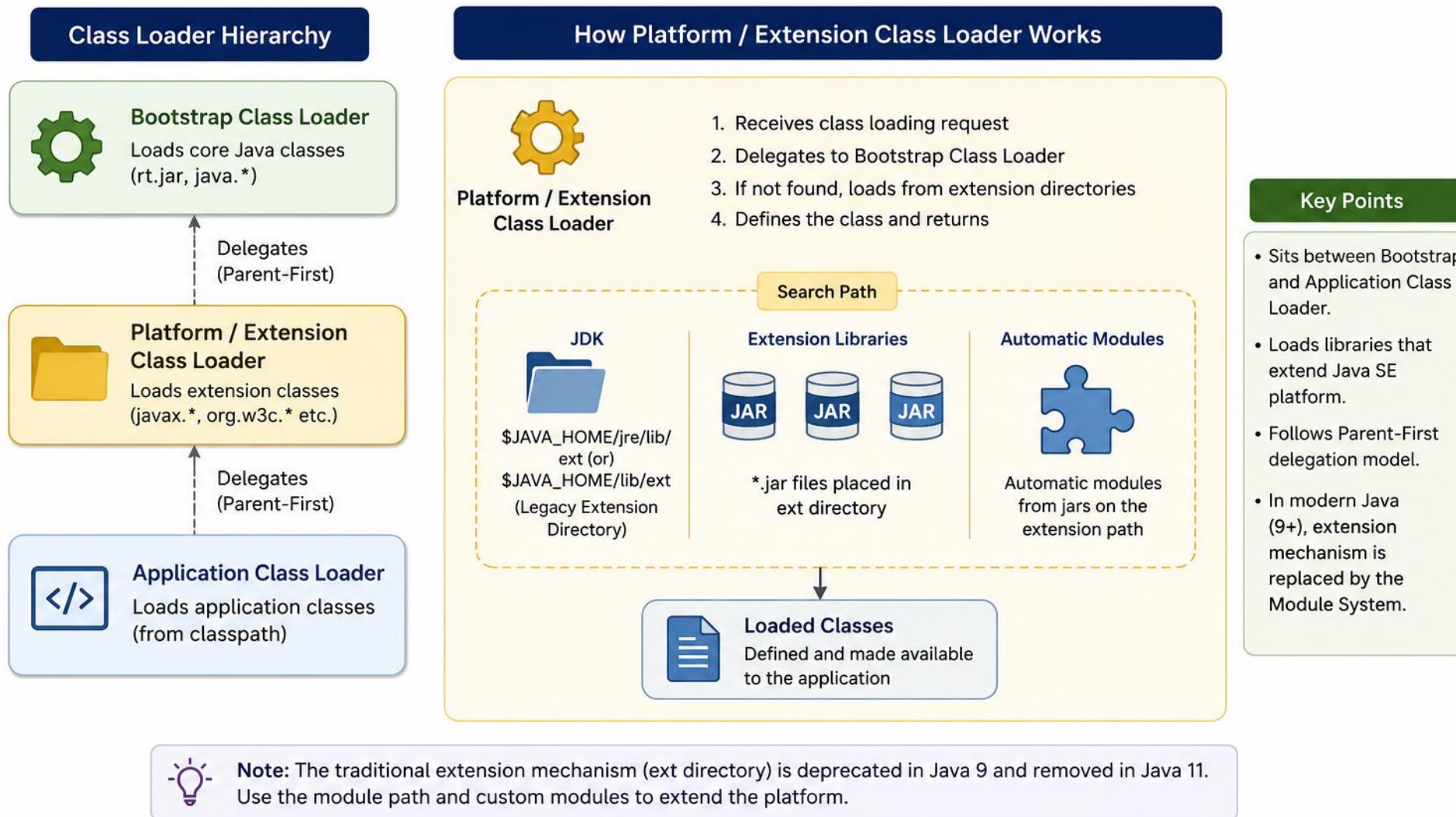
- Platform delegates up to Bootstrap first, then loads its own sources if Bootstrap can't.

## EXAMPLE: CHECKING THE PLATFORM CLASS LOADER

```
System.out.println(MyClass.class.getClassLoader());  
// jdk.internal.loader.ClassLoaders$  
// PlatformClassLoader@5e481248
```

# Platform / Extension Class Loader (Java)

Loads platform specific classes and extension libraries that are not part of the core JDK but are provided to extend the Java SE platform.



# APPLICATION CLASS LOADER (SYSTEM CLASS LOADER)

*The gateway for your application classes — the last loader in the delegation hierarchy.*

- Child of the Platform Class Loader; implemented in Java; default loader for user-defined classes.
- Loads from the application classpath: current directory, JAR files, directories, -cp / -classpath.
- If Bootstrap and Platform loaders don't find a class, the request finally reaches this loader — if still not found, a ClassNotFoundException is thrown.

## WHAT DOES IT LOAD?

| TYPE                        | EXAMPLES                                           |
|-----------------------------|----------------------------------------------------|
| User-defined Classes        | com.example.MyClass, org.myapp.service.UserService |
| Application Libraries       | third-party JARs, custom utility libraries         |
| Application Packages        | com.company.app.*, org.project.module.*            |
| Resources (via ClassLoader) | config.properties, log4j.xml, templates/           |

**KEY TAKEAWAY** Each class loader has its own namespace, enabling isolation — you can create custom loaders by extending ClassLoader.

# HOW THE APPLICATION CLASS LOADER WORKS

*The last loader in the delegation chain — where most of your own code gets loaded.*

- 1 Application code references a class.
- 2 Application Class Loader delegates up to Platform, then Bootstrap.
- 3 If neither finds it, Application Class Loader searches the classpath.
- 4 If found, it loads and returns the class.
- 5 If still not found, a `ClassNotFoundException` is thrown.

## KEY POINT

- This is the loader most developers interact with directly — it's what `-cp` / `-classpath` configures.

## EXAMPLE: CHECKING THE APPLICATION CLASS LOADER

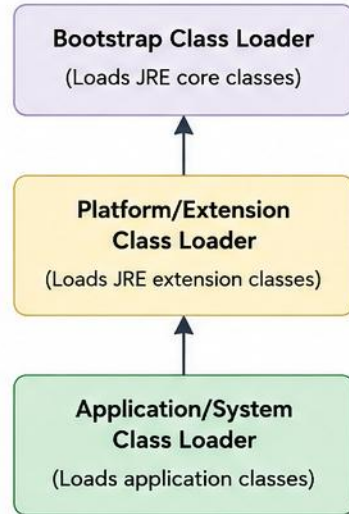
```
System.out.println(MyClass.class.getClassLoader());  
// jdk.internal.loader.ClassLoaders$  
//   AppClassLoader@18b4aac2
```

# Application/System Class Loader



The Application (or System) Class Loader is the default class loader that comes with the JVM. It is responsible for loading classes from the application classpath.

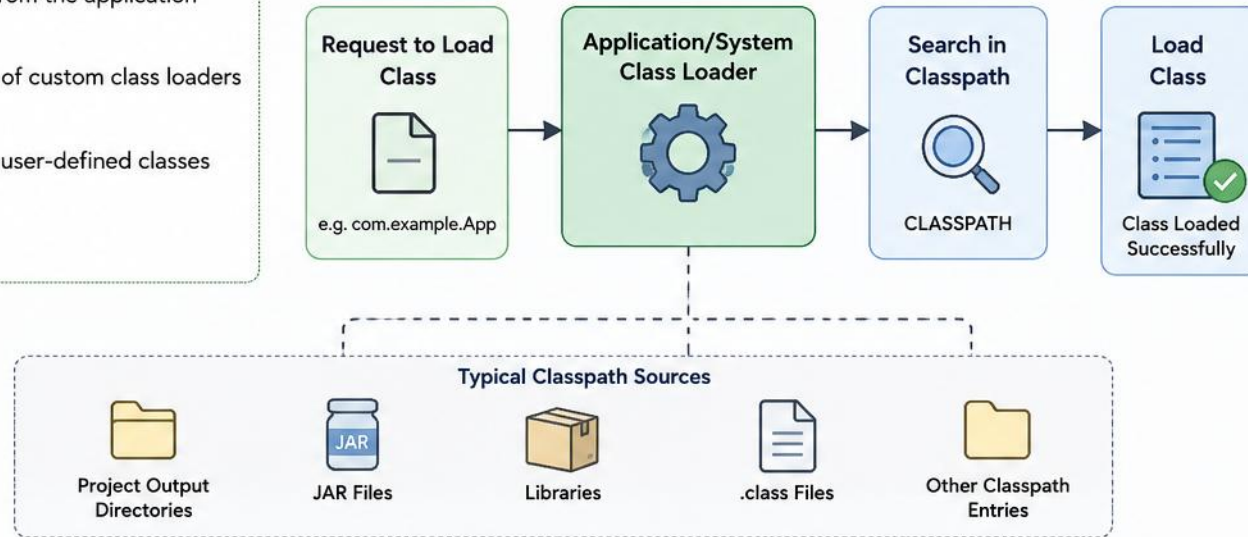
## Class Loader Delegation Hierarchy



## Role

- Loads classes from the application classpath.
- It is the parent of custom class loaders (if any).
- Typically loads user-defined classes and libraries.

## Class Loading Flow



## Key Points

- ✓ It is also called the "System Class Loader".
- ✓ It uses the parent delegation model.
- ✓ You can get a reference to it in Java using: `ClassLoader.getSystemClassLoader()`
- ✓ It loads classes from the `-cp` (or `-classpath`) specified when running the application.

## Example (Java)

```
ClassLoader loader =  
    ClassLoader.getSystemClassLoader();  
System.out.println(loader);  
// Output: sun.misc.Launcher$AppClassLoader@18b4aac2
```

# CLASS LOADING LIFECYCLE

*From .class file to ready-to-use class — seven phases, grouped into three stages.*



| PHASE          | WHAT HAPPENS                                                 | KEY POINT                             |
|----------------|--------------------------------------------------------------|---------------------------------------|
| Loading        | JVM locates .class file, reads binary, creates Class object. | Uses class loader hierarchy.          |
| Verification   | Verifies bytecode format & semantic correctness.             | Protects JVM from invalid bytecode.   |
| Preparation    | Allocates memory for static fields, sets defaults.           | No actual values assigned yet.        |
| Resolution     | Converts symbolic references to direct references.           | May trigger loading of other classes. |
| Initialization | Executes static initializers (<clinit>), only once.          | Only phase that runs app code.        |
| Using          | Class is ready for use.                                      | Normal program execution phase.       |
| Unloading      | Class becomes eligible for GC.                               | Occurs when its ClassLoader is GC'd.  |

**KEY TAKEAWAY** A `LinkageError` is thrown if verification fails; classes loaded by different `ClassLoaders` are treated as different, even from the same .class file.

# BENEFITS OF THE CLASS LOADING LIFECYCLE

*Why the JVM enforces this exact seven-phase sequence, every time.*



## JVM Security & Stability

Ensures the runtime stays secure and stable.



## Prevents Malicious Bytecode

Blocks execution of malicious or malformed bytecode.



## Optimizes Memory

Unloading frees memory when classes are no longer needed.



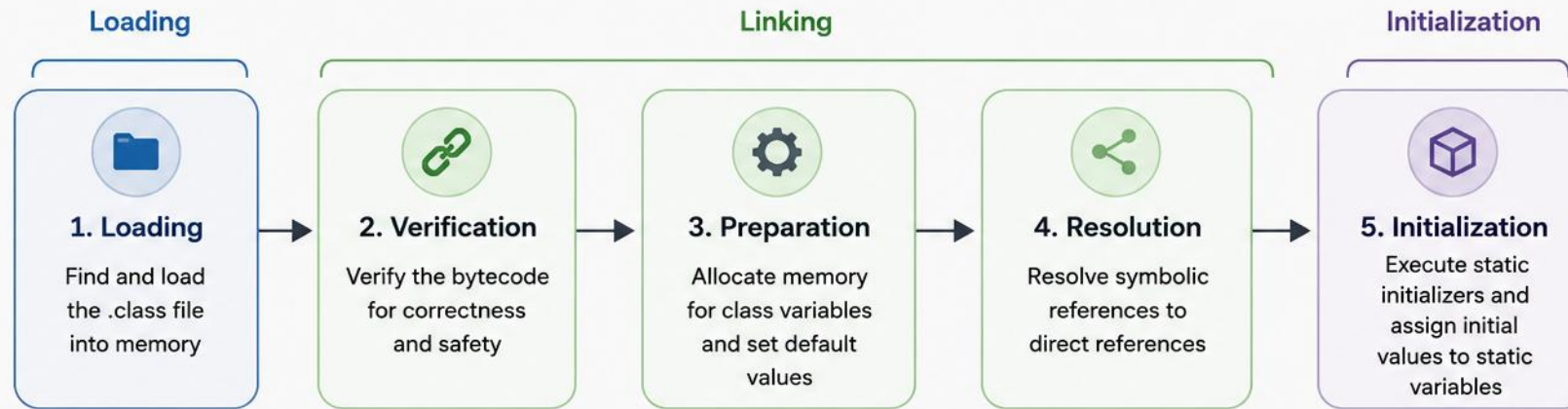
## Structured Management

Gives the JVM a structured way to manage classes.

**KEY TAKEAWAY** If initialization fails, `ExceptionInInitializerError` is thrown — and unloading is never guaranteed, since it depends on garbage collection.

# Class Loading Lifecycle

The process of loading a class into the JVM



## Detailed Overview

|                          |                                                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>1. Loading</b>        | The class loader locates the .class file (from classpath, JAR, etc.) and loads the binary data into the method area. |
| <b>2. Verification</b>   | The JVM verifies the bytecode for format, semantic correctness, and security (type safety).                          |
| <b>3. Preparation</b>    | Memory is allocated for the class's static variables and set to default values (0, null, false, etc.).               |
| <b>4. Resolution</b>     | Symbolic references (e.g., to classes, methods, fields) are resolved to direct references.                           |
| <b>5. Initialization</b> | Static initializers and static blocks are executed in order. Static variables are assigned their defined values.     |

## Key Points

-  Loading is done by Class Loader Subsystem.
-  Linking is the process of preparing the class for use.
-  Initialization occurs only once per class in a **ClassLoader**.
-  After initialization, the class is ready for use.



# TRY IT YOURSELF — EXERCISE 4: WATCH CLASS LOADING

*Reinforces: the Bootstrap / Platform / Application class loaders you just saw.*

| STEP        | WHAT TO DO                                                                        |
|-------------|-----------------------------------------------------------------------------------|
| Goal        | Run Hello with -verbose:class and see which loader loaded it vs. a JDK class      |
| File        | Hello.class (already compiled in Exercise 1)                                      |
| Command     | java -verbose:class Hello                                                         |
| Compare     | java.lang.String is Bootstrap-loaded; Hello is Application-loaded                 |
| Key concept | Bootstrap loads core JDK classes; the Application loader loads your own classpath |
| Reference   | exercises/exercise-04-class-loading.md                                            |

```
$ cd examples/module-01-exercises
$ java -verbose:class Hello
$ java -verbose:class Hello | findstr Hello // Windows: filter to just your class
$ java -verbose:class Hello | grep Hello // macOS/Linux: same filter
```

**TRY IT NOW** Complete Exercise 4 before continuing — see exercises/exercise-04-class-loading.md.

# JVM MEMORY ARCHITECTURE OVERVIEW

*How the JVM organizes and manages memory — divided into specific runtime data areas.*

## SHARED BY ALL THREADS

- Method Area (Metaspace): class metadata.
- Runtime Constant Pool: literals & symbolic refs.
- Heap: all objects and arrays.
- Code Cache: JIT-compiled native code.

## PRIVATE TO EACH THREAD

- JVM Stack: frames for method calls.
- Native Method Stack: native (C/C++) calls.
- PC Register: address of current instruction.

- The Method Area was called Permanent Generation (PermGen) in Java 7 and earlier; in Java 8+ it is Metaspace, outside the heap.
- The Heap is the only area where objects are allocated; use -Xms and -Xmx to set minimum/maximum heap size.

**KEY TAKEAWAY** StackOverflowError comes from exhausted stack memory; OutOfMemoryError comes from exhausted heap or metaspace.

# JVM MEMORY — KEY FEATURES & TUNING

*Automatic management, backed by a handful of tuning flags you'll use constantly.*



**Automatic  
Memory Mgmt**

JVM allocates/reclaims automatically.



**Platform  
Independence**

Same program runs anywhere with a JVM.



**Security**

Isolated memory areas for safe execution.



**Multithreading  
Support**

Per-thread structures ensure safety.



**High  
Performance**

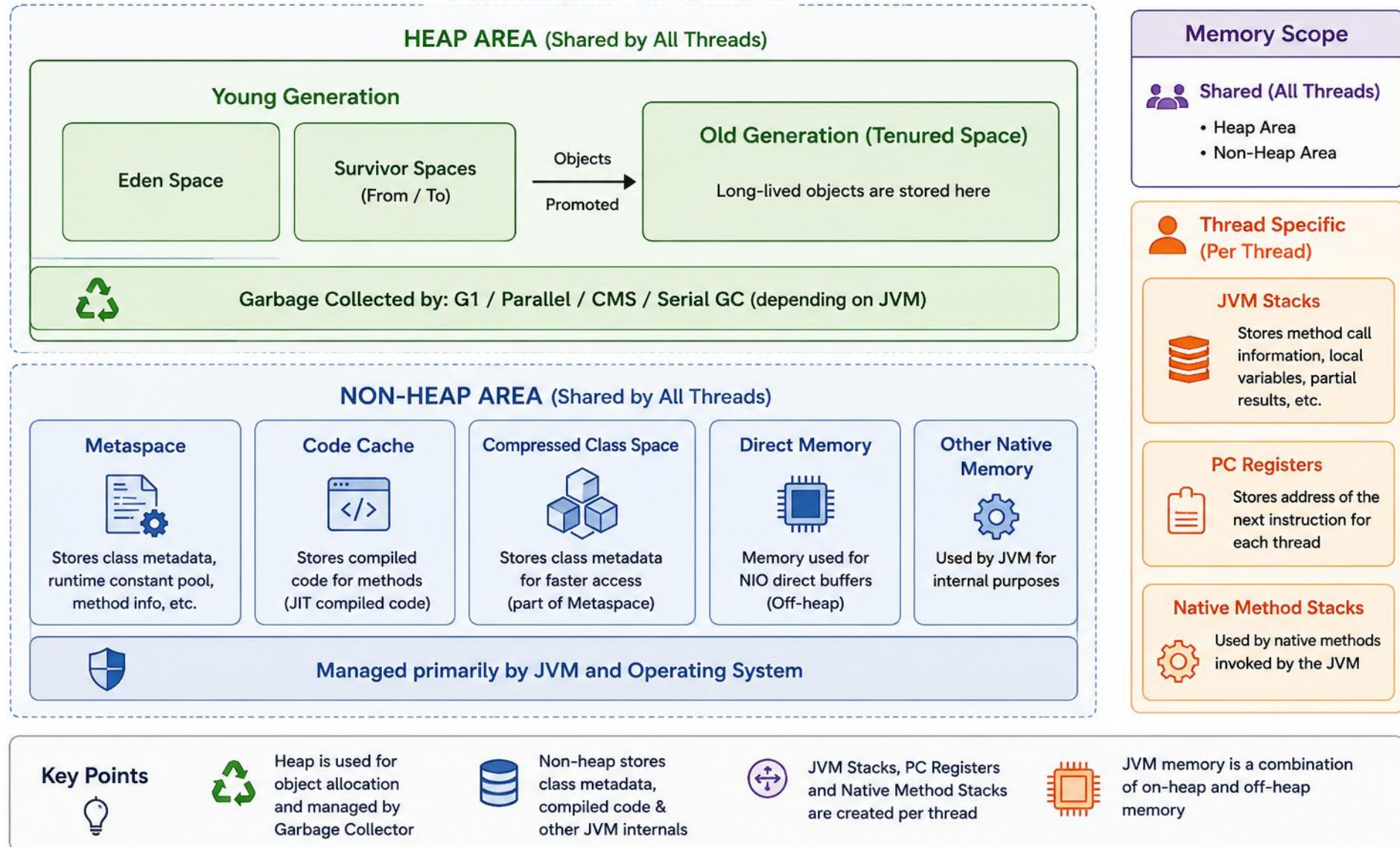
JIT compilation and optimized memory use.

| OPTION                                                | PURPOSE                          |
|-------------------------------------------------------|----------------------------------|
| <code>-Xms&lt;size&gt; / -Xmx&lt;size&gt;</code>      | Initial / maximum heap size      |
| <code>-XX:MetaspaceSize / -XX:MaxMetaspaceSize</code> | Initial / maximum metaspace size |
| <code>-Xss&lt;size&gt;</code>                         | Thread stack size                |
| <code>-XX:+UseG1GC</code>                             | Use the G1 Garbage Collector     |

**KEY TAKEAWAY** Monitor memory with JConsole, VisualVM, or JMC — and tune heap size based on real application needs, not guesswork.

# JVM MEMORY ARCHITECTURE

## JVM Runtime Data Areas



# HEAP MEMORY IN JAVA

*The home of objects — a runtime data area shared by all threads and managed by the Garbage Collector.*

- All object instances and arrays live here; created using the new keyword.
- Heap size is configurable; objects are reclaimed by the GC once no longer reachable.

## HEAP STRUCTURE — GENERATIONS



### Young Generation

- New objects in Eden; survivors move to S0/S1 after Minor GC.



### Old Generation

- Long-lived objects move here after multiple GC cycles; Major GC applies.

**KEY TAKEAWAY** Reference variables live in the Stack; the objects they point to live in the Heap — multiple references can share one object.

# HEAP & GARBAGE COLLECTION

*Minor and Major GC clean different generations of the heap.*

| GC TYPE            | WHEN IT OCCURS              | WHAT IT CLEANS                 | AFFECTS      |
|--------------------|-----------------------------|--------------------------------|--------------|
| Minor GC           | Young Gen (Eden/S0/S1) full | Unreferenced Young Gen objects | Minor pause  |
| Major GC (Full GC) | Old Gen full or triggered   | Unreferenced Old Gen objects   | Longer pause |

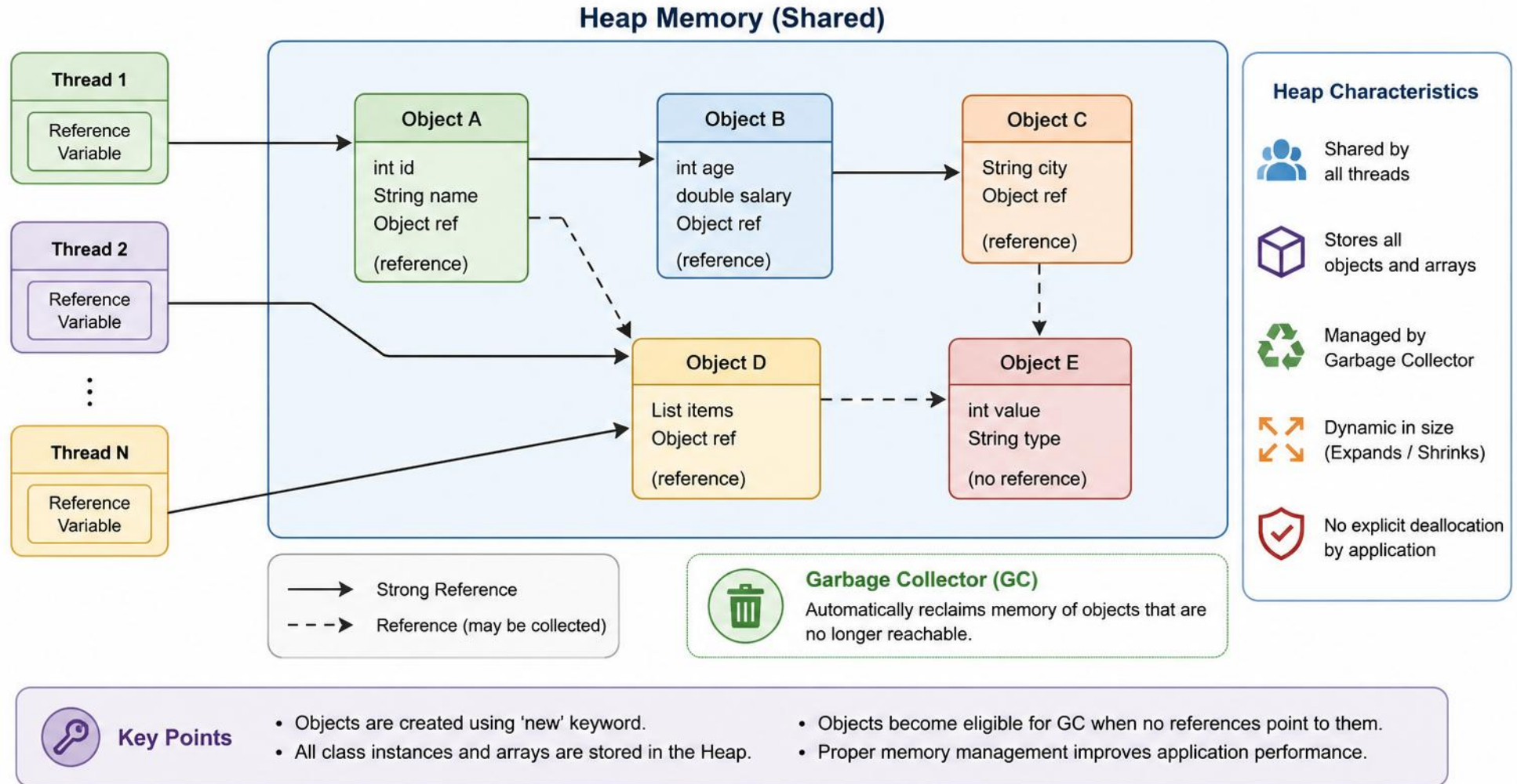
- Mark-Sweep: Mark reachable objects from GC Roots (stack/static refs); Sweep removes unmarked, unreachable objects.

| OPTION                          | DESCRIPTION                 | EXAMPLE           |
|---------------------------------|-----------------------------|-------------------|
| -Xms<size> / -Xmx<size>         | Initial / maximum heap size | -Xms512m / -Xmx2g |
| -XX:+HeapDumpOnOutOfMemoryError | Dump heap when OOM occurs   | (no value)        |

**KEY TAKEAWAY** Heap memory is the heart of Java's dynamic memory management — understand it to write memory-efficient, well-tuned applications.

# Java Heap Memory

The Heap is a runtime data area in the JVM where objects are allocated and stored. It is shared by all threads and managed by the Garbage Collector.



# STACK MEMORY IN JAVA

*The home of method calls and local variables — every thread has its own Stack.*

- Stores local variables, method parameters, and return addresses; follows LIFO order.
- Memory is automatically allocated and deallocated — no manual management needed.
- Stack size is limited and typically smaller than the Heap.

## STACK FRAME (ACTIVATION RECORD)



## COMMON JVM OPTIONS FOR STACK

| OPTION                  | PURPOSE                      |
|-------------------------|------------------------------|
| -Xss<size>              | Sets thread stack size       |
| -XX:ThreadStackSize=<k> | Sets thread stack size in KB |
| -XX:+PrintFlagsFinal    | View the default stack size  |

**KEY TAKEAWAY** Default stack size varies by platform (typically 1MB on 64-bit systems) — the frame is pushed on call, popped on return.

# STACK VS HEAP & COMMON ERRORS

*The two memory areas every Java developer must be able to reason about.*

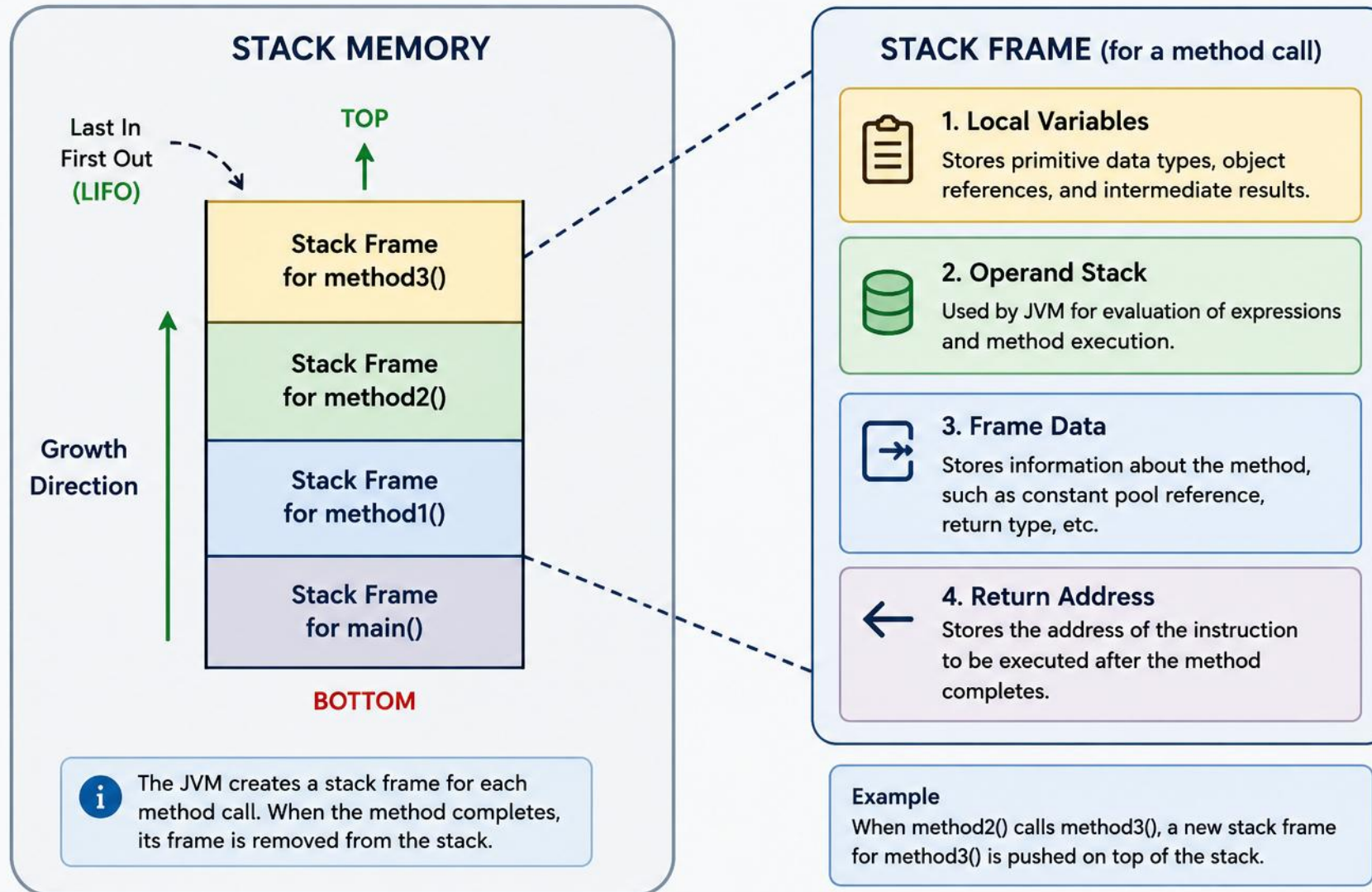
| FEATURE      | STACK                              | HEAP                       |
|--------------|------------------------------------|----------------------------|
| Scope        | Per thread                         | Shared by all threads      |
| Stores       | Locals, parameters, return address | Objects and arrays         |
| Size         | Smaller, limited                   | Larger, configurable       |
| Management   | Automatic push/pop                 | Garbage collected          |
| Access Speed | Faster                             | Relatively slower          |
| Lifetime     | Method call ends                   | Until no references remain |

- `StackOverflowError`: stack memory full due to deep recursion or too many nested calls.
- `OutOfMemoryError`: unable to create new native thread — insufficient stack memory or OS thread limits.

**KEY TAKEAWAY** Stack memory is fast, thread-specific, and automatically managed — critical for every method call in Java.

# STACK MEMORY IN JAVA

The stack memory stores method calls and local variables.



# TRY IT YOURSELF — EXERCISE 5: VARIABLES AND DATA TYPES

*Reinforces: local variables live in the stack frame you just saw.*

| STEP        | WHAT TO DO                                                                                                 |
|-------------|------------------------------------------------------------------------------------------------------------|
| Goal        | Create Variables.java with several primitive types and one String; print each                              |
| File        | Variables.java (in examples/module-01-exercises/)                                                          |
| Compile     | javac Variables.java                                                                                       |
| Run         | java Variables                                                                                             |
| Key concept | int, long, double, boolean, and char are stored directly in the method's local variable slots on the stack |
| Reference   | exercises/exercise-05-variables.md                                                                         |

```
$ cd examples/module-01-exercises
$ javac Variables.java
$ java Variables // prints age, population, price, enrolled, grade, and name
$ javap -c Variables // istore / iload instructions show each local variable's stack slot
```

**TRY IT NOW** Complete Exercise 5 before continuing — see exercises/exercise-05-variables.md.

# TRY IT YOURSELF — EXERCISE 6: METHODS AND PARAMETERS

*Reinforces: every method call pushes a new frame onto the stack you just saw.*

| STEP        | WHAT TO DO                                                                                        |
|-------------|---------------------------------------------------------------------------------------------------|
| Goal        | Create Methods.java with two methods that take parameters and return a value                      |
| File        | Methods.java (in examples/module-01-exercises/)                                                   |
| Compile     | javac Methods.java                                                                                |
| Run         | java Methods — main calls add(...) and greet(...)                                                 |
| Key concept | Each call gets its own short-lived stack frame holding locals, parameters, and the return address |
| Reference   | exercises/exercise-06-methods.md                                                                  |

```
$ cd examples/module-01-exercises
$ javac Methods.java
$ java Methods // prints 30 and "Hello, Aman!"
$ javap -c Methods // invokestatic calls add/greet; a new frame opens for each
```

**TRY IT NOW** Complete Exercise 6 before continuing — see exercises/exercise-06-methods.md.

# METASPACE IN JAVA

*The native memory for class metadata — replaces PermGen from Java 8 onward.*

- Stores class metadata: class info, methods, fields, constant pool, and more.
- Allocated in native memory outside the JVM heap; grows dynamically as classes load.
- Class metadata is reclaimed when classes are unloaded and no longer in use.

| FEATURE          | PERMGEN (JAVA 7-)                | METASPACE (JAVA 8+)                |
|------------------|----------------------------------|------------------------------------|
| Memory Location  | Inside JVM heap space            | Native memory outside heap         |
| Size             | Fixed (-XX:PermSize/MaxPermSize) | Dynamic, grows as needed           |
| OutOfMemoryError | OOME: PermGen space              | OOME: Metaspace                    |
| Tuning           | Difficult to tune                | Easier, with simple limits         |
| Class Unloading  | Supported                        | Supported, with better scalability |
| Default Size     | Smaller                          | Larger and more flexible           |

**KEY TAKEAWAY** Metaspace is easier to tune than PermGen and scales better for class unloading.

# WHAT DOES METASPACE STORE?

*Everything the JVM needs to know about a loaded class — never the objects themselves.*

| CATEGORY        | DESCRIPTION                                                    |
|-----------------|----------------------------------------------------------------|
| Class Metadata  | Structure of each loaded class.                                |
| Method Metadata | Bytecode and metadata for each method.                         |
| Field Metadata  | Field names, types, and modifiers.                             |
| Constant Pool   | Literals and symbolic references used by the class.            |
| Other Metadata  | Annotations, generic signatures, and related class-level data. |

**KEY TAKEAWAY** Metaspace stores metadata about classes — actual objects and arrays always live on the Heap.

# METASPACE — FLOW & TUNING

*Class loading feeds Metaspace; objects still go to the Heap.*



| OPTION                             | PURPOSE                                 |
|------------------------------------|-----------------------------------------|
| -XX:MetaspaceSize=<size>           | Initial metaspace size                  |
| -XX:MaxMetaspaceSize=<size>        | Maximum metaspace size                  |
| -XX:+PrintMetaspaceStatistics      | Print metaspace statistics              |
| -XX:+TraceClassLoading / Unloading | Trace class loading/unloading           |
| -XX:MinMetaspaceFreeRatio=<n>      | Min free space % before GC (default 40) |
| -XX:MaxMetaspaceFreeRatio=<n>      | Max free space % after GC (default 70)  |

**KEY TAKEAWAY** Metaspace grows dynamically, supports class unloading, and proper tuning helps avoid OutOfMemoryError: Metaspace.

# EXAMPLE: VIEW METASPACE USAGE

*Two command-line tools to inspect Metaspace at runtime.*

## jcmm <pid> VM.metaspace

```
Metaspace      used 45.2MB, capacity 47.8MB, committed 48.0MB, reserved 1088.0MB
Class space    used 5.1MB, capacity 5.5MB, committed 5.5MB, reserved 1024.0MB
Total class count: 4213
```

## jstat -gc <pid>

| S0    | S1  | E    | O    | M     | CCS  | YGC | YGCT  |
|-------|-----|------|------|-------|------|-----|-------|
| 512.0 | 0.0 | 1024 | 8192 | 46208 | 5632 | 12  | 0.045 |

**KEY TAKEAWAY** jcmm VM.metaspace shows a detailed breakdown; jstat -gc's M/CCS columns track Metaspace and compressed class space over time.

# OBJECT CREATION AND MEMORY ALLOCATION

*From the new keyword to an object living in the Heap, with its reference in the Stack.*



| # | STEP                   | DETAILS                                            |
|---|------------------------|----------------------------------------------------|
| 1 | new keyword            | JVM encounters new Person(101, "Alice").           |
| 2 | Class Loading          | JVM loads Person if not already loaded.            |
| 3 | Memory Allocation      | Allocates Heap memory based on class structure.    |
| 4 | Constructor Invocation | Calls Person(int, String) to initialize fields.    |
| 5 | Reference Assignment   | Reference stored in variable p in the stack frame. |

**KEY TAKEAWAY** Objects are always created in the Heap; reference variables always live in the Stack.

# OBJECT LIFECYCLE & COMMON MISTAKES

*From creation to garbage collection, and the pitfalls to avoid along the way.*



- `==` compares references, not contents — a common source of subtle bugs.
- Objects are stored in the Heap, not the Stack — the Stack only holds references.
- Setting a reference to null does not immediately delete the object; the GC frees memory later.
- Creating unnecessary objects increases memory usage and GC pressure.

**KEY TAKEAWAY** `new` creates an object in the Heap, the constructor initializes it, and a reference is stored in the Stack.

# MEMORY LAYOUT OF AN OBJECT IN HEAP

*Every object on the Heap has the same three-part internal structure.*



## Object Header

- Class metadata, hashCode, GC info, and lock (monitor) info.



## Instance Data

- The actual fields of the class and all its superclasses.

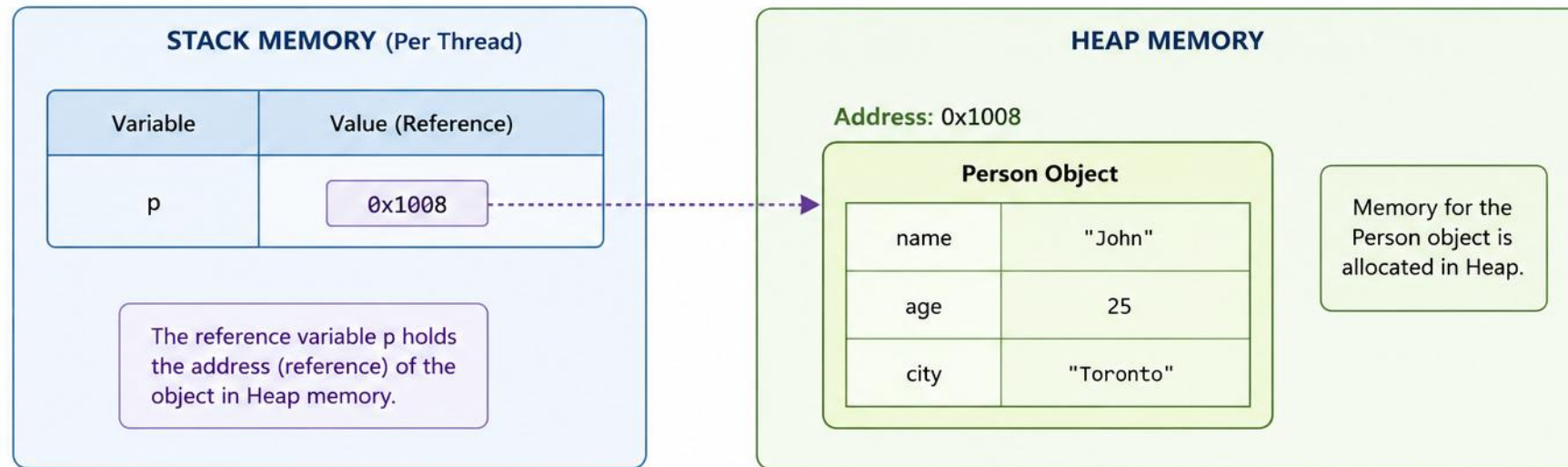
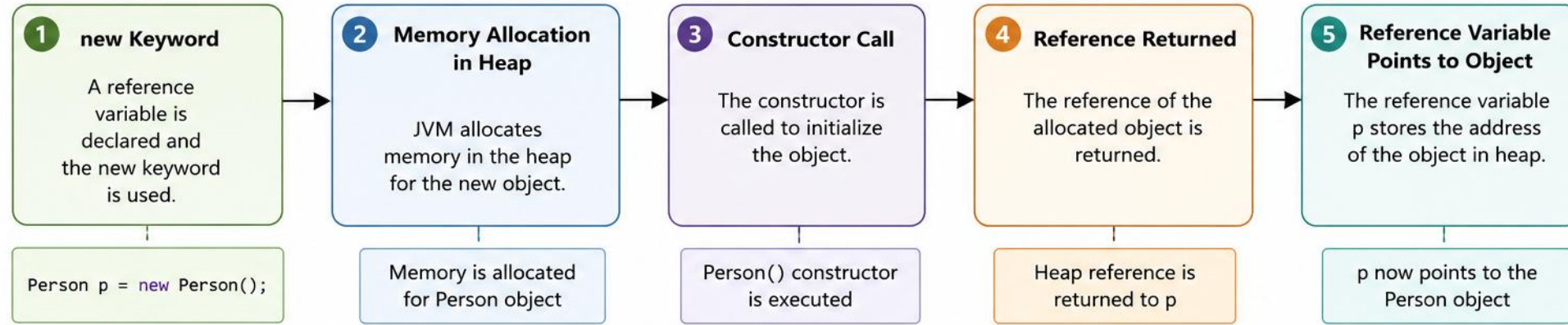


## Padding / Alignment

- Optional padding so the object size aligns to 8-byte boundaries.

**KEY TAKEAWAY** Actual object size varies depending on the fields declared, the JVM implementation, and the CPU architecture.

# Object Creation and Memory Allocation in Java



## Key Points

- Objects are always created in Heap memory.

- Reference variables are stored in Stack memory.

- Stack holds references, Heap holds actual objects.

- When there are no references, the object is eligible for Garbage Collection.



# TRY IT YOURSELF — EXERCISE 7: OBJECTS AND CLASSES

*Reinforces: the heap object layout you just saw.*

| STEP        | WHAT TO DO                                                                                                      |
|-------------|-----------------------------------------------------------------------------------------------------------------|
| Goal        | Create Person.java with fields, a constructor, and a method; instantiate it in main                             |
| File        | Person.java (in examples/module-01-exercises/)                                                                  |
| Compile     | javac Person.java                                                                                               |
| Run         | java Person                                                                                                     |
| Key concept | new Person(...) allocates the object's fields on the heap; person itself is only a reference, held on the stack |
| Reference   | exercises/exercise-07-objects.md                                                                                |

```
$ cd examples/module-01-exercises
$ javac Person.java
$ java Person // prints: Aman is 21 years old
$ javap -c Person // putfield stores name/age into the new object on the heap
```

**TRY IT NOW** Complete Exercise 7 before continuing — see exercises/exercise-07-objects.md.

# HEAP VS STACK COMPARISON

*Java uses Stack memory for method execution and Heap memory for objects and dynamic data.*

| FEATURE           | STACK                                | HEAP                         |
|-------------------|--------------------------------------|------------------------------|
| What it stores    | Primitives, references, method calls | Objects and arrays           |
| Speed             | Very fast                            | Slower than stack            |
| Size              | Small                                | Large                        |
| Memory Allocation | Static allocation                    | Dynamic allocation           |
| Management        | Automatically removed                | Garbage collected by JVM     |
| Lifetime          | Method call ends                     | Until no references remain   |
| Examples          | Local variables, return addresses    | Objects, Arrays, Collections |
| Safety            | Each thread has its own stack        | Shared among all threads     |

**KEY TAKEAWAY** int x is a primitive in the Stack; String name is a reference in the Stack pointing to a "Java" object in the Heap.

# WHY GARBAGE COLLECTION IS NEEDED

*Java creates many objects during execution — when they're no longer needed, GC automatically reclaims their memory.*

## Without GC

- Memory keeps filling up — eventually OutOfMemoryError.

## With GC

- Unused memory is reclaimed — application runs smoothly.

## WHY GARBAGE COLLECTION IS NEEDED

### Prevents Leaks

Releases unreferenced memory.

### Improves Performance

Frees memory to run efficiently.

### Prevents OOM

Avoids JVM crashes.

### Simplifies Dev

No manual memory freeing.

### Better Reuse

Memory reused for new objects.

**KEY TAKEAWAY** How it works, high level: Mark reachable objects from GC Roots → Sweep unreferenced objects → Compact (optional) → memory reused.

# OBJECT REACHABILITY & CONSEQUENCES OF NO GC

*Not every object is treated the same way by the collector.*

| TYPE                  | DESCRIPTION                                   | EXAMPLE                         |
|-----------------------|-----------------------------------------------|---------------------------------|
| Reachable (Live)      | Accessible directly/indirectly from GC Roots  | Objects referenced by variables |
| Unreachable (Garbage) | No references point to these objects          | Orphaned objects                |
| Weakly Reachable      | Only weak references exist; eligible for GC   | Soft/Weak/Phantom references    |
| Finalizable           | Has a finalize method; needs special handling | Objects awaiting finalization   |

- Without GC: memory exhaustion (OutOfMemoryError), performance degradation, unpredictable crashes.
- Poor resource utilization and harder maintenance — manual memory management is error-prone.

**KEY TAKEAWAY** Garbage Collection is essential for automatic memory management — it keeps applications efficient, stable and scalable.

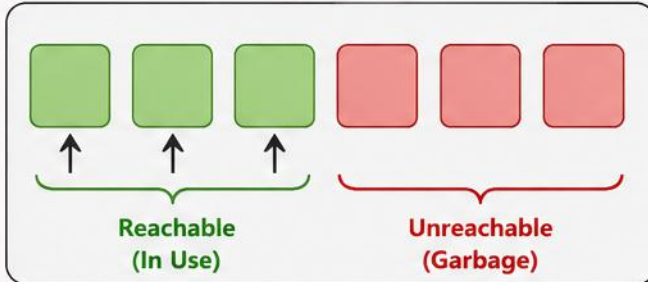
# Why Garbage Collection Needed in Java?

In Java, objects are created in heap memory. When objects are no longer used, they become **unreachable**. Garbage Collection (GC) automatically reclaims that memory, preventing memory leaks and **OutOfMemoryError**.

## Without Garbage Collection

Unreferenced objects remain in memory. Heap keeps filling up → Memory leak → **OutOfMemoryError**

Heap Memory



### Heap gets full

Application slows down and may crash with **OutOfMemoryError**

## Benefits of Garbage Collection



**Reclaims memory** occupied by unreachable objects



**Improves application** performance and stability



**Prevents memory leaks** and **OutOfMemoryError**

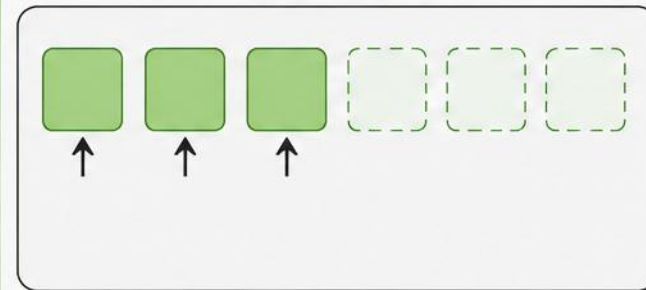


**Automatic memory management** – less developer effort

## With Garbage Collection

Unreachable objects are detected and memory is reclaimed for future use.

Heap Memory



### Memory is freed

Application runs smoothly with optimal memory usage



Live / Reachable Object



Unreachable Object (Garbage)



Reference (In Use)



Freed Memory



**Key Takeaway:** Garbage Collection is essential in Java to automatically free up memory from unreachable objects, ensuring better performance, reliability, and a better developer experience.

# OBJECT LIFECYCLE IN JAVA

*Every object goes through the same lifecycle, from creation to memory being reclaimed.*



- An object is reachable if it can be accessed directly or indirectly from GC Roots — stack variables, static variables, JNI references, and JVM internal references.
- Strong reference means reachable; no reference means unreachable and eligible for GC.
- You do not need to delete objects in Java — remove references when no longer needed and let the Garbage Collector do the rest.

**KEY TAKEAWAY** An object remains in memory as long as it is reachable — once unreachable, it becomes a GC candidate.

# Object Lifecycle in Java

From Creation to Garbage Collection

## Key Points

- ✓ An object is created in heap memory.
- ✓ It remains in use as long as it is reachable.
- ✓ When no references point to it, it becomes unreachable.
- ✓ Garbage Collector reclaims memory at some later time.
- ✓ The timing of garbage collection is not deterministic.

## Legend

- Active State
- Reachable State
- Unreachable State
- GC Eligibility
- Collected State



### 1. Creation

Object is created using **new** keyword.



### 2. In Use (Reachable)

Object is referenced by variable(s) and can be used.



### 3. Unreachable

No references point to this object anymore.



### 4. Eligible for Garbage Collection

JVM identifies unreachable objects as candidates for cleanup.



### 5. Garbage Collected

Memory occupied by the object is reclaimed by Garbage Collector.

## Example

```
Student s = new Student();  
s.setName("Alice");  
s = null; // no more reference  
// Eligible for GC  
// Garbage Collector reclaims it  
// at some later time
```

## Heap Memory



Object Created



Object in Use



Unreachable



GC Reclaims Memory



**Note:** Garbage collection in Java is automatic, but not immediate. The JVM decides when and how to reclaim memory.

# HOW GARBAGE COLLECTION WORKS IN JAVA

*GC identifies objects that are no longer reachable and reclaims their memory for reuse.*



## GC ROOTS — STARTING POINTS

- Local variables on the Stack, Static variables, Active Threads, JNI References, JVM Internal References.
- GC traverses the object graph from these roots and marks everything reachable; unmarked objects are unreachable and eligible for removal.

**KEY TAKEAWAY** Some GC algorithms compact the heap afterward, moving live objects together to reduce fragmentation.

# COMMON GC ALGORITHMS

*Four collectors, each trading off pause time, throughput, and compaction differently.*

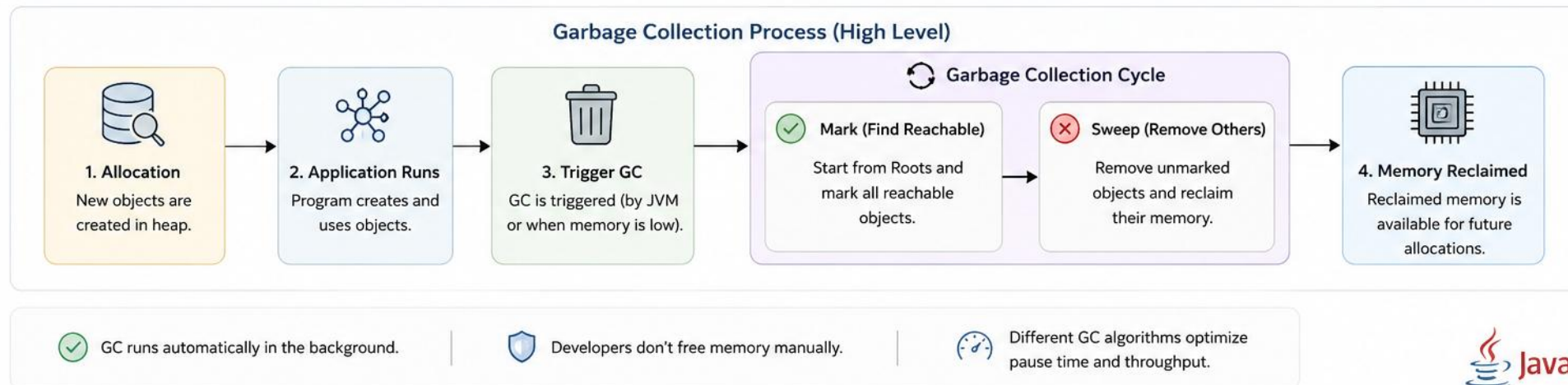
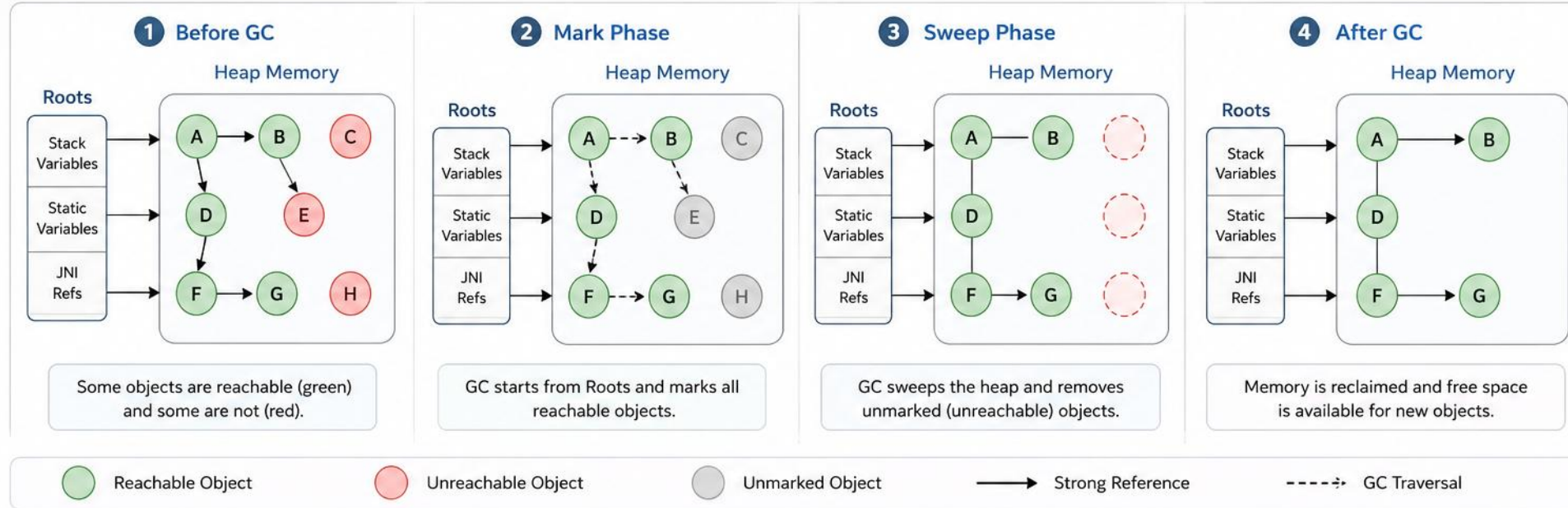
| ALGORITHM   | TYPE               | KEY IDEA                    | COMPACTION | USE CASE                 |
|-------------|--------------------|-----------------------------|------------|--------------------------|
| Serial GC   | Stop-the-world     | Mark-Sweep-Compact          | Yes        | Small heaps, simple apps |
| Parallel GC | STW, multithreaded | Parallel Mark-Sweep-Compact | Yes        | Throughput oriented      |
| CMS         | Mostly concurrent  | Mark-Sweep, no compact      | No         | Low pause, deprecated    |
| G1 GC       | Mostly concurrent  | Region-based, incremental   | Sometimes  | Large heaps, balanced    |

- Useful JVM options: -XX:+UseG1GC, -XX:+PrintGCDetails, -verbose:gc, -Xlog:gc\* (JDK 9+).

**KEY TAKEAWAY** GC runs automatically — developers don't free memory manually, but choosing and tuning the right algorithm still matters.

# How Garbage Collection Works in Java

Garbage Collection reclaims memory occupied by objects that are no longer reachable.



# OVERVIEW OF G1 GARBAGE COLLECTOR

*Garbage-First — a server-style collector designed for large heaps, default since Java 9.*



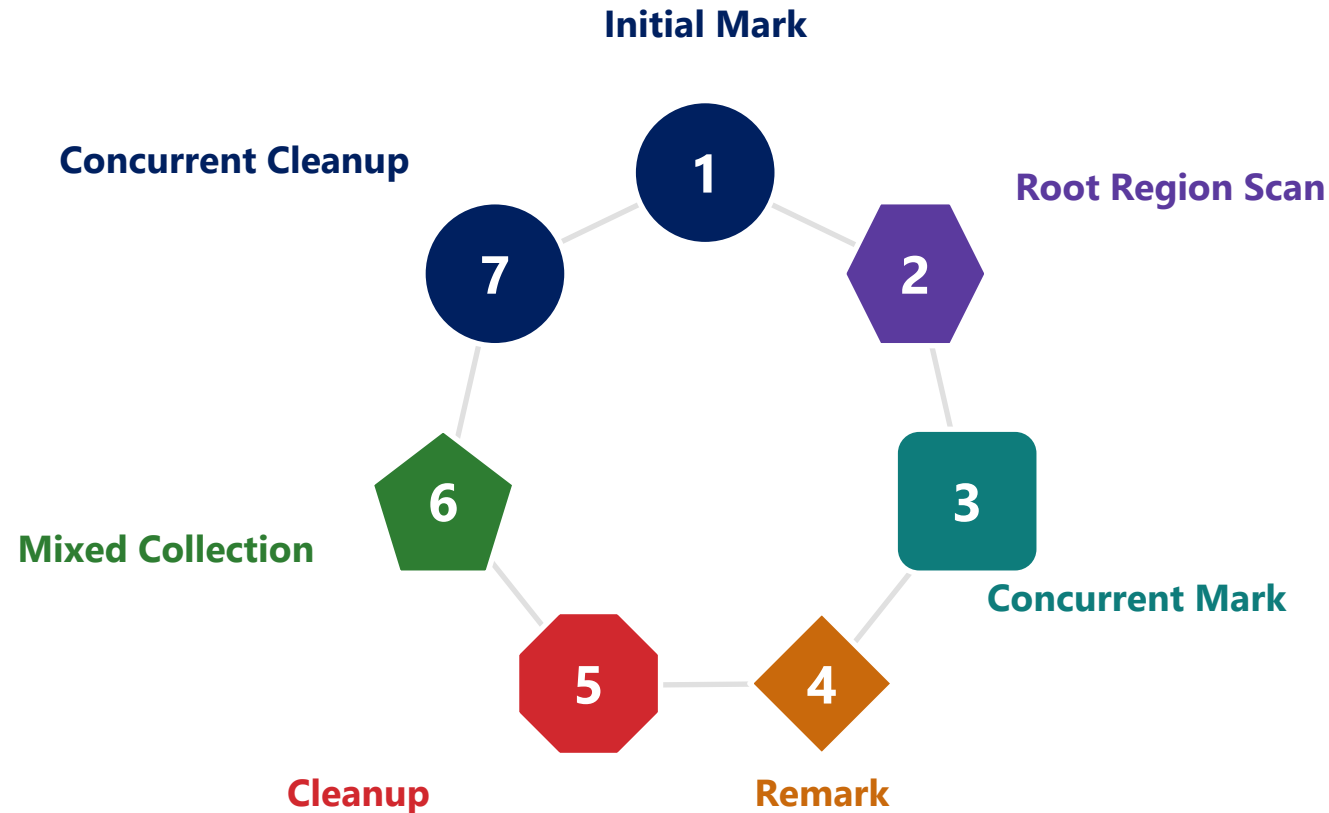
## HEAP LAYOUT — REGION BASED



**KEY TAKEAWAY** G1 divides the heap into many equal-sized regions (a power of 2 between 1 MB and 32 MB, chosen automatically).

# G1 GC CYCLE & KEY OPTIONS

*G1 works in cycles, alternating short stop-the-world pauses with concurrent phases.*



**KEY TAKEAWAY** Key options: `-XX:+UseG1GC` · `-XX:MaxGCPauseMillis (200)` · `-XX:InitiatingHeapOccupancyPercent (45)`. Humongous objects are moved only during a Full GC.

# G1 GC CYCLE — DETAILS & KEY OPTIONS

*What actually happens during each phase, and the flags that control them.*

| PHASE              | TYPE             | WHAT HAPPENS                                                |
|--------------------|------------------|-------------------------------------------------------------|
| Initial Mark       | STW              | Marks GC Roots and directly reachable objects.              |
| Root Region Scan   | STW, short       | Scans Root regions (e.g. Survivor/Old) for references.      |
| Concurrent Mark    | Concurrent       | Marks live objects across the heap while the app runs.      |
| Remark             | STW              | Finalizes marking; processes reference queues.              |
| Cleanup            | STW + Concurrent | Reclaims fully-empty regions; updates statistics.           |
| Mixed Collection   | STW              | Evacuates live objects from selected regions to free space. |
| Concurrent Cleanup | Concurrent       | Clears remembered sets, prepares for the next collection.   |

## MORE JVM OPTIONS FOR G1

| OPTION                | PURPOSE                                |
|-----------------------|----------------------------------------|
| -XX:G1HeapRegionSize  | Region size, 1M–32M (auto)             |
| -XX:ParallelGCThreads | Number of GC worker threads (auto)     |
| -XX:ConcGCThreads     | Number of concurrent GC threads (auto) |

# G1 VS OTHER COLLECTORS

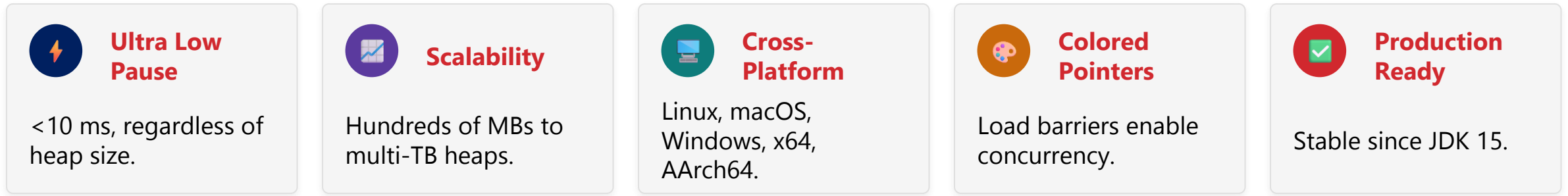
*How G1 compares against Serial/Parallel and CMS across the dimensions that matter.*

| ASPECT            | SERIAL / PARALLEL  | CMS                          | G1                              |
|-------------------|--------------------|------------------------------|---------------------------------|
| Pause Time        | Long               | Shorter than Serial/Parallel | Short, predictable (goal-based) |
| Throughput        | Highest            | Lower (concurrent overhead)  | High, tunable                   |
| Compaction        | Yes (STW)          | No (fragmentation risk)      | Yes, incremental                |
| Heap Size Support | Small–Medium       | Medium                       | Large (multi-GB+)               |
| Tuning Complexity | Low                | High                         | Moderate                        |
| Default in Java   | Up to 8 (Parallel) | Removed in Java 14           | 9+ (most cases)                 |

**KEY TAKEAWAY** G1 became the default collector in Java 9 because it balances throughput and pause time far better than its predecessors at scale.

# OVERVIEW OF ZGC

*A scalable, low-latency collector — production-ready since JDK 15, aiming for pauses under 10 ms.*



## HOW ZGC WORKS — HIGH LEVEL



**KEY TAKEAWAY** Most ZGC work is concurrent with application threads — only short pauses handle root marking, remapping and finalization.

# ZGC — KEY FEATURES

*Six characteristics that let ZGC hold sub-10ms pauses even on multi-terabyte heaps.*



## Low Pause Times

- <10 ms, independent of heap size.



## Concurrent by Design

- Marking, relocating, reclamation all concurrent.



## Load Barrier

- Handles concurrent movement/remapping on every load.



## Region-Based

- Heap divided into 2 MB regions by default.



## Dynamic Resizing

- Grows and shrinks heap with low latency.



## Generational (JDK 21+)

- Generational mode improves performance further.

- Key options: -XX:+UseZGC · -Xms / -Xmx · -XX:ZCollectionInterval · -Xlog:gc\* (recommended).

**KEY TAKEAWAY** Use ZGC when every millisecond matters — it is designed for large heaps and latency-sensitive applications.

# ZGC VS OTHER COLLECTORS

*Where ZGC fits against Serial/Parallel and G1 on latency-critical workloads.*

| ASPECT                 | SERIAL / PARALLEL      | G1                       | ZGC                           |
|------------------------|------------------------|--------------------------|-------------------------------|
| Pause Time             | Long                   | Short, predictable       | Ultra-low (<10ms)             |
| Throughput             | Highest                | High, tunable            | Slightly lower than G1        |
| Scalability            | Small–Medium heaps     | Large heaps              | Multi-TB heaps                |
| Latency Predictability | Low                    | Moderate                 | Very high                     |
| Best Use Case          | Batch jobs, small apps | General-purpose services | Latency-sensitive, huge heaps |

## MORE JVM OPTIONS FOR ZGC

| OPTION             | PURPOSE                                                |
|--------------------|--------------------------------------------------------|
| -XX:ZUncommitDelay | Delay before uncommitting unused memory (default 300s) |
| -XX:ConcGCThreads  | Number of concurrent GC threads (auto)                 |

# Overview of ZGC

ZGC (Z Garbage Collector) is a scalable, low-latency garbage collector designed for modern, large-heap applications.

## Key Features



### Low Latency

Designed to keep GC pauses typically under 10ms, regardless of heap size.



### Scalability

Handles multi-terabyte heaps and large numbers of application threads.



### Concurrent by Design

Most of the work is done concurrently with the application.



### Load Barriers

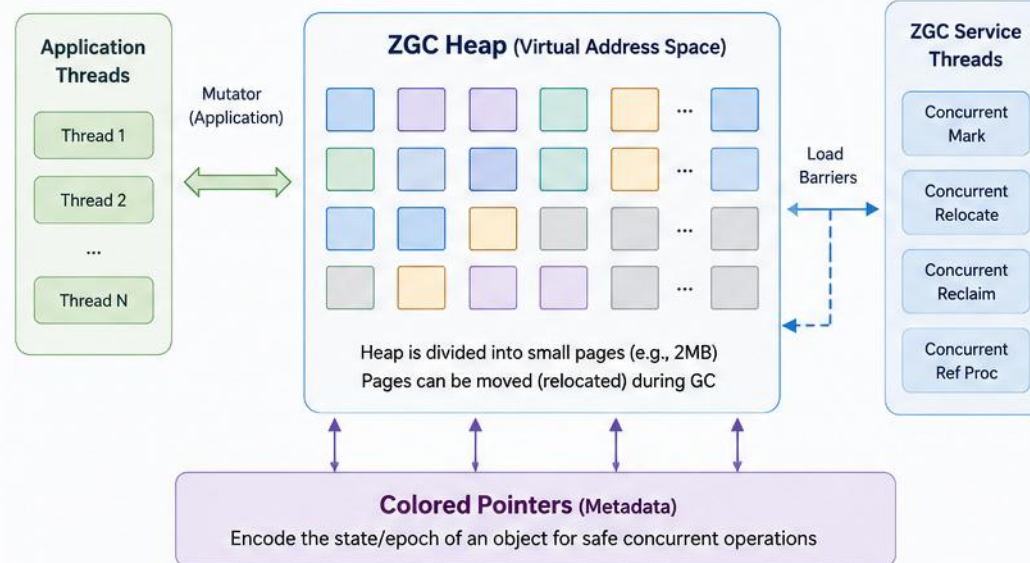
Uses load barriers and colored pointers to enable concurrent compaction.



### High Throughput

Built for production workloads that require both low latency and high throughput.

## ZGC Architecture (High Level)



## How ZGC Works



### 1 Concurrent Mark

Finds live objects concurrently while the application runs.



### 2 Concurrent Relocate

Live objects are copied to new locations concurrently.



### 3 Concurrent Reclaim

Unused memory is reclaimed concurrently.



### 4 Reference Processing

References are processed concurrently.



### 5 Repeat

The cycle repeats to keep latency low.

## ZGC Cycle (Simplified)



## Benefits



### Consistent Low Pauses

Keeps pause times predictable and short, even at large scales.



### Large Heap Support

Efficient for heaps from GBs to multiple TBs.



### Production Ready

Built for real-world applications with minimal tuning.



### Future Proof

Continuously improved with new JDK releases.



**In Short:** ZGC is a concurrent, scalable, and low-latency garbage collector that enables Java applications to handle large heaps with predictable performance and minimal pause times.



# GARBAGE COLLECTION BEST PRACTICES

*Good GC performance starts with good code, the right JVM configuration, and continuous monitoring.*

## THE GOAL



**Low Pause Times**



**High Throughput**



**Efficient Memory**



**Stable & Reliable**



**Lower Cost**



### Write Memory-Efficient Code

- Avoid leaks (unclosed resources, static collections).
- Minimize object creation in tight loops; reuse objects.
- Prefer primitives over objects in collections.



### Choose the Right GC

- <10ms latency → ZGC; JDK 8-14 → Shenandoah.
- Balanced → G1 (default, JDK 9+).
- Max throughput → Parallel; simple apps → Serial.

**KEY TAKEAWAY** For most applications on JDK 9+, G1 GC is a good general-purpose choice.

# GC BEST PRACTICES — SIZING, TUNING & ALLOCATION

*Four more practices that keep GC pauses short and predictable.*



## Size the Heap Correctly

- Too small → frequent GC, high CPU. Too large → long pauses.
- Consider -Xms = -Xmx for stable production performance.



## Tune GC Settings

- -XX:+UseG1GC -XX:MaxGCPauseMillis=200
- -XX:InitiatingHeapOccupancyPercent=45



## Reduce Allocation Rate

- Avoid short-lived objects in loops.
- Use object pools/caches; avoid unnecessary boxing.



## Avoid Memory Leaks

- Use try-with-resources; unregister listeners.
- Use WeakHashMap / WeakReference / SoftReference.

**KEY TAKEAWAY** Set realistic pause goals, adjust the initiating occupancy percent, and size GC thread counts to match your hardware.

# GC BEST PRACTICES — MONITOR, ANALYZE & AVOID

*Measure, analyze, tune, validate, repeat.*

- Monitor GC in Production: pause times (avg/max/p99), frequency, throughput, heap usage. Tools: jstat, jcmd/GC logs, VisualVM, Prometheus + Grafana.
- Analyze GC Logs: look for long pauses, Full GCs, high allocation rate, humongous allocations, promotion failures.

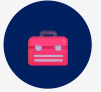
## COMMON GC ANTI-PATTERNS TO AVOID

- Creating too many objects unnecessarily; ignoring memory leaks; setting heap too large.
- Over-tuning without data — tune based on metrics, not guesswork; not monitoring in production.

**KEY TAKEAWAY** Measure → Analyze → Tune → Validate → Repeat. Good GC practices mean low latency, high throughput, and stable systems.

# GC BEST PRACTICES — MODERN APIS & LOAD TESTING

*Two more practices: write GC-friendlier code, and validate every change under real load.*



## Use Modern APIs & Patterns

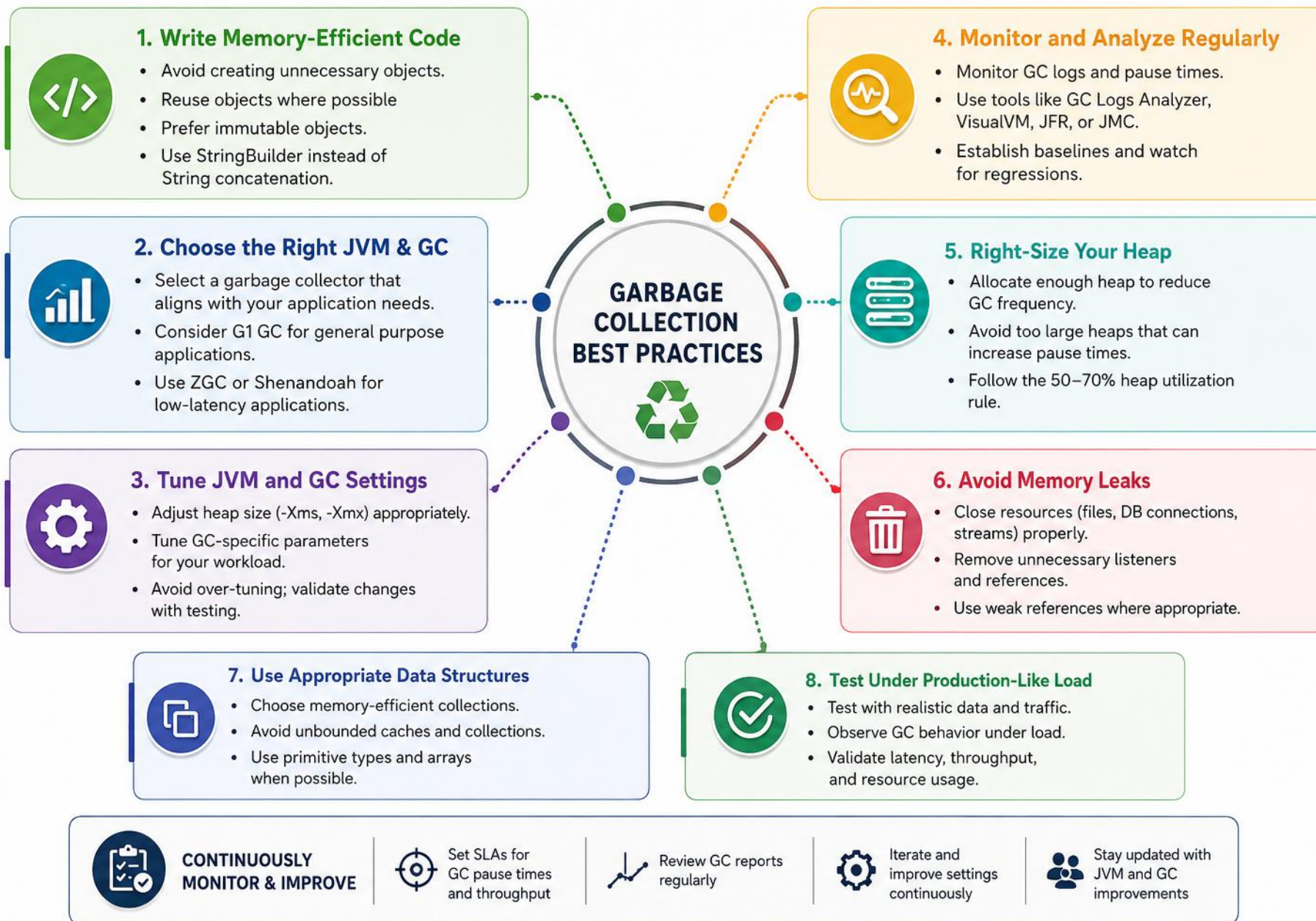
- Use try-with-resources; use streams carefully (avoid unneeded intermediate objects).
- Prefer immutable objects; size collections with a known initial capacity.
- Use record/value-based classes wisely.



## Test and Load Test

- GC behavior changes with workload — load test to find the right settings.
- Validate GC behavior after every change.
- Staging performance should match production.

**KEY TAKEAWAY** GC tuning is never "set and forget" — modern APIs reduce allocation pressure, and load testing proves your settings hold up.



# JVM STARTUP SEQUENCE

*From the java command to your main() method — a well-defined sequence of steps.*

- 1 Load JVM: the JVM process starts.
- 2 Initialize JVM internal components.
- 3 Load Bootstrap Class Loader (core classes).
- 4 Initialize subsystems: Threads, GC, JIT.
- 5 Parse command-line options (-Xms, -XX...).
- 6 Set up classpath and module path.
- 7 Load the main class via the Application Class Loader.
- 8 Verify bytecode & prepare for execution.
- 9 Resolve symbolic references of the main class.
- 10 Invoke main() and start program execution.

**KEY TAKEAWAY** Most startup time is spent in class loading, verification, and JIT warm-up — keep the classpath clean and lightweight.

# JVM STARTUP — USEFUL OPTIONS & TROUBLESHOOTING

*The flags and tools you'll reach for when startup misbehaves.*

| OPTION                                           | PURPOSE                      |
|--------------------------------------------------|------------------------------|
| <code>-Xms&lt;size&gt; / -Xmx&lt;size&gt;</code> | Initial / maximum heap size  |
| <code>-XX:+UseG1GC</code>                        | Use the G1 Garbage Collector |
| <code>-verbose:class</code>                      | Print class-loading info     |
| <code>-Dkey=value</code>                         | Set a system property        |

- Use `-verbose:class` to see class loading; `-Xlog:gc*` (JDK 9+) or `-XX:+PrintGCDetails` for GC logs.
- Use `-XX:+PrintCompilation` to see JIT activity; check classpath/module path if the main class isn't found.

**KEY TAKEAWAY** Command-line options and classpath control startup behavior — bytecode verification ensures safety before `main()` ever runs.

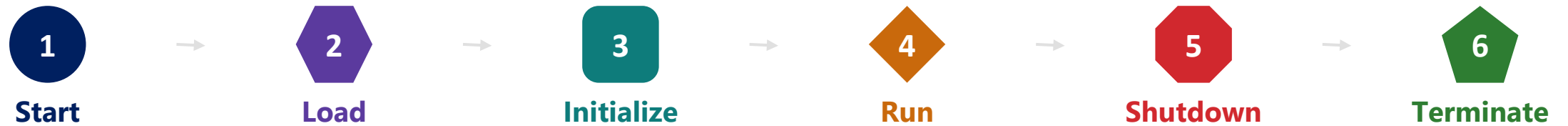
# JVM STARTUP SEQUENCE

Step-by-Step Flow from java Command to First Execution



# APPLICATION LIFECYCLE INSIDE JVM

*From launch to termination — the JVM loads, runs, manages, and finally terminates the application.*



- Start: OS creates the process; JVM is loaded and performs basic setup.
- Load: Class Loaders load required .class files into the Method Area/Metaspace.
- Initialize: Verification, preparation, resolution, and static initialization run.
- Run: main() executes; objects are created, threads run, GC runs in the background.
- Shutdown: shutdown hooks execute, non-daemon threads finish, resources are released.
- Terminate: JVM stops, all memory returns to the OS, process exits with a status code.

**KEY TAKEAWAY** Shutdown hooks allow graceful cleanup before the JVM process exits.

# RUNTIME DATA AREAS & APPLICATION STATES

*Which memory area is active during each phase of the lifecycle.*

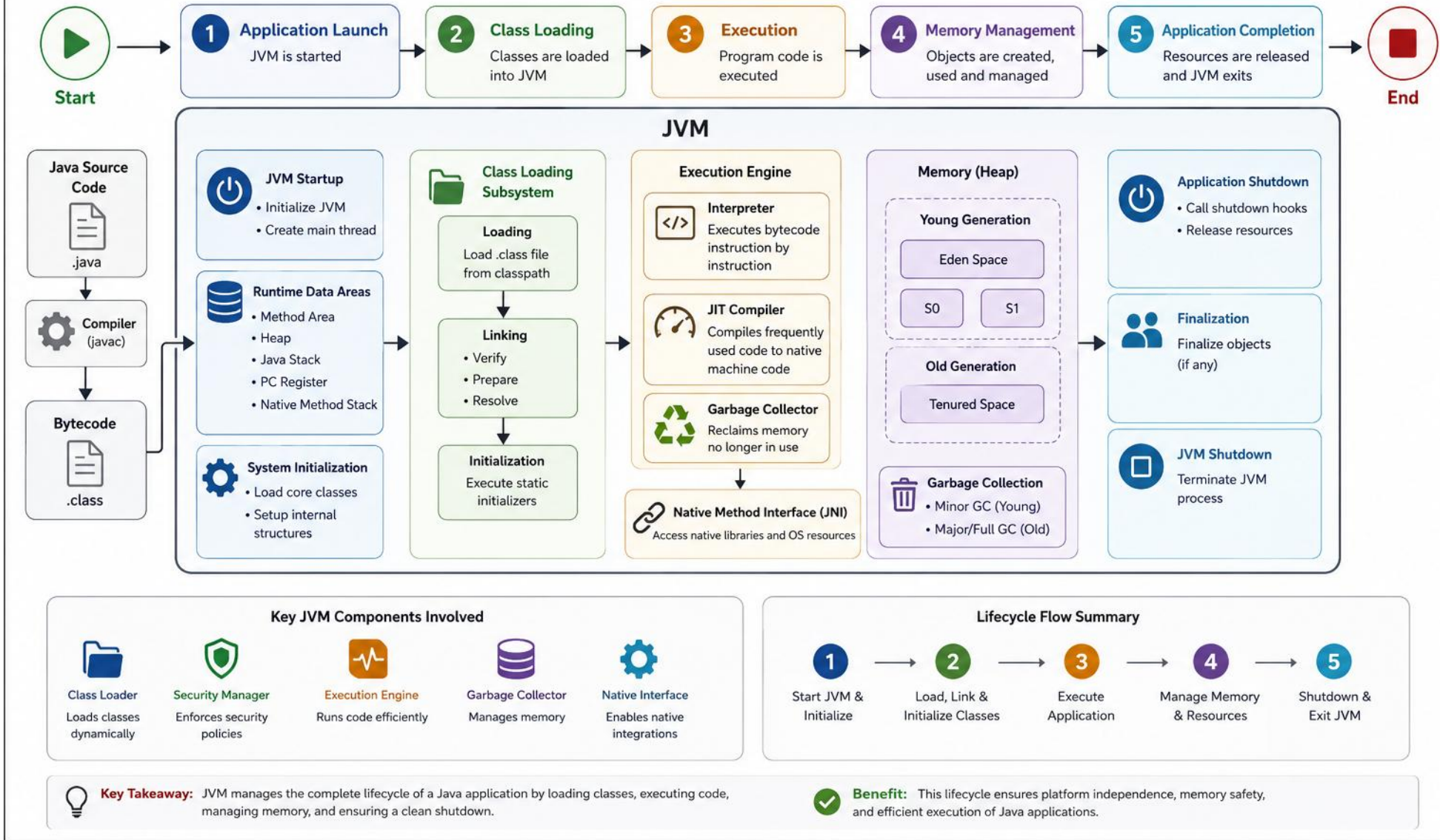
| AREA                    | USED IN          | PURPOSE                                                |
|-------------------------|------------------|--------------------------------------------------------|
| Method Area (Metaspace) | Load, Initialize | Class metadata, constant pool, method info.            |
| Heap                    | Run              | Stores objects and arrays.                             |
| Java Stacks             | Run              | Method call frames and local variables.                |
| Code Cache              | Run              | Stores JIT-compiled code.                              |
| PC Register             | Run              | Stores the address of the next instruction per thread. |
| Native Method Stack     | Run              | Used by native (JNI) methods.                          |

- Application States: New → Loading → Linking → Initialized → Running → Shutting Down → Terminated.

**KEY TAKEAWAY** Use jcmd, jmap, jstack, VisualVM, or JFR to monitor the JVM during any of these lifecycle phases.

# Application Lifecycle Inside JVM

Journey of a Java Application from Start to Finish



# COMMON JVM PERFORMANCE PROBLEMS

*Symptoms, causes, impact, and how to fix — issues come from code, memory, GC, threads, I/O or configuration.*



## CPU & Threads

Tight loops, contention, thread explosion.



## Memory & GC

Leaks, excessive collection, OOM.



## I/O & Disk

Blocking calls, slow queries.



## Application Code

Inefficient algorithms, logging.



## JVM Configuration

Wrong heap/GC/thread settings.

- Identify the symptom, find the root cause, and apply the right fix — never tune blindly.
- The next slide walks through ten specific problems, from high CPU usage to poor JVM configuration.

**KEY TAKEAWAY** Most JVM performance problems trace back to memory management, garbage collection, thread usage, I/O, or inefficient code.

# TEN COMMON PERFORMANCE PROBLEMS (1-5)

*Symptom, cause, and fix for the issues you'll encounter most often in production.*

| PROBLEM           | SYMPTOMS                             | CAUSES                                 | IMPACT                          | HOW TO FIX                             |
|-------------------|--------------------------------------|----------------------------------------|---------------------------------|----------------------------------------|
| High CPU Usage    | CPU near 100%, slow response         | Tight loops, excessive object creation | High latency, poor throughput   | Profile with jstack/JFR; optimize code |
| Excessive GC      | Frequent/long GC pauses              | Too many short-lived objects           | High pause time, low throughput | Reduce allocations; right-size heap    |
| High Memory / OOM | Gradual growth, OOM crashes          | Memory leak, uncleared caches          | OOM, crashes, service downtime  | Heap dump + MAT; fix the leak          |
| Thread Contention | Many WAITING/BLOCKED threads         | Synchronization, I/O in locks          | Poor scalability, high latency  | Analyze jstack; reduce lock scope      |
| Too Many Threads  | High thread count, context switching | Thread-per-request, unbounded pools    | High overhead, low throughput   | Use bounded thread pools, async I/O    |

**KEY TAKEAWAY** Five more common performance problems continue on the next slide.

# TEN COMMON PERFORMANCE PROBLEMS (6-10)

*Symptom, cause, and fix for the issues you'll encounter most often in production.*

| PROBLEM              | SYMPTOMS                         | CAUSES                             | IMPACT                             | HOW TO FIX                         |
|----------------------|----------------------------------|------------------------------------|------------------------------------|------------------------------------|
| Slow I/O             | High I/O wait                    | Blocking I/O, slow queries         | Increased latency, low throughput  | Optimize queries, use async I/O    |
| Class Loading Issues | High load time, Metaspace growth | Dynamic loading, classloader leaks | Startup delays, higher memory      | Fix leaks, proper delegation       |
| Inefficient Code     | High CPU/memory, poor throughput | Bad algorithms, excess logging     | High resource use, slow app        | Profile & optimize hot paths       |
| Large Heap / Long GC | Long Full GC pauses              | Oversized heap, wrong GC tuning    | Long pauses, timeouts              | -Xms=-Xmx; tune G1 region/IHOP     |
| Poor JVM Config      | Unstable performance             | Wrong heap/GC/thread settings      | Resource waste, unpredictable perf | Review/tune options with real data |

**KEY TAKEAWAY** Tools to identify these: jstat, jstack, jmap, JFR, VisualVM, and GC logs.

# PERFORMANCE PROBLEMS — GENERAL BEST PRACTICES

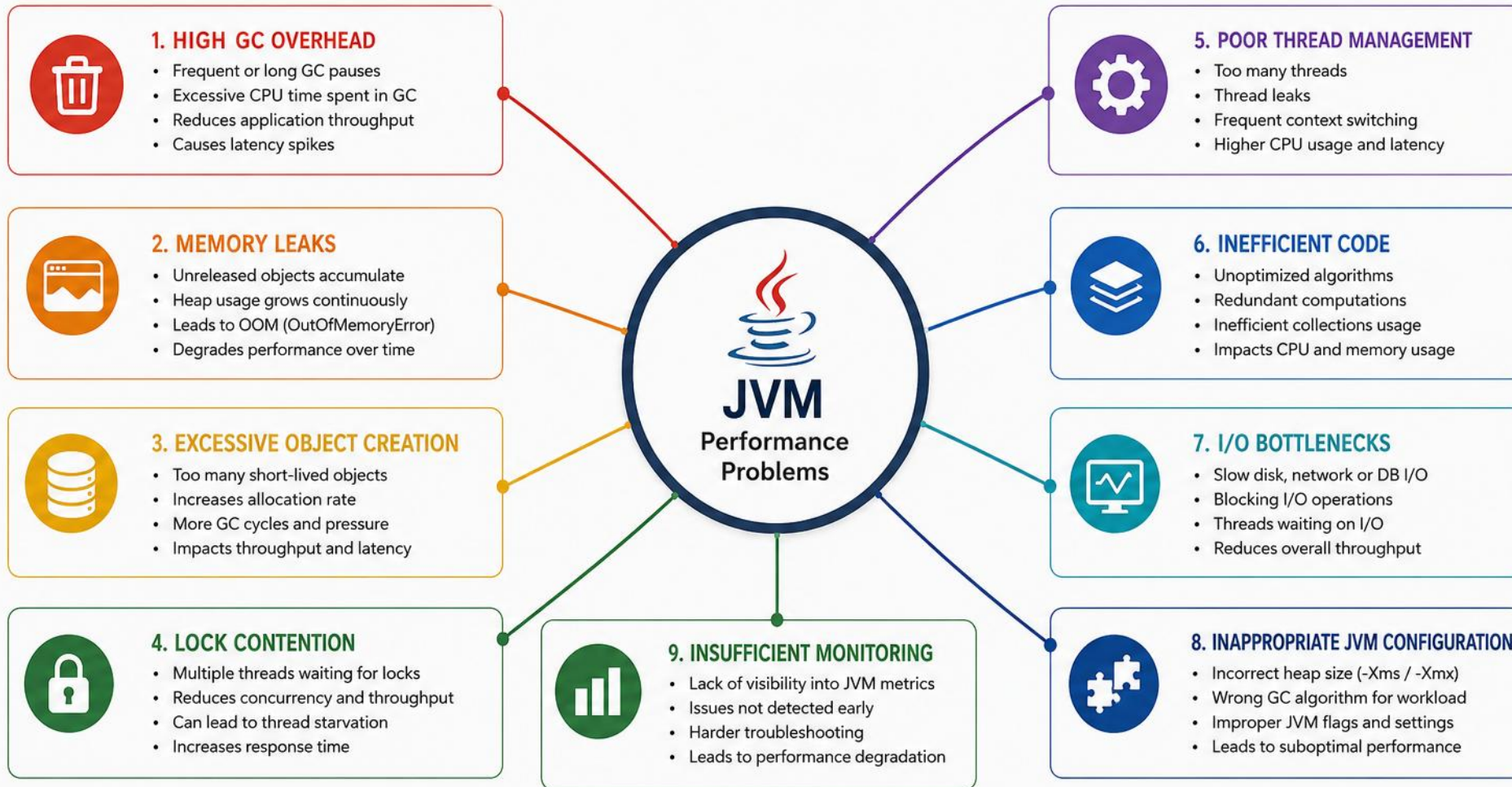
*The discipline that catches most issues before they reach production.*

- Monitor in production continuously; set alerts on CPU, memory, GC, and threads.
- Understand your workload before you tune it.
- Profile before optimizing — don't guess.
- Change one thing at a time, and test under real load.

**KEY TAKEAWAY** Measure → Analyze → Fix → Verify → Monitor — the same cycle applies to every category of performance problem.

# COMMON JVM PERFORMANCE PROBLEMS

Key issues that impact JVM performance in production systems



**IMPACT:**



Lower Throughput



Higher Latency



Increased Resource Usage



Poor User Experience

# OUTOFMEMORYERROR & MEMORY LEAKS

*OOME occurs when the JVM cannot allocate memory; leaks happen when unneeded objects stay strongly referenced.*

- JVM memory areas that can run out: Heap (objects), Metaspace (class metadata), Stack (threads), Code Cache (JIT), Direct Memory (NIO/off-heap).

| MEMORY ERROR TYPE              | OCCURS WHEN                         | TYPICAL CAUSE                          |
|--------------------------------|-------------------------------------|----------------------------------------|
| Java heap space                | Heap memory full                    | Too many objects, memory leaks         |
| Metaspace                      | Metaspace full                      | Too many classes, classloader leak     |
| StackOverflowError             | Thread stack full                   | Deep recursion, too many calls         |
| Unable to create native thread | OS cannot create threads            | Too many threads, OS limits            |
| GC overhead limit exceeded     | >98% time in GC, <2% heap recovered | Memory leak, or heap too small         |
| Direct buffer memory           | Direct/off-heap memory full         | NIO direct buffers, Netty/Unsafe usage |

**KEY TAKEAWAY** Different OOME types point to different exhausted memory areas — the error text tells you where to look first.

# MEMORY LEAKS — CAUSES & DIAGNOSIS

*Objects that are no longer needed but remain strongly reachable cause gradual memory growth.*

## COMMON CAUSES

- Collections never cleared; unbounded caches with no eviction policy.
- Listeners/callbacks registered but never removed.
- ThreadLocal misuse, especially in thread pools.
- ClassLoader leaks and unnecessary static references.

## HOW TO DIAGNOSE

- 1 Monitor heap usage, GC activity, thread count.
- 2 Take a heap dump: `jmap -dump:live,format=b,...`
- 3 Analyze with Eclipse MAT / VisualVM / YourKit.
- 4 Check GC logs for frequent GC and long pauses.

**KEY TAKEAWAY** Quick check: memory keeps rising, Full GC runs but memory doesn't drop, heap dump shows many objects that should be dead — likely a leak.

# HOW TO PREVENT MEMORY LEAKS

*Design for bounded memory, and clean up aggressively.*

- Design for bounded memory usage: limit caches, queues, and collections.
- Always clear collections and remove listeners when no longer needed.
- Use weak/soft references for caches where appropriate; close resources with try-with-resources.
- Use ThreadLocal.remove(), especially in thread pools; avoid unnecessary static/global references.
- Monitor production and set alerts on memory usage and GC; right-size the heap with -Xms/-Xmx.

**KEY TAKEAWAY** Monitor, analyze, and fix the root cause before increasing heap size — good design plus cleanup equals stable, scalable applications.

# Out of Memory Error and Memory Leaks

Understanding the causes, behavior and prevention strategies

## 1. OUT OF MEMORY ERROR (OOM)

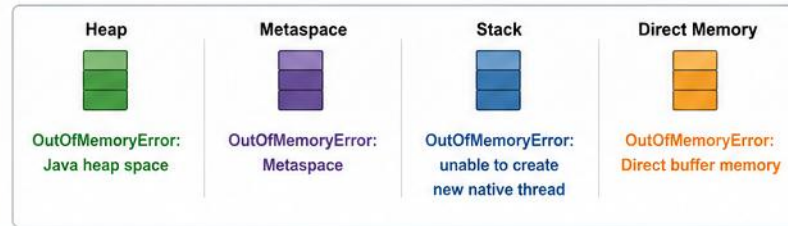
Occurs when the JVM cannot allocate enough memory for an object or processing.



**java.lang.OutOfMemoryError**

Application crashes or becomes unstable

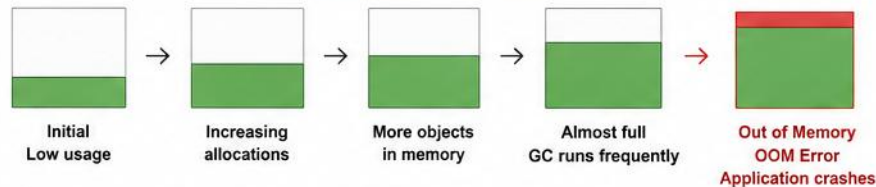
### Where OOM Happens



### Typical OOM Example (Heap)

```
List<int[]> list = new ArrayList<>();
while (true) {
    list.add(new int[1024 * 1024]); // 1 MB
}
// Eventually throws:
// java.lang.OutOfMemoryError: Java heap space
```

### Heap Exhaustion Over Time



### How to Prevent OOM

- ✓ **Right-size the heap** (-Xmx and -Xms)  
Allocate appropriate memory for the application.
- ✓ **Optimize memory usage**  
Avoid creating unnecessary objects and large collections.
- ✓ **Use efficient data structures & caching**  
Prefer streaming, paging, and lazy loading.
- ✓ **Monitor memory**  
Use tools like VisualVM, JFR, GC logs to identify issues.
- ✓ **Release resources**  
Close streams, connections, and other resources.
- ✓ **Avoid memory fragmentation**  
Tune GC settings for better performance.
- ✓ **Load test your application**  
Simulate real workload to find memory limits early.

## 2. MEMORY LEAKS

Occurs when objects that are **no longer needed** are still **referenced (directly or indirectly)** so GC cannot reclaim memory.



### Memory Leak Impact

- Continuous memory growth
- Performance degradation
- Eventually leads to OOM

### Common Causes of Memory Leaks



#### Static Collections

Objects added to static collections are never released.

```
static List<User> users = new ArrayList<>();
users.add(new User(...));
```



#### Unreleased Event Listeners / Callbacks

Listeners registered but never unregistered keep references.

```
button.addActionListener(this);
// forgot to removeActionListener(this);
```



#### Improper Cache Usage

Cache grows without bounds or entries are never evicted.

```
Map<String, Object> cache = new HashMap<>();
cache.put(key, value); // keeps growing
```



#### Long-Lived Threads

Threads holding references prevent objects from being GC'ed.

```
new Thread(() -> {
    while(true) { /* holds reference */ }
}).start();
```



#### Unclosed Resources

Files, DB connections, streams not closed retain memory.

```
InputStream is = new FileInputStream(file);
// not closed
```

### How to Prevent Memory Leaks

- ✓ **Remove references when done**  
Set references to **null** or remove from collections.
- ✓ **Unregister listeners / callbacks**  
Always remove listeners in **finally** blocks or close methods.
- ✓ **Use bounded caches**  
Use size-limited caches with expiration (e.g., Caffeine, Guava Cache).
- ✓ **Close resources properly**  
Use **try-with-resources** for streams, connections, and sockets.
- ✓ **Monitor & analyze**  
Use Heap Dump, MAT, VisualVM to detect and fix leaks.

### Tools to Detect & Diagnose



GC Logs



VisualVM



JFR (Java Flight Recorder)



Heap Dump



MAT (Memory Analyzer Tool)



JMC (Java Mission Control)

# JVM MONITORING BASICS

*Observe – Understand – Optimize. Monitoring helps you diagnose issues and optimize performance in production.*



**App Health**



**Better  
Performance**



**Faster  
Diagnosis**



**Capacity  
Planning**



**Prevent  
Outages**

## WHAT CAN WE MONITOR?

- CPU Usage, Memory Usage (Heap, Non-Heap, Metaspace, Direct Memory).
- Garbage Collection activity, pause time, and frequency.
- Threads: live, blocked, and deadlocks; Class Loading: loaded/unloaded classes.
- Application Metrics: throughput, latency, errors, custom metrics.
- I/O & Network: disk I/O, network throughput.

**KEY TAKEAWAY** Consistent monitoring gives visibility into JVM behavior and detects problems before they become outages.

# JDK MONITORING TOOLS (BUILT-IN)

*Seven tools that ship with every JDK, no extra installation required.*

| TOOL     | USE                                   | EXAMPLE                                        |
|----------|---------------------------------------|------------------------------------------------|
| jps      | List running JVM processes            | jps -l                                         |
| jstat    | GC, memory, class statistics          | jstat -gcutil <pid> 5s                         |
| jmap     | Heap dump, memory maps                | jmap -dump:live,format=b,file=heap.hprof <pid> |
| jstack   | Thread dump (stack traces)            | jstack <pid> > thread.txt                      |
| jcmd     | Multipurpose diagnostics (JDK 9+)     | jcmd <pid> GC.heap_info                        |
| jinfo    | JVM configuration & system properties | jinfo -flags <pid>                             |
| VisualVM | Visual monitoring/profiling           | visualvm                                       |

**KEY TAKEAWAY** Run these tools as the same user that owns the JVM process you're inspecting.

# KEY JVM METRICS & WHAT THEY TELL YOU

*Ten signals worth watching, and what a bad value means.*

| METRIC             | WHAT IT INDICATES                     | WATCH FOR                     |
|--------------------|---------------------------------------|-------------------------------|
| CPU Usage          | How much CPU the JVM process consumes | Sustained high usage          |
| Heap Used (%)      | How much heap memory is in use        | Consistently >70-80%          |
| GC Pause Time      | Time JVM pauses for GC                | Long pauses, high frequency   |
| GC Count           | How often GC runs                     | Rapidly increasing count      |
| Threads (Live)     | Number of live threads                | Steadily climbing count       |
| Threads (Blocked)  | Threads waiting for locks/resources   | High blocked count            |
| Class Loading Rate | Classes loaded per second             | Unusually high rate           |
| Metaspace Used     | Native memory for class metadata      | Steady growth = possible leak |
| Latency p95/p99    | Request response time                 | Increasing latency            |
| Throughput / QPS   | Requests handled per second           | Sudden drops                  |

**KEY TAKEAWAY** Baseline normal behavior, watch for deviations, and correlate JVM metrics with application metrics.

# MONITORING ARCHITECTURE

*How JVM metrics travel from a running process to a dashboard someone actually looks at.*



**KEY TAKEAWAY** Metrics collectors (JMX, agents) feed a monitoring system (Prometheus, Datadog), which powers dashboards and alerts.

# JVM MONITORING BASICS

Monitor JVM to ensure performance, stability and optimal resource usage

## What is JVM Monitoring?



JVM Monitoring is the process of collecting and analyzing data from the JVM to understand its health, performance and resource usage.

## Why Monitor JVM?

- ✓ Detect performance bottlenecks
- ✓ Prevent OutOfMemoryError
- ✓ Optimize resource utilization
- ✓ Improve application stability
- ✓ Understand behavior under load
- ✓ Faster issue detection & resolution

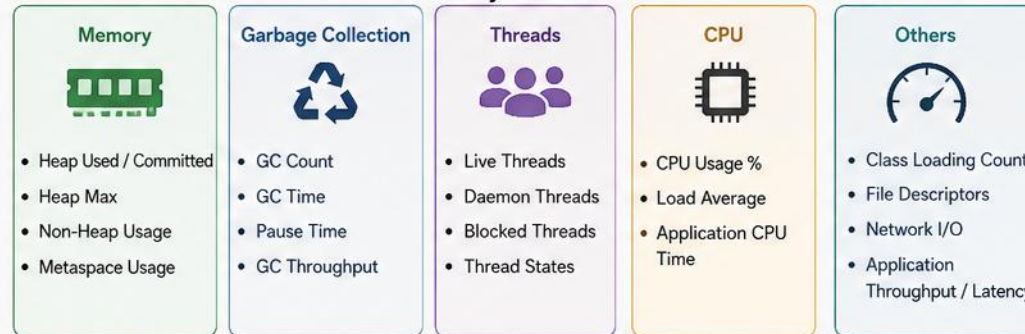
## Key Areas to Monitor

- 🧠 Memory Usage (Heap & Non-Heap)
- ♻️ Garbage Collection (GC)
- 👤 Thread Activity
- 🕒 CPU Usage
- 📄 Class Loading
- 🌐 I/O & Network
- 📊 Application Throughput & Latency
- 🖥️ System Resources (OS)

## JVM PROCESS



## Key Metrics



## Common Tools



**JConsole (Built-in)**  
GUI tool included in JDK for basic monitoring.



**VisualVM (Free)**  
Powerful visual tool for monitoring & profiling.



**JMC (Java Mission Control)**  
Advanced monitoring, alerts & diagnostics.



**Prometheus + Grafana**  
Open-source monitoring & visualization stack.

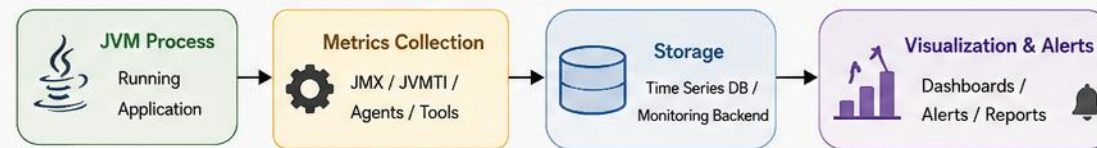


**Datadog / New Relic / Dynatrace**  
APM tools for enterprise monitoring.



**Command Line Tools**  
jstat, jmap, jstack, jcmd, jinfo, jps

## Monitoring Data Flow



## Best Practices

- ☑ Monitor in all environments (Dev, Test, Prod)
- ☑ Set baseline metrics & thresholds
- 🔔 Use alerts for early detection
- 📅 Regularly review GC & memory trends
- ☑ Correlate JVM metrics with application & OS metrics

# INSTRUCTOR DEMO: COMPILATION AND BYTECODE

*From Java source code to JVM instructions — a live walkthrough.*



- Write Java source with a `main()` method and a helper `add()` method.
- Compile: `javac` generates `Hello.class` — syntax checked, semantics analyzed, bytecode generated.
- Run: Class Loader loads `Hello.class`, Bytecode Verifier checks safety, Execution Engine runs it, JIT optimizes hot code.
- Platform independence: the same `Hello.class` runs on Windows, Linux, and macOS JVMs and produces the same output.

**KEY TAKEAWAY** `javac` compiles `.java` → `.class`; the JVM loads, verifies, and executes that bytecode on any platform.

# COMPILATION DEMO — COMMANDS

*The exact commands used in the walkthrough.*

| COMMAND                              | PURPOSE                                      |
|--------------------------------------|----------------------------------------------|
| <code>javac Hello.java</code>        | Compile Java source file                     |
| <code>javac -d out Hello.java</code> | Compile and place .class in out/             |
| <code>javac *.java</code>            | Compile all .java files in current directory |
| <code>javap -c Hello</code>          | Disassemble bytecode in human-readable form  |
| <code>java Hello</code>              | Run compiled program                         |
| <code>java -cp .:lib/* Hello</code>  | Run with a custom classpath                  |

```
$ javac Hello.java
$ javap -c Hello
$ java Hello // prints program output
$ javap -c -v Hello // verbose: shows the constant pool too
```

**KEY TAKEAWAY** Compile with javac, inspect with javap -c, then run with java and compare output.

# COMPILATION DEMO — BYTECODE ANATOMY

*What a few core instructions mean.*

| INSTRUCTION                     | MEANING                           | EXAMPLE                       |
|---------------------------------|-----------------------------------|-------------------------------|
| <code>bipush 10</code>          | Push byte constant 10 onto stack  | <code>bipush 10</code>        |
| <code>istore_1 / iload_1</code> | Store / load int with local var 1 | <code>istore_1</code>         |
| <code>getstatic #3</code>       | Get a static field                | <code>getstatic #3</code>     |
| <code>invokestatic #2</code>    | Call a static method              | <code>invokestatic #2</code>  |
| <code>invokevirtual #5</code>   | Call an instance method           | <code>invokevirtual #5</code> |
| <code>iadd / istore_1</code>    | Add two ints / store as int       | <code>iadd</code>             |

```
0: bipush 10      // push 10
2: istore_1       // store into local var 1
3: iload_1 ... iadd // load + add
```

Modern javac compiles string concatenation to `invokedynamic / makeConcatWithConstants` — not repeated `StringBuilder` calls.

# TRY IT YOURSELF — EXERCISE 8: INSPECT BYTECODE

*Reinforces: the bytecode anatomy you just walked through.*

| STEP        | WHAT TO DO                                                                        |
|-------------|-----------------------------------------------------------------------------------|
| Goal        | Disassemble Person (or Hello) with javap and note three bytecode instructions     |
| File        | Person.class (already compiled in Exercise 7)                                     |
| Command     | javap -c Person                                                                   |
| Look for    | aload_0, putfield, invokespecial in the constructor; getfield in display()        |
| Key concept | javac compiles once; javap only shows the bytecode; java is what actually runs it |
| Reference   | exercises/exercise-08-javap.md                                                    |

```
$ cd examples/module-01-exercises
$ javap -c Person
$ javap -c -v Person // verbose: also shows the constant pool
$ java Person // confirms the disassembled bytecode is what actually executed
```

**TRY IT NOW** Complete Exercise 8 before continuing — see exercises/exercise-08-javap.md.

# INSTRUCTOR DEMO: MEMORY ALLOCATION

*See how objects are created and stored in JVM memory, step by step.*

- 1 JVM loads class HelloMemory; metadata goes to the Method Area.
- 2 main() starts; a Stack frame is created.
- 3 new HelloMemory(101, "Alice") is executed.
- 4 JVM allocates memory for the object on the Heap.
- 5 Reference obj is stored in the Stack frame.
- 6 Object fields id and name are stored in the Heap.

**KEY TAKEAWAY** The Stack holds references. The Heap holds actual objects.

# MEMORY AREAS USED & END OF MAIN()

*What happens to the object once its method frame is gone.*

| AREA                | USED FOR                                  | IN THIS EXAMPLE                              |
|---------------------|-------------------------------------------|----------------------------------------------|
| Method Area         | Class metadata, static variables          | Stores HelloMemory class info                |
| Heap                | Objects and arrays                        | Stores HelloMemory object and String "Alice" |
| Java Stack          | Method calls, local variables, references | Stores reference obj in main() frame         |
| PC Register         | Address of the next instruction           | Not shown directly in this example           |
| Native Method Stack | Used by native (JNI) methods              | Not used in this example                     |

## LIVE MEMORY VIEW (jvisualvm) — HEAP HISTOGRAM

| CLASS            | INSTANCES | SIZE |
|------------------|-----------|------|
| HelloMemory      | 1         | 24 B |
| java.lang.String | 1         | 24 B |
| char[]           | 6         | 32 B |
| Total            | 8         | 80 B |

- When main() completes, its Stack frame is popped and the reference obj is gone.
- If no other references exist, the object on the Heap becomes unreachable and the GC reclaims it later.

**KEY TAKEAWAY** Hands-on idea: create objects in a loop and watch Heap usage grow in VisualVM, then set obj = null and observe reclamation.

# LAB OVERVIEW — JVM AND COMPILATION

*Build HelloWorld, Calculator, Employee and MemoryDemo — the Northstar onboarding checklist proving you understand how the JVM compiles, loads, and executes Java code.*



- Objectives: write and compile HelloWorld, Calculator, Employee and MemoryDemo; understand compilation; inspect bytecode with javap.
- Run Java programs on the JVM, understand class loading and execution flow, verify platform independence.
- Prerequisites: Lab 0 complete, JDK 21 installed (javac, java, javap), VS Code or IntelliJ IDEA.

**KEY TAKEAWAY** Lab 1 builds JVM literacy before Spring Boot, Maven, or the Customer Management Platform: source → bytecode → JVM load → execute.

# PRE-LAB EXERCISES & SUCCESS CRITERIA

*Eight pre-lab exercises to complete before Lab 1, from Hello World to inspecting bytecode.*

| # | EXERCISE                     | FOCUS AREA                       | KEY COMMANDS                                          |
|---|------------------------------|----------------------------------|-------------------------------------------------------|
| 1 | Hello World                  | Basic program & execution        | <code>javac Hello.java; java Hello</code>             |
| 2 | Platform Independence (WORA) | Run bytecode without recompiling | <code>java Hello</code>                               |
| 3 | Control Flow                 | if, for, while, switch           | <code>javac ControlFlow.java; java ControlFlow</code> |
| 4 | Watch Class Loading          | Observe JVM class loading        | <code>java -verbose:class Hello</code>                |
| 5 | Variables and Data Types     | Local variables, types           | <code>javac Variables.java</code>                     |
| 6 | Methods and Parameters       | Method calls, stack usage        | <code>javac Methods.java</code>                       |
| 7 | Objects and Classes          | Object creation, fields          | <code>javac Person.java</code>                        |
| 8 | Inspect Bytecode             | Bytecode structure               | <code>javap -c Person</code>                          |

- Success criteria: compile without errors, inspect and explain bytecode, run programs and explain output, understand class loading and execution.

**KEY TAKEAWAY** Try `-Xint` and `-Xcomp` to compare interpreted vs. compiled execution, and `-verbose:class` to watch class loading live.

# LAB 1 TASKS AND DELIVERABLES

*Write, compile, inspect, run, analyze, and submit — building HelloWorld, Calculator, Employee, and MemoryDemo.*

| # | TASK                     | DESCRIPTION                                                         | TOOLS                 |
|---|--------------------------|---------------------------------------------------------------------|-----------------------|
| 1 | Hello World              | Print "Hello, JVM!" and run it.                                     | javac; java           |
| 2 | Inspect Bytecode (javap) | Disassemble HelloWorld.class; read getstatic / ldc / invokevirtual. | javap -c              |
| 3 | Calculator               | add(10, 20) → Sum = 30; observe stack frames and iadd.              | javac; java; javap -c |
| 4 | Employee (Heap Objects)  | new Employee(101, "Aman"); map stack reference to heap object.      | javac; java           |
| 5 | Observe Class Loading    | Trace JDK and Employee classes loading with -verbose:class.         | java -verbose:class   |
| 6 | MemoryDemo               | Allocate 100,000 Employees; discuss heap pressure and -Xmx.         | javac; java           |
| 7 | Clean & Recompile        | Delete *.class, rebuild all four programs, re-verify output.        | javac (rebuild)       |

**KEY TAKEAWAY** Each task builds toward the Northstar onboarding checklist: four working programs, bytecode literacy, and a clear stack-vs-heap mental model.

# DELIVERABLES & EVALUATION CRITERIA

*What to submit, and how it's scored.*

- Deliverables: four source files (HelloWorld, Calculator, Employee, MemoryDemo), compile/run screenshots, javap -c bytecode capture, class-loading evidence, and seven short lab-report answers.

| EVALUATION CRITERIA                      | WEIGHT |
|------------------------------------------|--------|
| Environment Readiness                    | 10%    |
| Compile, Run & Inspect Bytecode          | 40%    |
| Heap Model & Class Loading               | 20%    |
| MemoryDemo, Heap Flags & Recompile       | 15%    |
| Evidence, Analysis & Failure Experiments | 15%    |

**KEY TAKEAWAY** Compile before running, fix all errors first, and capture clean evidence for all four programs before submitting.

# ANALYSIS QUESTIONS (ANSWER IN REPORT)

*Six questions every Lab 1 report must answer, tying HelloWorld, Calculator, Employee and MemoryDemo back to the concepts.*

- 1 What does javac do, and what file does it produce?
- 2 What is bytecode, and why is it not machine code?
- 3 Why is bytecode platform-independent?
- 4 What is the role of the JVM in executing your program?
- 5 Where are objects stored, and where are method calls and stack frames stored?
- 6 What happens when a class is loaded by the JVM?

**KEY TAKEAWAY** These questions map directly to the module's core topics — compilation, bytecode, class loading, and stack memory.

# MODULE 1 SUMMARY

*JVM, compilation, and program execution fundamentals — the full journey from code to output.*



- The JVM is the runtime engine that loads and executes Java bytecode, providing platform independence.
- `javac` compiles `.java` source into platform-independent `.class` bytecode.
- The JVM loads classes, verifies bytecode, manages memory, and executes code starting from `main()`.
- Local variables and method calls live on the Stack; objects are created in the Heap.

**KEY TAKEAWAY** Write → Compile → Run → Repeat. Bytecode is the bridge between your source code and the JVM.

# KEY CONCEPTS & WHAT YOU CAN DO NOW

*A one-page glossary, plus the skills you're walking away with.*

| CONCEPT               | MEANING                                     |
|-----------------------|---------------------------------------------|
| Source Code           | Human-readable .java files                  |
| Compilation           | javac converts .java into .class            |
| Bytecode              | .class files containing JVM instructions    |
| JVM Execution         | Loads, verifies, and executes bytecode      |
| Stack / Heap          | Method frames & locals / objects and arrays |
| Platform Independence | Same .class runs on any JVM                 |

- You can now write and compile Java programs confidently, inspect bytecode with javap, and explain how the JVM loads and executes your code.

**KEY TAKEAWAY** You've built a strong foundation for mastering the JVM — memory, GC, and performance are next.

# MODULE 1 KNOWLEDGE CHECK — MULTIPLE CHOICE (1–6)

*Choose the best answer. Correct answers are marked ✓.*

**1. What is the primary role of the JVM?**

- A) To edit and write Java source code   B) To compile Java code into machine code   **C) To load, verify and execute Java bytecode ✓**   D) To store Java source files

**2. Which command compiles a Java source file?**

- A) java Hello.java   **B) javac Hello.java ✓**   C) jvmc Hello.java   D) compile Hello.java

**3. What file is generated after successful compilation?**

- A) Hello.exe   **B) Hello.class ✓**   C) Hello.out   D) Hello.bin

**4. What is the format of JVM bytecode?**

- A) Platform-dependent machine code   B) Human-readable text   **C) Platform-independent binary instructions ✓**   D) High-level source code

**5. Which component loads the .class file into the JVM?**

- A) Execution Engine   **B) Class Loader ✓**   C) Bytecode Verifier   D) JIT Compiler

**6. Where are local variables and method call info stored?**

- A) Heap   B) Method Area   **C) Java Stack ✓**   D) PC Register

**KEY TAKEAWAY** Six more multiple-choice questions continue on the next slide.

# MODULE 1 KNOWLEDGE CHECK — MULTIPLE CHOICE (7–12)

*Choose the best answer. Correct answers are marked ✓.*

**7. Where are objects and arrays stored in the JVM?**

- A) Java Stack   **B) Heap ✓**   C) Method Area   D) PC Register

**8. What does the Bytecode Verifier do?**

- A) Optimizes bytecode for faster execution   B) Converts bytecode to machine code   **C) Ensures bytecode is safe and follows JVM rules ✓**   D) Manages memory allocation in heap

**9. From which method does the JVM start execution?**

- A) start()   **B) main(String[] args) ✓**   C) run()   D) execute()

**10. Which tool views bytecode instructions of a .class file?**

- A) java -cp   **B) javap ✓**   C) javac -d   D) jdb

**11. Which is true about the same .class file?**

- A) It runs only on the OS where compiled   B) It must be recompiled for each platform   **C) It can run on any platform with a JVM ✓**   D) It is executed directly by the OS

**12. Which component interprets or JIT compiles and executes bytecode?**

- A) Class Loader   **B) Execution Engine ✓**   C) Native Libraries   D) Java Stack

**KEY TAKEAWAY** 12 questions, testing the full span of this module — from JVM architecture to class loading to bytecode.

# MODULE 1 KNOWLEDGE CHECK — TRUE/FALSE & FILL IN THE BLANKS

*Sections B and C, with answers shown.*

## SECTION B — TRUE OR FALSE

1. Java source code (.java) is human-readable. **True ✓**
2. Bytecode is platform-dependent. **False ✓**
3. The JVM uses the Class Loader to load .class files. **True ✓**
4. Method calls and local variables are stored on the stack. **True ✓**
5. The same .class file can run on Windows and Linux with a JVM. **True ✓**

## SECTION C — FILL IN THE BLANKS

1. Converting .java to .class is called → **Compilation**
2. The area in JVM where objects are stored is called → **Heap**
3. The JVM component that verifies bytecode for safety is the → **Bytecode Verifier**
4. The command to disassemble a .class file is → **javap**
5. Local variables are stored in the → **Stack area**

**KEY TAKEAWAY** Bonus question: why is Java bytecode called platform-independent? Because the same .class file runs on any JVM, regardless of OS.

# MODULE 1 KEY TAKEAWAYS (PART 1)

*You explored how Java code is compiled to bytecode and executed by the JVM.*



- JVM is the Runtime Engine: loads, verifies and executes Java bytecode, providing platform independence.
- Compilation: Source to Bytecode — javac converts .java into platform-neutral .class bytecode.
- JVM Architecture: Class Loader, Bytecode Verifier, Execution Engine, Runtime Data Areas, JNI, Native Libraries.
- Class Loading Process: Loading → Linking → Initialization, via the Delegation Model.
- Memory Management: Heap for objects, Stack for method frames, local variables and references.

**KEY TAKEAWAY** Execution starts from main(): JVM looks for public static void main(String[] args) as the entry point.

# MODULE 1 KEY TAKEAWAYS (PART 2)

*Why these fundamentals matter for everything that comes next.*

- Bytecode Verification Ensures Safety: the Verifier checks bytecode follows JVM rules, preventing unsafe or malicious operations.
- Key Tools: javac (compiler), javap (disassembler), java (JVM launcher).
- Same .class, Different Platforms: once compiled, the same bytecode runs on Windows, Linux, macOS, and more.
- Stack Stores Method Execution Data: each call creates a frame holding local variables, operand stack, and return info.
- Heap Stores Objects and Arrays: created with new, and reclaimed automatically by the Garbage Collector.



## Performance

Tune with JVM  
internals knowledge.



## Debugging

Class loading, stack,  
heap, bytecode.



## Security

Verification protects  
unsafe code.



## Portability

Platform  
independence,  
everywhere.



## Foundation

Base for memory, GC,  
concurrency.

**KEY TAKEAWAY** Write Java code in .java files. Compile with javac. The JVM loads, verifies and executes bytecode. Understand the flow to build better, faster applications.

# Great work

You've built a strong foundation for mastering the JVM — how Java code compiles, loads, and runs.

Memory management and performance tuning are next in your toolkit.